



**Unidade Curricular de Bases de Dados
2025/2026**

**ENEI-Gest: Sistema Integrado de Gestão do Encontro
Nacional de Estudantes de Informática**

**João Taveira, João Neiva, Miguel Gonçalves, João Durães
a108559 a108579 a108478 a109065**

Abril, 2026

Resumo

Neste projeto, desenvolvemos um sistema de gestão de bases de dados relacionais criado para apoiar a organização do ENEI, que se vai realizar na cidade de Braga. O nosso objetivo central foi desenhar uma solução capaz de simplificar e centralizar a logística de um evento tecnológico de grande dimensão. Para isso, o sistema foca-se na gestão eficiente das inscrições e emissão de bilhetes, na alocação de espaços físicos para as várias sessões, e no controlo organizado de oradores e patrocinadores.

O nosso trabalho foi estruturado em várias fases fundamentais: definição do sistema, levantamento e análise de requisitos, modelação conceptual, modelação lógica e, por fim, a implementação física. A validação rigorosa que realizámos ao longo destas etapas foi essencial para garantir a integridade dos dados e assegurar que construímos um sistema robusto, capaz de responder com precisão às pesquisas e necessidades reais dos utilizadores.

Para transformar esta ideia em realidade, aplicámos na prática as ferramentas e metodologias que consolidamos nas aulas da unidade curricular. Recorremos ao "brModelo" para criar a modelação conceptual, o que nos permitiu visualizar de forma clara as entidades e as suas relações. Posteriormente, utilizamos o "MySQL Workbench" para desenhar o modelo lógico e dar vida à base de dados, criando toda a estrutura de tabelas e inserindo os dados iniciais. Esta abordagem garantiu-nos uma transição segura e funcional do papel para o ambiente real do servidor de base de dados.

Área de Aplicação: Análise, planeamento, modelação, arquitetura e implementação de um sistema de gestão de bases de dados.

Palavras-Chave: ENEI, Gestão de Eventos, Inscrições, brModelo, MySQL Workbench, Modelação Conceptual, Modelação Lógica, Implementação Física.

Índice

Resumo	1
1 Definição do Sistema	8
1.1 Contexto de aplicação	8
1.2 Motivação e Objetivos do Trabalho	8
1.3 Análise da Viabilidade do processo	9
1.4 Recursos e Equipa de Trabalho	10
1.5 Plano de Execução do Projeto	10
2 Levantamento e Análise de Requisitos	12
2.1 Método de Levantamento e de Análise de Requisitos Adotado	12
2.2 Organização dos Requisitos Levantados	12
Requisitos de descrição	13
Requisitos de manipulação	14
Requisitos de controlo	14
2.3 Análise e Validação Geral dos Requisitos	14
3 Modelação Conceptual	16
3.1 Apresentação da Abordagem de Modelação Realizada	16
3.2 Identificação e Caracterização das Entidades	16
3.3 Identificação e Caracterização dos Relacionamentos	18
3.4 Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos	26
3.5 Apresentação e Explicação do Diagrama ER Produzido	30
4 Modelação Lógica	32
4.1 Construção e Validação do Modelo de Dados Lógico	32
4.2 Apresentação e Explicação do Modelo Lógico Produzido	32
4.3 Normalização de Dados	36
4.4 Validação do Modelo com Interrogações do Utilizador	37
4.5 Criação de utilizadores da base de dados	46
4.6 Povoamento da base de dados	47
4.7 Cálculo do espaço da base de dados (inicial e taxa de crescimento anual)	50
4.8 Definição e caracterização de vistas de utilização em SQL	53
4.9 Tradução das interrogações do utilizador para SQL	55
4.10 Indexação do Sistema de Dados	57
4.11 Implementação de procedimentos, funções e gatilhos	58
5 Conclusões e Trabalho Futuro	61
5.1 Conclusões	61
5.2 Trabalho Futuro	61
Referências	62
Lista de Siglas e Acrónimos	63
Anexos	64
Anexo 5: Figuras do Projeto	69

Índice de Figuras

1. Figura 1: Ilustração do Relacionamento Utilizador - Inscrição (pág. 19)
2. Figura 2: Ilustração do Relacionamento Inscrição - Pagamento (pág. 20)
3. Figura 3: Ilustração do Relacionamento Inscrição - Bilhete (pág. 21)
4. Figura 4: Ilustração do Relacionamento Sessão - Palestra (pág. 21)
5. Figura 5: Ilustração do Relacionamento Orador - Palestra (pág. 22)
6. Figura 6: Ilustração do Relacionamento Evento - Sessão (pág. 23)
7. Figura 7: Ilustração do Relacionamento Organizador - Evento (pág. 24)
8. Figura 8: Ilustração do Relacionamento Evento - Bilhete (pág. 24)
9. Figura 9: Ilustração do Relacionamento Evento - Patrocinador (pág. 25)
10. Figura 10: Ilustração do Relacionamento Evento - Local/Espaço (pág. 26)
11. Figura 11: Ilustração do diagrama ER produzido (pág. 31)
12. Figura 12: Ilustração do modelo lógico produzido no MySQL Workbench (pág. 36)
13. Figura 13: Instruções SQL para criar e utilizar a base de dados (pág. 40)
14. Figura 14: Tabela lógica do Utilizador (pág. 40)
15. Figura 15: Implementação correspondente ao Utilizador (pág. 40)
16. Figura 16: Tabela lógica do Organizador (pág. 40)
17. Figura 17: Implementação correspondente ao Organizador (pág. 40)
18. Figura 18: Tabela lógica do Local (pág. 41)
19. Figura 19: Implementação correspondente ao Local (pág. 41)
20. Figura 20: Tabela lógica do Orador (pág. 41)
21. Figura 21: Implementação correspondente ao Orador (pág. 41)
22. Figura 22: Tabela lógica do Patrocinador (pág. 42)
23. Figura 23: Implementação correspondente ao Patrocinador (pág. 42)
24. Figura 24: Tabela lógica do Evento (pág. 42)
25. Figura 25: Implementação correspondente ao Evento (pág. 42)
26. Figura 26: Tabela lógica da Sessão (pág. 43)
27. Figura 27: Implementação correspondente à Sessão (pág. 43)
28. Figura 28: Tabela lógica do Bilhete (pág. 43)
29. Figura 29: Implementação correspondente ao Bilhete (pág. 43)
30. Figura 30: Tabela lógica da Palestra (pág. 44)
31. Figura 31: Implementação correspondente à Palestra (pág. 44)
32. Figura 32: Tabela lógica da Inscrição (pág. 44)

- 33. Figura 33: Implementação correspondente à Inscrição (pág. 44)
- 34. Figura 34: Tabela lógica do Pagamento (pág. 45)
- 35. Figura 35: Implementação correspondente ao Pagamento (pág. 45)
- 36. Figura 36: Tabela lógica do Patrocínio (pág. 45)
- 37. Figura 37: Implementação correspondente ao Patrocínio (pág. 45)
- 38. Figura 38: Tabela lógica de Apresenta (pág. 46)
- 39. Figura 39: Implementação correspondente a Apresenta (pág. 46)
- 40. Figura 40: Criação do utilizador "gestor_conteudos" e devidas permissões (pág. 46)
- 41. Figura 41: Criação do utilizador "gestor_participantes" e devidas permissões (pág. 46)
- 42. Figura 42: Inserção de dados na tabela Utilizador com SQL (pág. 47)
- 43. Figura 43: Inserção de dados na tabela Organizador com SQL (pág. 47)
- 44. Figura 44: Inserção de dados na tabela Local com SQL (pág. 47)
- 45. Figura 45: Inserção de dados na tabela Orador com SQL (pág. 48)
- 46. Figura 46: Inserção de dados na tabela Patrocinador com SQL (pág. 48)
- 47. Figura 47: Inserção de dados na tabela Evento com SQL (pág. 48)
- 48. Figura 48: Inserção de dados na tabela Sessao com SQL (pág. 48)
- 49. Figura 49: Inserção de dados na tabela Bilhete com SQL (pág. 49)
- 50. Figura 50: Inserção de dados na tabela Palestra com SQL (pág. 49)
- 51. Figura 51: Inserção de dados na tabela Inscrição com SQL (pág. 49)
- 52. Figura 52: Inserção de dados na tabela Pagamento com SQL (pág. 49)
- 53. Figura 53: Inserção de dados na tabela Patrocínio com SQL (pág. 49)
- 54. Figura 54: Inserção de dados na tabela Apresenta com SQL (pág. 50)
- 55. Figura 55: Vista que demonstra a agenda pública consolidada com os respetivos oradores (pág. 54)
- 56. Figura 56: Vista que demonstra os detalhes dos participantes, inscrições e pagamentos (pág. 54)
- 57. Figura 57: Exemplo de utilização da vista "DetalhesInscricoes" (pág. 55)
- 58. Figura 58: Declaração dos IDs para utilização nas consultas (pág. 55)
- 59. Figura 59: Cálculo do lucro total de um evento com SQL (pág. 55)
- 60. Figura 60: Listar as palestras de um orador com SQL (pág. 55)
- 61. Figura 61: Contagem de inscrições confirmadas por evento com SQL (pág. 56)
- 62. Figura 62: Verificar o método de pagamento mais popular com SQL (pág. 56)
- 63. Figura 63: Listar utilizadores com inscrições pendentes com SQL (pág. 56)
- 64. Figura 64: Listar os *top 3* oradores mais ativos com SQL (pág. 57)
- 65. Figura 65: Criação de índices para a tabela Inscrição (pág. 57)
- 66. Figura 66: Criação de índice para a tabela Palestra (pág. 57)
- 67. Figura 67: Criação de índice para a tabela Sessão (pág. 58)

- 68. Figura 68: Função que calcula a receita total (bilheteira e patrocínios) de um evento (pág. 58)
- 69. Figura 69: Gatilho responsável por atualizar o estado da inscrição após o pagamento (pág. 59)
- 70. Figura 70: Procedimento para gerar a agenda final e ordenada de um evento (pág. 59)
- 71. Figura 71: Gatilho responsável por atualizar as vagas caso desistência (pág. 60)

Índice de Tabelas

1. Tabela 1: Requisitos de descrição (pág. 13)
2. Tabela 2: Requisitos de manipulação de dados (pág. 14)
3. Tabela 3: Requisitos de controlo e segurança (pág. 14)
4. Tabela 4: Caracterização das entidades do sistema (pág. 17)
5. Tabela 5: Caracterização dos relacionamentos do sistema (pág. 19)
6. Tabela 6: Caracterização dos atributos da entidade Utilizador (pág. 26)
7. Tabela 7: Caracterização dos atributos da entidade Inscrição (pág. 27)
8. Tabela 8: Caracterização dos atributos da entidade Pagamento (pág. 27)
9. Tabela 9: Caracterização dos atributos da entidade Evento (pág. 27)
10. Tabela 10: Caracterização dos atributos da entidade Organizador (pág. 28)
11. Tabela 11: Caracterização dos atributos da entidade Sessão (pág. 28)
12. Tabela 12: Caracterização dos atributos da entidade Orador (pág. 28)
13. Tabela 13: Caracterização dos atributos da entidade Palestra (pág. 29)
14. Tabela 14: Caracterização dos atributos da entidade Bilhete (pág. 29)
15. Tabela 15: Caracterização dos atributos da entidade Patrocinador (pág. 29)
16. Tabela 16: Caracterização dos atributos da entidade Local/Espaço (pág. 30)
17. Tabela 17: Caracterização dos atributos do relacionamento Patrocina (pág. 30)
18. Tabela 18: Caracterização da tabela lógica do Utilizador (pág. 32)
19. Tabela 19: Caracterização da tabela lógica do Organizador (pág. 33)
20. Tabela 20: Caracterização da tabela lógica do Local (pág. 33)
21. Tabela 21: Caracterização da tabela lógica do Orador (pág. 33)
22. Tabela 22: Caracterização da tabela lógica do Patrocinador (pág. 33)
23. Tabela 23: Caracterização da tabela lógica do Evento (pág. 34)
24. Tabela 24: Caracterização da tabela lógica da Sessão (pág. 34)
25. Tabela 25: Caracterização da tabela lógica da Palestra (pág. 34)
26. Tabela 26: Caracterização da tabela lógica do Bilhete (pág. 34)
27. Tabela 27: Caracterização da tabela lógica da Inscrição (pág. 35)
28. Tabela 28: Caracterização da tabela lógica do Pagamento (pág. 35)
29. Tabela 29: Caracterização da tabela lógica do Patrocínio (pág. 35)
30. Tabela 30: Caracterização da tabela lógica de Apresenta (pág. 35)
31. Tabela 31: Tipos de dados utilizados e o seu respetivo tamanho em bytes (pág. 50)
32. Tabela 32: Espaço ocupado pela tabela Utilizador com um registo (pág. 50)

- 33. Tabela 33: Espaço ocupado pela tabela Evento com um registo (pág. 51)
- 34. Tabela 34: Espaço ocupado pela tabela Inscrição com um registo (pág. 51)
- 35. Tabela 35: Espaço ocupado pela tabela Pagamento com um registo (pág. 51)
- 36. Tabela 36: Espaço total ocupado pelas tabelas com o povoamento inicial (pág. 52)
- 37. Tabela 37: Variações do espaço ocupado pelas tabelas após um ano (pág. 53)

1. Definição do Sistema

1.1 Contexto de aplicação

Há não muito tempo, a organização de eventos académicos na Universidade do Minho era pautada pela simplicidade. Os encontros de estudantes resumiam-se a palestras em pequenos auditórios, onde as inscrições eram controladas de forma quase artesanal, através de folhas de cálculo dispersas ou formulários online rudimentares. A logística fluía com tranquilidade, e a equipa de estudantes encarregue da organização conseguia gerir o fluxo de participantes, os poucos oradores convidados e os patrocínios locais com relativa facilidade.

Nesse cenário de escala reduzida, um simples ficheiro de Excel partilhado assumia o papel de "quartel-general" de toda a informação. Era o ponto central de gestão, onde eventuais imprevistos ou sobreposições de horários se resolviam rapidamente com uma troca de palavras nos corredores do Campus de Gualtar, refletindo o espírito de proximidade e improvisado da comunidade estudantil.

No entanto, a tranquilidade desta rotina académica estava prestes a sofrer mudanças logísticas irreversíveis. A escolha da cidade de Braga para acolher a edição de 2026 do ENEI transformou o panorama da noite para o dia. Este já não era apenas um encontro local, era um dos maiores eventos tecnológicos universitários do país, prometendo atrair milhares de estudantes de universidades de norte a sul de Portugal.

A magnitude do evento trouxe consigo bastantes oportunidades, mas também uma complexidade sem precedentes. Grandes gigantes tecnológicas, atraídas pelo talento concentrado na cidade, juntaram-se como patrocinadores de peso. Os grandes pavilhões e anfiteatros de Braga rapidamente se encheram com uma grelha frenética de palestras simultâneas, sessões de networking e atividades, recebendo oradores de renome que exigiam uma coordenação ao minuto.

Contudo, nos bastidores, o cenário era de alerta máximo. Com o aumento súbito de milhares de inscrições, a venda de diferentes tipos de bilhetes e a necessidade crítica de alocar participantes a espaços com lotação restrita com vista a evitar a sobrelotação de anfiteatros, os sistemas manuais do passado não conseguiam corresponder às necessidades que um evento desta magnitude implica. A comissão organizadora, que outrora controlava tudo num simples documento, viu-se subitamente soterrada numa montanha de dados desconexos sobre pagamentos pendentes, dados de oradores e conflitos de horários.

O caos iminente deixou claro que a ambição do ENEI 2026 ultrapassava a barreira do improvisado. Braga enfrentava agora a urgência de uma abordagem altamente estruturada e sofisticada. Tornou-se imperativo o desenho e a implementação do ENEI-Gest — um Sistema de Gestão de Bases de Dados robusto, capaz de impor ordem na informação, garantir a segurança dos registos e orquestrar de forma imaculada o maior evento informático do ano.

1.2 Motivação e Objetivos do Trabalho

Confrontada com o crescimento exponencial de inscrições e a enorme complexidade logística associada a um dos maiores eventos tecnológicos universitários de Portugal, a comissão organizadora do ENEI 2026 percebeu que as ferramentas de gestão artesanais do passado já não eram suficientes. Era vital uma reestruturação tecnológica profunda para evitar o colapso logístico e garantir o sucesso do evento na cidade de Braga.

Neste contexto, a organização tomou uma decisão crucial: apostar no talento local. Para enfrentar a tarefa de digitalizar e organizar o evento, recorreram a uma equipa dedicada composta por quatro estudantes de Ciências da Computação da Universidade do Minho, especializados no planeamento, arquitetura e implementação de sistemas íntegros e escaláveis de armazenamento de dados.

Desta parceria nasceu o plano para o *ENEI-Gest*. Ficou assim estabelecida a conceção e implementação de um modelo de base de dados relacional, totalmente personalizado para resolver o caos logístico iminente, assumindo como principais objetivos:

- **Gerir e centralizar as inscrições:** Controlar o fluxo de milhares de participantes e a venda das diferentes tipologias de bilhetes, proporcionando uma experiência de utilizador rápida e sem falhas;
- **Controlar lotações e espaços:** Organizar de forma exímia a alocação de salas e anfiteatros, prevenindo a sobrelotação e garantindo a segurança e o conforto de todos os presentes nas diversas sessões;
- **Coordenar a agenda do evento:** Supervisionar os horários das palestras e atividades, bem como gerir a informação, histórico e alocação dos oradores convidados;
- **Administrar os patrocínios:** Supervisionar de forma eficiente as parcerias financeiras com as empresas tecnológicas, associando estrategicamente os patrocinadores aos respetivos eventos de exposição;
- **Garantir a integridade e segurança:** Proteger rigorosamente a informação pessoal e sensível dos utilizadores, bem como os registos de pagamentos, contra falhas ou ameaças externas;
- **Facilitar a análise de dados (Estatísticas):** Permitir a interrogação rápida e precisa da base de dados para extrair métricas de desempenho, tais como as taxas de ocupação das palestras e os lucros gerados por inscrições;
- **Criar um legado tecnológico estruturado:** Desenvolver um modelo operacional e arquitetural tão robusto que possa ser facilmente reutilizado, escalado e adaptado para as futuras edições do encontro noutras cidades do país.

1.3 Análise da Viabilidade do processo

Antes de se avançar para a implementação definitiva do sistema de gestão de base de dados, realizámos uma análise detalhada da sua viabilidade, procurando identificar as vantagens estratégicas que o *ENEI-Gest* traria para a comissão organizadora do evento.

A implementação deste sistema garantirá à organização um controlo absoluto e um conhecimento aprofundado sobre as relações entre participantes, as diferentes tipologias de bilhetes vendidos e a ocupação real dos espaços físicos. Prevemos que esta centralização de dados resulte numa redução de cerca de 55% nos erros de registo e duplicação de inscrições, problemas que, em edições anteriores geridas manualmente, causavam constrangimentos significativos no dia do evento.

Os parâmetros relativos à lotação dos auditórios, os identificadores únicos de cada estudante e o histórico de participações dos oradores serão avaliados em tempo real. Desta forma, será promovida uma gestão muito mais ágil das sessões ativas e uma rapidez sem precedentes na resolução de conflitos de horários. Com a utilização deste sistema, estimamos uma melhoria na eficiência logística de aproximadamente 40%, permitindo que a equipa de voluntários se foque no apoio aos participantes em vez de tarefas administrativas repetitivas.

Do ponto de vista financeiro, espera-se um aumento direto na transparência e no controlo das receitas. A falta de um sistema integrado de pagamentos e inscrições levava a uma incerteza de cerca de 12% sobre o rendimento final devido a desistências não contabilizadas ou falhas no controlo de acessos VIP. Com o *ENEI-Gest*, prevemos recuperar a totalidade dessa margem, garantindo a estabilidade financeira necessária para investir em futuras edições de maior escala.

Para além disso, a automação do processo de inscrição terá um efeito direto no alívio da carga de trabalho da equipa de secretariado do evento em cerca de 30%, permitindo redirecionar estes recursos humanos para a dinamização de atividades de *networking* e apoio direto aos oradores convidados.

É evidente que a implementação desta base de dados trará benefícios estruturais tanto para a organização como para os estudantes participantes. Tendo em conta os ganhos de eficiência e a segurança de dados projetada, acreditamos que o projeto é perfeitamente viável. A organização assumirá os custos operacionais de manutenção, ciente de que o retorno em termos de prestígio e organização profissional do ENEI Braga 2026 superará largamente o investimento inicial.

Confirmada a sua viabilidade técnica e logística, estabelecemos como previsão para a entrega final do sistema o dia 10 de maio de 2026. Para cumprir este prazo, mobilizámos os esforços da nossa equipa de desenvolvimento de forma a otimizar o cronograma e garantir a execução total do projeto proposto dentro dos mais elevados padrões de qualidade.

1.4 Recursos e Equipa de Trabalho

Para esta fase inicial do projeto, a equipa "**Foreign Four**" sentiu a necessidade de estudar o histórico e os métodos de atuação das edições anteriores do ENEI, de forma a compreender os modelos de gestão aplicados e aprofundar o conhecimento sobre as especificidades de um evento desta dimensão. Seguindo este estudo, organizámos um plano de execução detalhado que depende da articulação entre recursos humanos e materiais.

A nível de recursos humanos, a organização do ENEI 2026 é coordenada por uma comissão central, apoiada por uma equipa técnica de logística e por dezenas de voluntários que lidam diariamente com as operações no terreno. Para garantir que o modelo de dados refletisse a realidade do evento, realizámos diversas sessões de esclarecimento com os representantes da comissão e aplicámos inquéritos exploratórios aos voluntários, cujos testemunhos incidiram sobre as dificuldades sentidas na gestão do elevado fluxo de participantes e na coordenação de espaços.

Quanto aos recursos materiais, a equipa "**Foreign Four**" adotou uma abordagem híbrida e prática. Utilizámos métodos tradicionais, como o registo em papel e caneta para atas de reuniões e rascunhos de diagramas, que foram posteriormente transpostos para formato digital. Esta escolha permitiu uma maior agilidade no *brainstorming* inicial e na recolha de requisitos detalhados sobre a experiência dos utilizadores e a gestão das lotações.

Para além da equipa técnica, os representantes da comissão organizadora do ENEI desempenham um papel crucial na implementação do sistema. Atuando como porta-vozes da organização, a sua participação ativa em reuniões estratégicas permitiu que as preocupações logísticas e as sugestões dos voluntários fossem devidamente integradas no processo de desenho da base de dados. Esta abordagem colaborativa reforça a coesão do projeto, garantindo que as perspetivas de todos os envolvidos desde oradores a patrocinadores sejam salvaguardadas.

No âmbito desta estrutura dedicada, é imprescindível destacar a contribuição dos voluntários do departamento de logística do evento. Estes elementos são vitais na validação prática do sistema, assegurando que o *ENEI-Gest* é intuitivo e eficaz na resolução de problemas em tempo real. A colaboração estreita entre os voluntários e os engenheiros da "**Foreign Four**" fortalece a abordagem integral do projeto.

Por último, importa sublinhar que o sucesso da implementação deste sistema de gestão depende de todos os intervenientes do ENEI. Cada membro da organização desempenha um papel fundamental na alimentação correta de dados e na adaptação às novas ferramentas digitais. Através desta diversidade de experiências e de um ambiente de trabalho colaborativo, o projeto ganha a força necessária para transformar a gestão tecnológica do encontro em Braga.

1.5 Plano de Execução do Projeto

Para assegurar o desenvolvimento estruturado e o cumprimento dos prazos na entrega do sistema de gestão de base de dados *ENEI-Gest*, a equipa "**Foreign Four**" estabeleceu um planeamento cronológico que permitisse

distribuir de forma eficiente todas as tarefas essenciais para a sua concretização.

Com o objetivo de entregar uma primeira fase íntegra do projeto no prazo previsto, optou-se pela aplicação de uma estratégia sequencial de execução, dividindo o projeto em cinco fases fundamentais: Definição do Sistema, Levantamento e Análise de Requisitos, Modelação Conceptual, Modelação Lógica e, por fim, a Implementação Física e Conclusões.

Nesta perspetiva, a equipa delineou marcos internos para a conclusão de cada etapa de modelação. Esta organização lógica permitiu à "**Foreign Four**" garantir uma margem confortável para a revisão e reavaliação de todos os conteúdos, demonstrando um trabalho planeado e eficaz face à data limite de entrega oficial estipulada pelo docente.

Os trabalhos iniciaram-se com foco primeiramente na **Definição do Sistema**. Este período foi dedicado ao estudo do domínio de aplicação, o encontro ENEI e ao aprofundamento do conhecimento na área de gestão de eventos logísticos de grande escala. Destacaram-se, nesta fase, tarefas cruciais como a definição do contexto de aplicação, a motivação, a análise de viabilidade e a estruturação da própria equipa de trabalho. O esforço foi distribuído de forma equitativa, priorizando a pesquisa conjunta para assegurar que todos os elementos partilhassem uma compreensão sólida do desafio.

De seguida, a equipa avançou para a etapa mais sensível e preponderante: o **Levantamento de Requisitos**. Conscientes da sua relevância para o sucesso e coerência das fases subsequentes, atribuímos um período alargado a este processo. O tempo investido justificou-se pela necessidade de aplicar diversos métodos de recolha de informação, diversificando as abordagens através do envolvimento de todos os elementos em reuniões de *brainstorming*, análise de dados de eventos anteriores e registo em atas. A reta final desta fase foi dedicada exclusivamente a corporificar a informação obtida, dividindo o processo na organização, análise, validação e revisão final dos requisitos levantados.

Para a **Modelação Conceptual**, o planeamento previu um foco intensivo e colaborativo. Com os requisitos firmemente estabelecidos, a equipa pôde trabalhar em conjunto na identificação e caracterização das entidades nucleares do sistema e dos seus respetivos relacionamentos. Dada a criticidade dos atributos para a integridade futura da base de dados, o grupo optou por realizar reuniões plenárias para a sua definição detalhada. O culminar desta etapa traduziu-se na elaboração e explicação detalhada do Diagrama Entidade-Relacionamento (ER).

Finalmente, a etapa de **Modelação Lógica** concentrou os esforços da equipa num processo de otimização profunda. O trabalho centrou-se na construção e validação do modelo de dados lógico, procedendo-se simultaneamente à aplicação das regras de normalização para garantir uma visualização de dados intuitiva, eficiente e sem redundâncias. A fase encerrou com a validação do modelo através de interrogações de utilizador projetadas, seguida de uma revisão geral por toda a equipa, confirmando a robustez da arquitetura desenhada antes de se avançar em pleno para a implementação física.

A última fase do projeto, a Implementação Física, iniciou-se em meados de março e prolongou-se até início de maio. Este período foi dedicado à concretização prática do modelo lógico validado, compreendendo a criação efetiva da base de dados no MySQL, a definição de utilizadores e permissões, o povoamento inicial das tabelas com dados representativos e a implementação de mecanismos de otimização (índices, vistas, procedimentos e gatilhos). A equipa reservou também tempo para testes de integridade, validação das interrogações SQL e elaboração do relatório técnico final. A data limite inicialmente estabelecida para a entrega do projeto foi o dia 10 de maio de 2026, posteriormente ajustada para 15 de maio de 2026, assegurando uma margem adicional para revisão e aperfeiçoamento do trabalho desenvolvido.

2. Levantamento e Análise de Requisitos

Na sequência do decorrer das etapas do desenvolvimento, temos a necessidade de fazer um levantamento, organização e efetivação concreta dos requisitos que irão sustentar todo o sistema de gestão de base de dados.

2.1 Método de Levantamento e de Análise de Requisitos Adotado

De forma a garantir um eficiente desenvolvimento do sistema de gestão de base de dados para o ENEI 2026, a equipa "**Foreign Four**" dedicou-se a registar e documentar todas as condições e determinações da organização em relação à melhor forma de estruturar o modelo que envolve a gestão logística do evento. Para tal, recorreu-se aos seguintes métodos de levantamento de requisitos:

- **Reuniões:** Foram realizadas reuniões estratégicas com os representantes da Comissão Organizadora do ENEI, bem como com os responsáveis pelos departamentos de Segurança Informática e Logística. Estes encontros foram fundamentais para entender a melhor forma de gerir os dados relacionados com participantes, espaços físicos, sessões e oradores, bem como os fluxos financeiros e de patrocínios.
- **Inquérito:** Foi executado um questionário denominado "*Inquérito Estatístico de Opinião*", orientado aos potenciais participantes e colaboradores do evento. A principal meta deste instrumento foi aprofundar o conhecimento sobre as necessidades dos utilizadores finais e garantir a transversalidade das soluções propostas pela "**Foreign Four**" de acordo com o contexto real do caso em estudo.

O registo destes procedimentos é imprescindível para assegurar a consistência dos requisitos recolhidos a partir da intervenção dos diversos intervenientes da organização do ENEI. A documentação detalhada destes procedimentos, que serviu de base para a definição das regras de negócio, encontra-se visível nos anexos Anexo 1, Anexo 2, Anexo 3 e Anexo 4. Numa fase final do tratamento dos requisitos, a estruturação dos mesmos foi apresentada e validada pela Comissão Organizadora, garantindo a sua corroboração antes do início da fase de modelação.

2.2 Organização dos Requisitos Levantados

Após a definição do método de levantamento e análise de requisitos, e consequente aplicação do mesmo através das reuniões com a Comissão Organizadora do ENEI, procede-se à disposição organizada da informação recolhida.

Requisitos de descrição

Tipo	Nº	Data	Descrição do requisito	Fonte	Analista
Descrição	1	04/03/2026	O Utilizador/Participante é identificado pelo ID, Nome, Email, Telefone e Data de Registo.	Mariana Silva	João Neiva
	2	04/03/2026	O Evento possui identificador único, título, datas (início/fim) e capacidade máxima.	Tiago Costa	João Neiva
	3	04/03/2026	O Local/Espaço é caracterizado pelo nome, morada completa e lotação máxima permitida.	Tiago Costa	João Neiva
	4	04/03/2026	O Tipo de Bilhete define a designação (VIP/Estudante), o preço e as vagas disponíveis por evento.	Mariana Silva	João Neiva
	5	04/03/2026	A Inscrição associa o participante ao evento, registando a data e o estado (Pendente, Confirmado, Cancelado).	Mariana Silva	João Neiva
	6	04/03/2026	As Palestras contêm título, resumo e horários, estando obrigatoriamente integradas numa Sessão.	Tiago Costa	João Neiva
	7	04/03/2026	O Orador possui identificador, nome, breve biografia e afiliação (instituição ou empresa).	Tiago Costa	João Neiva
	8	14/03/2026	O Pagamento detalha o valor, método (MB Way, Cartão, Transferência) e a data da transação.	Org. ENEI	João Neiva
	9	14/03/2026	O Patrocinador é identificado pelo NIF, nome da empresa e pessoa de contacto.	Org. ENEI	João Neiva
	10	14/03/2026	A Sessão organiza as atividades do evento, com nome da atividade e horários específicos.	Org. ENEI	João Neiva
	11	14/03/2026	O Organizador/Promotor gere o evento, sendo identificado pelo nome da empresa e contactos.	Org. ENEI	João Neiva

Tabela 2.1 Requisitos de descrição.

Requisitos de manipulação

Tipo	Nº	Data	Descrição do requisito	Fonte	Analista
Manipulação	1	04/03/2026	Listar todos os eventos futuros que ainda possuam vagas disponíveis para inscrição.	Mariana Silva	João Durães
	2	04/03/2026	Consultar o histórico completo de eventos e sessões em que um determinado participante se inscreveu.	João Durães	João Durães
	3	04/03/2026	Registar o pagamento de uma inscrição, alterando o seu estado de "Pendente" para "Confirmado".	Mariana Silva	João Durães
	4	08/03/2026	Gerar relatório com o valor total de receitas por evento, permitindo o agrupamento por tipo de bilhete.	Mariana Silva	J. Taveira e M. Gonçalves
	5	08/03/2026	Obter a lista cronológica das palestras agendadas para um evento, incluindo os respetivos oradores.	Inês Correia	J. Taveira e M. Gonçalves
	6	08/03/2026	Permitir que o participante atualize os seus dados de contacto (telefone ou morada) no seu perfil.	Comissão Org.	J. Taveira e M. Gonçalves

Tabela 2.2 Requisitos de manipulação de dados.

Requisitos de controlo

Tipo	Nº	Data	Descrição do requisito	Fonte	Analista
Controlo	1	08/03/2026	Apenas perfis de Administrador ou Organizador podem criar, editar ou remover eventos e sessões.	Ricardo Mendes	J. Taveira e M. Gonçalves
	2	08/03/2026	Um participante comum está restrito à consulta e edição exclusiva dos seus próprios dados pessoais.	João Taveira	J. Taveira e M. Gonçalves
	3	08/03/2026	A alteração da lotação máxima de um local é uma operação exclusiva do administrador da base de dados.	Inês Correia	J. Taveira e M. Gonçalves
	4	08/03/2026	É terminantemente proibido apagar registos de pagamentos; correções devem usar transações de estorno.	M. Gonçalves	J. Taveira e M. Gonçalves

Tabela 2.3 Requisitos de controlo e segurança.

2.3 Análise e Validação Geral dos Requisitos

Como foi previamente referido, nesta última etapa do levantamento, a organização dos requisitos sofreu um processo rigoroso e crucial de validação para atestar a sua consistência e integridade. Deste modo, atendeu-se não só aos resultados exploratórios do *Inquérito Estatístico de Opinião*, mas sobretudo ao *feedback* recolhido na reunião final de validação com a organização do evento, de forma a afinar os pormenores logísticos e resolver eventuais ambiguidades.

Durante essa reunião final, cuja ata se encontra detalhada no Anexo 4, todos os requisitos anteriormente levantados foram minuciosamente avaliados e aprovados em conjunto pelos membros da equipa "**Foreign Four**" e pela Comissão

Organizadora do ENEI. No decorrer da sessão, surgiram pequenas retificações essenciais para a robustez do sistema. Ricardo Mendes, responsável pela segurança informática, solicitou uma clarificação sobre a identificação dos participantes. Ficou formalmente acordado que, além do identificador único (ID) gerado pela base de dados, o campo de *email* terá de ter uma restrição de unicidade (único para cada registo), prevenindo assim a duplicação intencional ou acidental de contas pelo mesmo estudante.

Adicionalmente, Tiago Costa levantou uma questão pertinente quanto à gestão de lotações perante desistências. Como resposta, Miguel Gonçalves clarificou e validou a adição de uma regra de negócio fundamental: sempre que o estado de uma inscrição transitar para "Cancelado", o sistema deverá repor imediatamente a vaga disponível no respetivo tipo de bilhete, garantindo a maximização contínua da ocupação dos espaços e anfiteatros.

Assim sendo, a equipa "**Foreign Four**" tem agora a devida certeza de que o caderno de encargos se encontra em total conformidade com as exigências técnicas e operacionais da Comissão Organizadora do ENEI. A aprovação unânime destes requisitos assegura que a implementação da base de dados refletirá fielmente a complexidade do evento, prevenindo falhas futuras. Concluída esta etapa de validação fina, é então seguro transitar para a fase subsequente, onde será desenvolvido o modelo conceptual, solidamente fundamentado pelas regras agora estabelecidas.

3. Modelação Conceptual

3.1 Apresentação da Abordagem de Modelação Realizada

Com a conclusão da fase de levantamento e respetiva validação dos requisitos, torna-se imperativo transitar para a organização formal da informação recolhida. Para o efeito, a equipa "**Foreign Four**" dedicou-se ao desenvolvimento de um modelo conceptual capaz de estruturar e representar, de forma coerente e rigorosa, os dados logísticos vitais para o funcionamento do ENEI. Para materializar esta abstração do mundo real, recorreu-se à construção de um Diagrama Entidade-Relacionamento (ER).

A seleção da ferramenta usada na construção deste modelo conceptual revelou-se uma tarefa estratégica no processo de desenvolvimento da base de dados. Uma escolha adequada agiliza consideravelmente toda a fase de desenho, garantindo que a equipa desenvolvedora do sistema trabalha sobre uma plataforma perfeitamente integrada e compreensível. Neste sentido, decidiu-se utilizar uma abordagem baseada na notação *Chen*, por ser a convenção preferencial e mais dominada pelos elementos do grupo. Consequentemente, a ferramenta eleita para a prototipagem gráfica foi o *brModelo*, justificada pela sua inegável clareza visual e facilidade na manipulação de entidades, atributos e relacionamentos.

Para dar início à modelação conceptual propriamente dita, é necessário, numa primeira fase, isolar e identificar todas as entidades únicas e relevantes extraídas do caderno de encargos. Em seguida, procede-se ao mapeamento lógico dos relacionamentos entre as mesmas e, por fim, à alocação dos atributos correspondentes, tanto para as entidades como para eventuais relacionamentos com cardinalidade complexa. Identificadas estas componentes estruturais, será então possível proceder às identificações e caracterizações detalhadas de cada uma delas nas secções subsequentes deste documento.

3.2 Identificação e Caracterização das Entidades

Tendo em conta os requisitos recolhidos e validados pela Comissão Organizadora do ENEI, foi possível identificar as seguintes entidades nucleares para o sistema: Utilizador, Evento, Local/Espaço, Organizador/Promotor, Inscrição, Pagamento, Sessão, Orador, Palestra, Patrocinador e Bilhete.

Designação	Descrição	Sinónimos	Ocorrências
Utilizador	Entidade que representa as pessoas que se registam na plataforma para participar.	Participante	Um utilizador realiza 1 ou mais inscrições.
Evento	Entidade central que representa o encontro/evento em si.	Encontro	Um evento é organizado por 1 promotor, ocorre em 1 local, possui 1 ou mais sessões, tem 1 ou mais bilhetes e é alvo de 1 ou mais inscrições.
Local/Espaço	Entidade que representa o local físico onde o evento decorre.	Recinto, Anfiteatro	Um local acolhe 1 ou mais eventos.
Organizador	Entidade que representa a entidade ou comissão que cria e gere o evento.	Promotor, Comissão	Um organizador organiza 1 ou mais eventos.
Inscrição	Entidade associativa que regista a participação de um utilizador num evento.	Registo	Uma inscrição pertence a 1 utilizador, é referente a 1 evento e gera 1 pagamento.
Pagamento	Entidade que regista os detalhes da transação financeira de uma inscrição.	Transação, Fatura	Um pagamento é gerado por 1 inscrição.
Sessão	Entidade que representa um bloco de atividades dentro do evento.	Painel, Bloco	Uma sessão pertence a 1 evento e inclui 1 ou mais palestras.
Orador	Entidade que representa a pessoa convidada para apresentar conteúdo.	Palestrante, Convidado	Um orador apresenta 1 ou mais palestras.
Palestra	Entidade que representa o conteúdo específico apresentado.	Apresentação	Uma palestra é integrada em 1 sessão e apresentada por 1 orador.
Patrocinador	Entidade que representa as empresas que financiam o evento.	Sponsor, Parceiro	Um patrocinador patrocina 1 ou mais eventos.
Bilhete	Entidade que representa as diferentes opções de compra e acesso.	Ingresso, Modalidade	Um bilhete pertence a 1 evento.

Tabela 3.1 Caracterização das entidades do sistema.

Definidas as entidades, é importante passar à enumeração e descrição dos atributos únicos que caracterizam cada uma delas, tendo sempre por base os respetivos requisitos de descrição delineados na Tabela 2.1.

Utilizador

Com base no requisito 1 (Tabela 2.1), um utilizador é identificado de forma única pelo seu *Id_Utilizador* e caracterizado com base no seu *Nome*, *Email*, *Telefone* e *DataRegisto*.

Evento

Segundo o requisito 2 (Tabela 2.1), um evento é identificado pelo seu *Id_Evento* e caracterizado por um *Titulo*, *DataInicio*, *DataFim* e pela sua *CapacidadeMaxima*.

Local/Espaço

Partindo do requisito 3 (Tabela 2.1), um espaço físico é identificado pelo seu *ID_local* e caracterizado pelo seu *Nome*, *Morada* completa e *LotacaoMaxima*.

Organizador/Promotor

Com base no requisito 11 (Tabela 2.1), um organizador é identificado pelo *Id_organizador* e descrito pelo *NomeEmpresa*, *Contacto* e *Email*.

Inscrição

Tendo por base o requisito 5 (Tabela 2.1), uma inscrição é identificada pelo seu *Id_Inscricao* e definida pela *DataInscricao* e pelo seu *Estado* (pendente, confirmado ou cancelado).

Pagamento

Segundo o requisito 8 (Tabela 2.1), os detalhes financeiros são identificados pelo *Id_Pagamentos* e caracterizados pelo *Valor*, *Metodo* (MB Way, Cartão, Transferência) e *DataPagamento*.

Sessão

Partindo do requisito 10 (Tabela 2.1), um bloco do evento é identificado pelo *ID_Sessao* e caracterizado pelo *NomeAtividade*, *HoraInicio* e *HoraFim*.

Orador

Com base no requisito 7 (Tabela 2.1), a pessoa que apresenta é identificada pelo *Id_Orador* e caracterizada pelo *Nome*, *Email*, *Biografia* e *InstituicaoEmpresa*.

Palestra

Segundo o requisito 6 (Tabela 2.1), o conteúdo é identificado pelo *Id_Palestra* e possui um *Titulo*, *Resumo*, *HoraInicio* e *HoraFim*.

Patrocinador

Tendo por base o requisito 9 (Tabela 2.1), a entidade financiadora é identificada pelo *ID_Patrocinador* e caracterizada pelo *nome_empresa*, *NIF* e *pessoa_contacto*.

Bilhete

Por último, partindo do requisito 4 (Tabela 2.1), a opção de acesso é identificada pelo *ID_bilhete* e caracterizada pelo seu *tipo* e *preço*.

3.3 Identificação e Caracterização dos Relacionamentos

Na modelação concetual surgirão vários relacionamentos entre entidades responsáveis por representar as diferentes interações entre os elementos definidos na base de dados.

Entidade	Multiplicidade	Relacionamento	Multiplicidade	Entidade (relacionada)
Utilizador	(1,1) obrigatório	Realiza	(0,n) parcial	Inscrição
Inscrição	(1,1) obrigatório	Gera	(1,1) obrigatório	Pagamento
Organizador	(1,1) obrigatório	Organiza	(1,n) obrigatório	Evento
Inscrição	(1,1) obrigatório	Corresponde	(1,1) obrigatório	Bilhete
Evento	(1,1) obrigatório	Possui	(1,n) obrigatório	Sessão
Sessão	(1,1) obrigatório	Inclui	(1,n) obrigatório	Palestra
Palestra	(1,n) obrigatório	Apresenta	(1,n) obrigatório	Orador
Evento	(1,1) obrigatório	Disponibiliza	(1,n) obrigatório	Bilhete
Evento	(0,n) parcial	Ocorre	(1,1) obrigatório	Local/Espaço
Patrocinador	(0,n) parcial	Patrocina	(1,n) obrigatório	Evento

Tabela 3.2 Caracterização dos relacionamentos do sistema.

Relacionamento Utilizador - Inscrição (Requisito 5, Tabela 2.1)

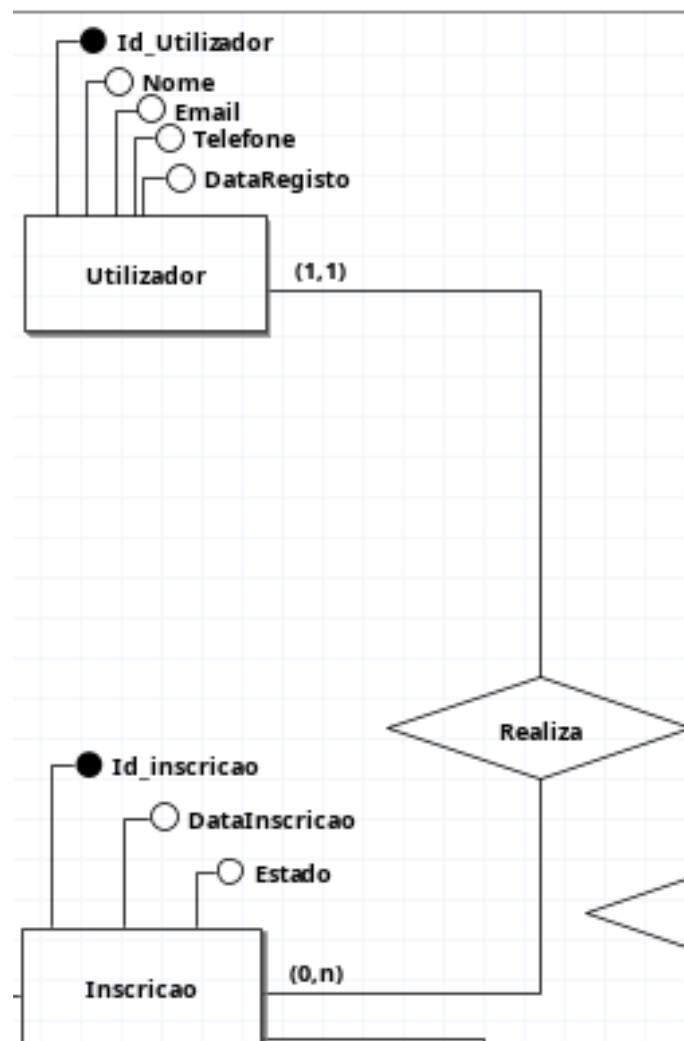


Figura 3.1 Ilustração do Relacionamento Utilizador - Inscrição.

Relacionamento: Utilizador realiza Inscrição.

Descrição: Este relacionamento formaliza a participação de um estudante ou profissional no evento. No sistema *ENEI-Gest*, um utilizador deve estar previamente registado para poder manifestar a sua intenção de presença através de uma inscrição. É através desta ligação que o sistema consegue rastrear quem são os participantes ativos e associar-lhes os respetivos bilhetes e pagamentos.

Cardinalidade: Utilizador (1,1) - Inscrição (0,n).

- **Utilizador (1,1):** Um utilizador pode registar-se na plataforma e não efetuar nenhuma inscrição imediata (0), ou pode realizar múltiplas inscrições ao longo do tempo (n), por exemplo, para diferentes edições ou subeventos específicos que exijam registos separados[cite: 80].
- **Inscrição (0,n):** Cada registo de inscrição no sistema pertence obrigatoriamente a um e apenas um utilizador único, garantindo a integridade dos dados e impedindo inscrições sem um responsável identificado[cite: 80].

Atributos: Este relacionamento é simples e não possui atributos próprios. Toda a informação temporal e de estado (como a *DataInscricao* e o *Estado*) é armazenada diretamente na entidade *Inscrição*[cite: 80].

Relacionamento Inscrição - Pagamento (Requisito 8 de Descrição e Requisito 3 de Manipulação)

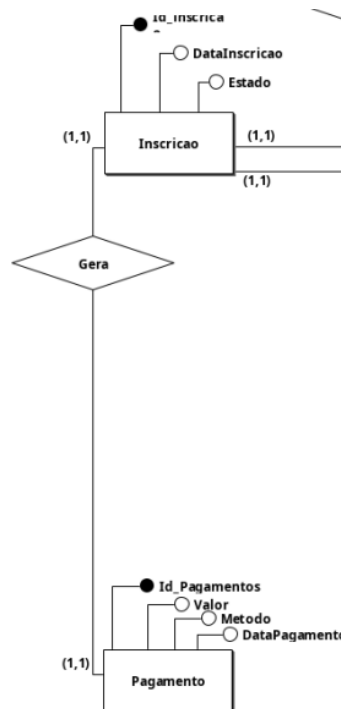


Figura 3.2 Ilustração do Relacionamento Inscrição - Pagamento.

Relacionamento: Inscrição gera Pagamento.

Descrição: Conforme estabelecido na fase de levantamento de requisitos, o *ENEI-Gest* necessita de controlar o fluxo financeiro das participações. Este relacionamento ilustra a obrigatoriedade de associar uma transação financeira a um registo no evento. Na prática, a geração do pagamento é o gatilho lógico que permite ao sistema alterar o estado da *Inscrição* de "Pendente" para "Confirmado".

Cardinalidade: Inscrição (1,1) - Pagamento (1,1). Trata-se de uma relação biunívoca (um-para-um) estrita:

- **Inscrição (1,1):** Uma inscrição no sistema gera, obrigatoriamente, um e apenas um registo de pagamento final para ser validada.

- **Pagamento (1,1):** Em sentido inverso, um registo de pagamento armazenado na base de dados (seja por MB Way, Transferência ou Cartão) diz respeito a uma e apenas uma inscrição, não sendo permitido agrupar faturas de múltiplas inscrições num único registo de pagamento.

Atributos: O relacionamento "Gera" não apresenta atributos próprios. Toda a informação descritiva da transação encontra-se diretamente encapsulada na entidade *Pagamento*.

Relacionamento Inscrição - Bilhete (Requisitos 4 e 5, Tabela 2.1)



Figura 3.3 Ilustração do Relacionamento Inscrição - Bilhete.

Relacionamento: Inscrição corresponde a Bilhete.

Descrição: Para gerir corretamente as vagas e as receitas do ENEI, cada registo de participação tem de estar associado a uma modalidade de acesso. Este relacionamento faz a ponte entre a inscrição realizada pelo utilizador e o tipo de bilhete escolhido (por exemplo, "Estudante", "Profissional" ou "VIP"), o que consequentemente dita o valor a ser pago e o decréscimo das vagas disponíveis.

Cardinalidade: Inscrição (0,n) - Bilhete (1,1).

- **Inscrição (0,n):** Uma inscrição tem de corresponder, de forma estrita e obrigatória, a um e apenas um tipo de bilhete. O participante não pode escolher "meio" bilhete, nem acumular dois tipos de bilhete na mesma inscrição.
- **Bilhete (1,1):** Por outro lado, um tipo de bilhete criado no sistema (ex: "Bilhete Estudante") pode não ter ainda nenhuma inscrição associada (0) ou estar associado a múltiplas inscrições simultaneamente (n), até que o limite de vagas seja atingido.

Atributos: Este relacionamento não possui atributos, uma vez que a informação pertinente já se encontra distribuída: o preço e o tipo estão na entidade *Bilhete*, enquanto o estado e a data estão na entidade *Inscrição*.

Relacionamento Sessão - Palestra (Requisitos 6 e 10, Tabela 2.1)

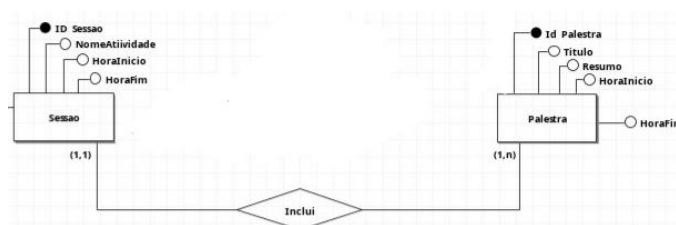


Figura 3.4 Ilustração do Relacionamento Sessão - Palestra.

Relacionamento: Sessão inclui Palestra.

Descrição: A organização do ENEI recorre a sessões (blocos de atividades) para estruturar a agenda do evento de forma lógica (por exemplo, "Sessão de Inteligência Artificial" ou "Painel da Manhã"). Este relacionamento demonstra a composição estrutural do programa, interligando esses grandes blocos aos seus conteúdos mais específicos, ou seja, as palestras individuais.

Cardinalidade: Sessão (1,1) - Palestra (1,n).

- **Sessão (1,1):** Uma sessão criada na grelha do evento tem de englobar, obrigatoriamente, uma ou mais palestras (1,n). No domínio do projeto, uma sessão vazia não faria sentido, pois serve exatamente como contentor de apresentações.
- **Palestra (1,n):** Em contrapartida, uma palestra planeada para o ENEI tem de estar rigorosamente alocada a uma e apenas uma sessão principal (1,1). Isto previne anomalias no sistema e garante que não existem palestras "órfãs" perdidas fora da agenda oficial.

Atributos: Este relacionamento estrutural não requer atributos próprios. O controlo temporal já está desenhado nas respetivas entidades: a *Sessão* define o horizonte temporal alargado do bloco (HoraInicio e HoraFim), enquanto a *Palestra* dita a duração exata dessa apresentação dentro desse mesmo bloco.

Relacionamento Orador - Palestra (Requisitos 6 e 7, Tabela 2.1)

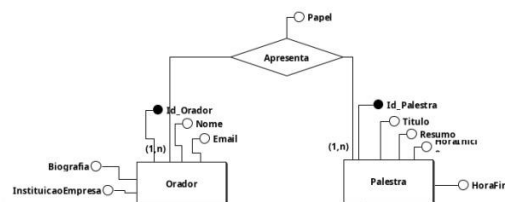


Figura 3.5 Ilustração do Relacionamento Orador - Palestra.

Relacionamento: Orador apresenta Palestra.

Descrição: O núcleo de transmissão de conhecimento do ENEI reside nas suas palestras. Esta ligação associa os convidados (oradores) ao conteúdo específico que vão partilhar. A estrutura foi desenhada para suportar a complexidade de eventos reais, permitindo não só palestras individuais, mas também painéis de debate ou mesas-redondas com múltiplos convidados.

Cardinalidade: Orador (1,n) - Palestra (1,n). Trata-se de uma relação de muitos-para-muitos (M:N).

- **Orador (1,n):** Um orador convidado para o ENEI tem de estar associado a pelo menos uma palestra, podendo intervir em várias sessões diferentes ao longo do evento (1,n).
- **Palestra (1,n):** Uma palestra planeada na grelha necessita de, pelo menos, um orador para ser concretizada, mas pode perfeitamente contar com vários oradores em simultâneo (1,n), como no caso de debates ou apresentações conjuntas.

Atributos do Relacionamento: Devido à cardinalidade M:N, este relacionamento possui um atributo próprio e fundamental: o **Papel**. Este atributo regista a função específica de um determinado orador numa dada palestra (por exemplo, "Orador Principal", "Moderador" ou "Comentador"). Sendo um atributo do relacionamento, permite que a mesma pessoa assuma papéis distintos em palestras diferentes.

Relacionamento Evento - Sessão (Requisitos 2 e 10, Tabela 2.1)

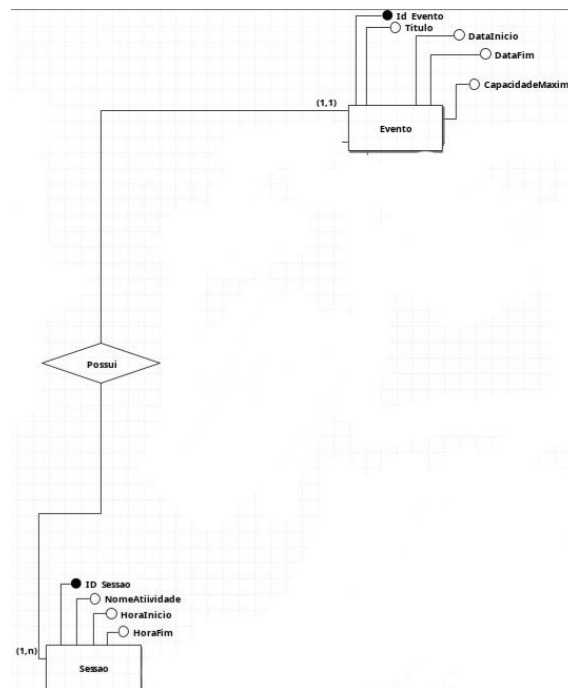


Figura 3.6 Ilustração do Relacionamento Evento - Sessão.

Relacionamento: Evento possui Sessão.

Descrição: Um evento da dimensão do ENEI não ocorre num único bloco de tempo contínuo; é organizado em diversas franjas horárias e áreas temáticas. Este relacionamento estabelece a espinha dorsal da agenda, ligando a entidade macro (Evento) aos seus blocos organizacionais (Sessões). É através desta hierarquia que o sistema sabe exatamente a que dias e grandes áreas as atividades pertencem.

Cardinalidade: Evento (1,1) - Sessão (1,n).

- **Evento (1,1):** Um evento (ou subevento) criado na plataforma tem de possuir, obrigatoriamente, uma ou mais sessões programadas (1,n). Um evento sem sessões seria um evento sem qualquer atividade prevista para os participantes.
- **Sessão (1,n):** Em contrapartida, cada sessão estruturada no sistema pertence, de forma exclusiva e obrigatória, a um e apenas um evento principal (1,1). Não existem sessões independentes ou partilhadas entre eventos distintos, garantindo a integridade da grelha de programação.

Atributos: Sendo um relacionamento de contenção hierárquica (um-para-muitos), não necessita de atributos próprios no diagrama conceptual. As informações temporais e descritivas estão devidamente alojadas nas respetivas entidades (*DataInicio* e *DataFim* no Evento; *HoraInicio* e *HoraFim* na Sessão). **Relacionamento Organizador - Evento**

(Requisitos 2 e 11, Tabela 2.1)

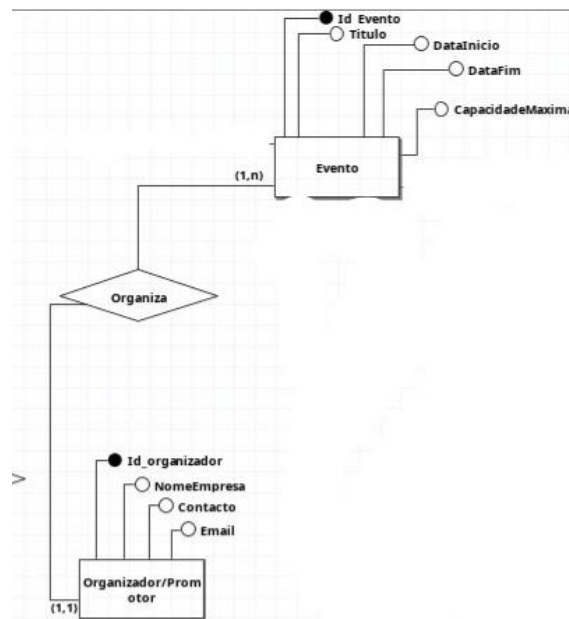


Figura 3.7 Ilustração do Relacionamento Organizador - Evento.

Relacionamento: Organizador/Promotor organiza Evento.

Descrição: Qualquer encontro ou subevento inserido na plataforma ENEI-Gest precisa de uma entidade responsável pela sua criação e gestão. Este relacionamento mapeia a autoria e a responsabilidade administrativa, garantindo que o sistema saiba sempre qual é a comissão ou empresa promotora que detém os direitos de gestão sobre um determinado evento.

Cardinalidade: Organizador (1,1) - Evento (1,n).

- **Organizador (1,1):** Um promotor ou comissão organizadora registada na base de dados tem de organizar, pelo menos, um evento (caso contrário, não faria sentido o seu registo estrutural), podendo ser responsável por múltiplos eventos ou edições ao longo do tempo (1,n).
- **Evento (1,n):** Pela perspetiva oposta, cada evento criado no sistema tem de estar obrigatoriamente associado a um e apenas um organizador principal (1,1). Isto previne conflitos de gestão e assegura que não existem eventos "órfãos" sem uma entidade promotora definida.

Atributos: Este relacionamento de dependência não possui atributos próprios. Os dados de contacto da entidade estão na tabela *Organizador/Promotor*, enquanto os dados do encontro estão na tabela *Evento*.

Relacionamento Evento - Bilhete (Requisitos 2 e 4, Tabela 2.1)

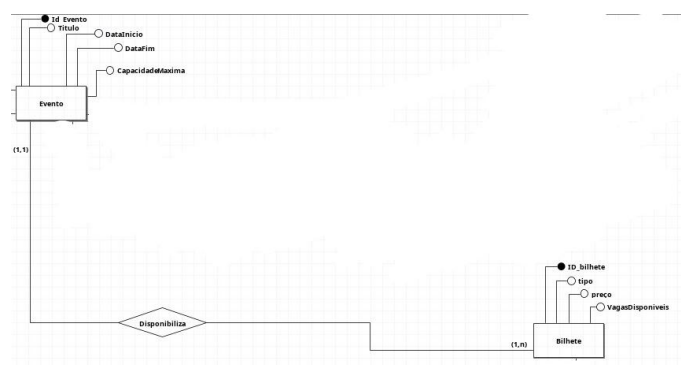


Figura 3.8 Ilustração do Relacionamento Evento - Bilhete.

Relacionamento: Evento disponibiliza Bilhete.

Descrição: Para gerir os acessos, as vagas e as receitas, o ENEI-Gest necessita de estruturar o seu preçário de forma flexível. Este relacionamento faz a ponte entre o evento (o acontecimento em si) e as diferentes opções comerciais que os utilizadores podem adquirir. Sem esta ligação, seria impossível ter diferentes patamares de preços para o mesmo encontro.

Cardinalidade: Evento (1,1) - Bilhete (1,n).

- **Evento (1,1):** Um evento registado na plataforma tem de disponibilizar, obrigatoriamente, pelo menos um tipo de bilhete (1) para que as pessoas se possam inscrever, podendo oferecer múltiplas modalidades de acesso em simultâneo (n).
- **Bilhete (1,n):** Do lado oposto, um tipo de bilhete configurado na base de dados (por exemplo, "Passe Geral 3 Dias - ENEI 2026") é exclusivo e pertence a um e apenas um evento (1,1). Os tipos de bilhete não são partilhados entre eventos diferentes, garantindo assim que a contabilidade e a gestão de vagas não sofrem anomalias.

Atributos: Este relacionamento não requer atributos próprios. As características comerciais, como o *tipo* e o *preço*, estão devidamente encapsuladas na entidade *Bilhete*.

Relacionamento Evento - Patrocinador (Requisitos 2 e 9, Tabela 2.1)

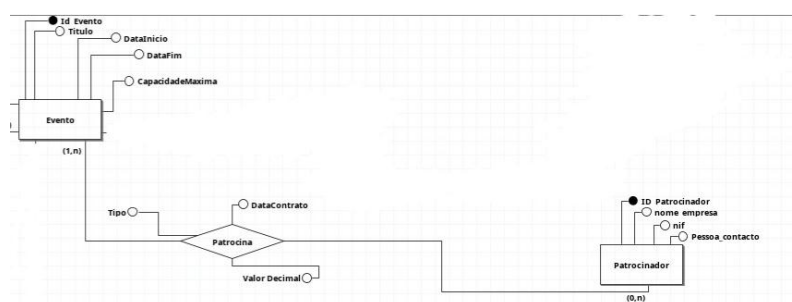


Figura 3.9 Ilustração do Relacionamento Evento - Patrocinador.

Relacionamento: Patrocinador patrocina Evento.

Descrição: A viabilidade financeira do ENEI depende fortemente de parcerias com o tecido empresarial. Este relacionamento documenta as ligações financeiras e logísticas entre as empresas parceiras (patrocinadores) e os eventos ou subeventos do encontro.

Cardinalidade: Evento (1,n) - Patrocinador (0,n). Trata-se de uma relação de muitos-para-muitos (M:N).

- **Patrocinador (0,n):** Uma empresa registada na base de dados do sistema tem de patrocinar, pelo menos, um evento (1), caso contrário não haveria motivo para o seu registo. Contudo, a mesma empresa pode financiar múltiplos eventos ou edições anuais do ENEI (n).
- **Evento (1,n):** Um evento criado na plataforma pode, numa fase inicial ou por natureza do seu formato, não ter qualquer patrocinador associado (0). No entanto, o cenário ideal e mais comum é que um evento receba o apoio de múltiplas entidades empregadoras e tecnológicas em simultâneo (n).

Atributos do Relacionamento: Uma vez que um patrocinador pode ter acordos diferentes consoante o evento, o relacionamento *Patrocina* possui atributos cruciais:

- **Tipo:** Define a categoria do patrocínio para aquele evento em específico (por exemplo, "Gold", "Silver", "Bronze" ou "Media Partner").
- **DataContrato:** Regista a data em que a parceria foi oficialmente firmada para aquele evento.
- **Valor Decimal:** Armazena o montante financeiro exato investido pelo patrocinador naquele evento específico, garantindo um rastreio rigoroso das receitas para efeitos de tesouraria.

Relacionamento Evento - Local/Espaço (Requisito 3, Tabela 2.1)

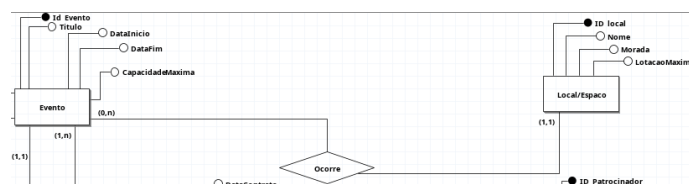


Figura 3.10 Ilustração do Relacionamento Evento - Local/Espaço.

Relacionamento: Evento ocorre em Local/Espaço.

Descrição: A logística do ENEI exige uma gestão rigorosa de infraestruturas físicas. Este relacionamento assegura a definição do local/espço onde o evento decorre[cite: 7, 9]. Esta ligação garante que os participantes saibam exatamente para onde se dirigir e permite à organização controlar as capacidades dos edifícios.

Cardinalidade: Evento (0,n) - Local/Espaço (1,1).

- **Evento (0,n):** Um evento planeado na plataforma ocorre, obrigatoriamente, num e apenas num local físico definido (1,1).
- **Local/Espaço (1,1):** Por outro lado, um local ou anfiteatro registado no sistema, caracterizado pelo seu ID, nome, morada e lotação máxima[cite: 10], pode não estar ainda alocado a nenhuma atividade (0) ou servir para acolher múltiplos eventos distintos ao longo dos dias (n).

Atributos: O relacionamento conceptual em si não necessita de atributos próprios. A ligação lógica e de controlo é feita através das próprias entidades, estando já prevista a inclusão de uma referência ao local (DlocalFk) diretamente nos atributos da entidade Evento.

3.4 Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos

Nesta etapa, a caracterização dos atributos das entidades e, possivelmente, dos relacionamentos passa a ser o alvo principal.

Entidade Utilizador

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Utilizador	Id_Utilizador	Identificador único do participante.	Não	Não	Não	Não	Sim
	Nome	Nome completo do participante.	Não	Não	Não	Não	Não
	Email	Endereço de correio eletrónico.	Não	Não	Não	Não	Não
	Telefone	Contacto(s) telefónico(s).	Não	Não	Sim	Não	Não
	DataRegisto	Data de criação da conta no sistema.	Não	Não	Não	Não	Não

Tabela 3.3 Caracterização dos atributos da entidade Utilizador.

Nenhum dos atributos da entidade **Utilizador** (**Id_Utilizador** - chave primária, **Nome**, **Email**, **Telefone** e **DataRegisto**) é **composto** ou **derivado**. Nenhum atributo pode ser **nulo**, sendo todos de preenchimento obrigatório no momento da criação de conta, logo, **não nulos**. Por fim, apenas o atributo **Telefone** é **multivalorado**, visto que um participante pode registar vários números de contacto.

Entidade Inscrição

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Inscrição	Id_Inscricao	Identificador único da inscrição.	Não	Não	Não	Não	Sim
	DataInscricao	Data e hora de submissão do registo.	Não	Não	Não	Não	Não
	Estado	Situação atual (Pendente/Confirmado).	Não	Não	Não	Não	Não

Tabela 3.4 Caracterização dos atributos da entidade Inscrição.

Nenhum dos atributos da entidade **Inscrição** (**Id_Inscricao** - chave primária, **DataInscricao** e **Estado**) é **composto** ou **derivado**. Nenhum atributo pode ser **nulo**, sendo todos os atributos **não nulos**. Por fim, nenhum atributo é **multivalorado**, visto que cada registo de participação possui apenas uma data e um estado únicos associados.

Entidade Pagamento

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Pagamento	Id_Pagamentos	Identificador único da transação.	Não	Não	Não	Não	Sim
	Valor	Montante total da operação.	Não	Não	Não	Não	Não
	Metodo	Via de pagamento (MBWay/Cartão).	Não	Não	Não	Não	Não
	DataPagamento	Data de liquidação do valor.	Não	Não	Não	Não	Não

Tabela 3.5 Caracterização dos atributos da entidade Pagamento.

Nenhum dos atributos da entidade **Pagamento** (**Id_Pagamentos** - chave primária, **Valor**, **Metodo** e **DataPagamento**) é **composto**, **derivado** ou **multivalorado**. Sendo uma entidade de controlo financeiro, todos os atributos são estritamente obrigatórios, logo, **não nulos**.

Entidade Evento

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Evento	Id_Evento	Identificador único do evento.	Não	Não	Não	Não	Sim
	Titulo	Nome ou designação do evento.	Não	Não	Não	Não	Não
	DataInicio	Data temporal de arranque.	Não	Não	Não	Não	Não
	DataFim	Data temporal de encerramento.	Não	Não	Não	Não	Não
	CapacidadeMaxima	Lotação global limite do evento.	Não	Não	Não	Não	Não

Tabela 3.6 Caracterização dos atributos da entidade Evento.

Nenhum dos atributos da entidade **Evento** (**Id_Evento** - chave primária, **Titulo**, **DataInicio**, **DataFim** e **CapacidadeMaxima**) é **composto**, **derivado** ou **multivalorado**. Nenhum dos atributos pode ser **nulo**, visto que todos representam dados essenciais para o planeamento, divulgação e estruturação do encontro na plataforma, sendo todos **não nulos**.

Entidade Organizador

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Organizador	Id_organizador	Identificador único do promotor.	Não	Não	Não	Não	Sim
	NomeEmpresa	Designação da comissão/empresa.	Não	Não	Não	Não	Não
	Contacto	Telefones de suporte.	Não	Não	Sim	Não	Não
	Email	Correio eletrónico institucional.	Não	Não	Não	Não	Não

Tabela 3.7 Caracterização dos atributos da entidade Organizador.

Nenhum dos atributos da entidade **Organizador** (**Id_organizador** - chave primária, **NomeEmpresa**, **Contacto** e **Email**) é **composto** ou **derivado**. Nenhum dos campos pode ser **nulo**, sendo todos **não nulos**. Por fim, apenas o atributo **Contacto** é **multivalorado**, permitindo que a comissão organizadora registe vários telefones de suporte simultaneamente.

Entidade Sessão

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Sessão	ID_Sessao	Identificador único do bloco horário.	Não	Não	Não	Não	Sim
	NomeAtividade	Designação temática da sessão.	Não	Não	Não	Não	Não
	HoraInicio	Horário de abertura da atividade.	Não	Não	Não	Não	Não
	HoraFim	Horário de encerramento previsto.	Não	Não	Não	Não	Não

Tabela 3.8 Caracterização dos atributos da entidade Sessão.

Nenhum dos atributos da entidade **Sessão** (**ID_Sessao** - chave primária, **NomeAtividade**, **HoraInicio** e **HoraFim**) é **composto**, **derivado** ou **multivalorado**. Para garantir a integridade da agenda do evento, todos os atributos são obrigatórios, sendo, por isso, **não nulos**.

Entidade Orador

Nenhum dos atributos da entidade **Orador** (**Id_Orador** - chave primária, **Nome**, **Email**, **Biografia** e **InstituicaoEmpresa**) é **composto**, **derivado** ou **multivalorado**. Apenas o atributo **Biografia** pode ser **nulo**, assumindo o cenário em que um orador convidado opta por não submeter um texto descritivo sobre a sua carreira, sendo todos os outros atributos **não nulos**.

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Orador	Id_Orador	Identificador único do palestrante.	Não	Não	Não	Não	Sim
	Nome	Nome do convidado.	Não	Não	Não	Não	Não
	Email	Contacto eletrónico do orador.	Não	Não	Não	Não	Não
	Biografia	Breve texto sobre o currículo.	Sim	Não	Não	Não	Não
	InstituicaoEmpresa	Empresa ou faculdade de origem.	Não	Não	Não	Não	Não

Tabela 3.9 Caracterização dos atributos da entidade Orador.

Entidade Palestra

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Palestra	Id_Palestra	Identificador único da apresentação.	Não	Não	Não	Não	Sim
	Titulo	Nome da palestra/workshop.	Não	Não	Não	Não	Não
	Resumo	Breve descrição do conteúdo.	Sim	Não	Não	Não	Não
	HoraInicio	Hora de início da apresentação.	Não	Não	Não	Não	Não
	HoraFim	Hora de término da apresentação.	Não	Não	Não	Não	Não

Tabela 3.10 Caracterização dos atributos da entidade Palestra.

Nenhum dos atributos da entidade **Palestra** (**Id_Palestra** - chave primária, **Titulo**, **Resumo**, **HoraInicio** e **HoraFim**) é **composto**, **derivado** ou **multivalorado**. Apenas o atributo **Resumo** pode ser **nulo**, permitindo o registo da atividade antes da submissão do resumo final, sendo todos os outros atributos **não nulos**.

Entidade Bilhete

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Bilhete	ID_bilhete	Identificador único da modalidade.	Não	Não	Não	Não	Sim
	tipo	Categoria do acesso (ex: Estudante, VIP).	Não	Não	Não	Não	Não
	preço	Valor unitário da modalidade.	Não	Não	Não	Não	Não
	VagasDisponiveis	Valor de Vagas Disponiveis.	Não	Não	Não	Não	Não

Tabela 3.11 Caracterização dos atributos da entidade Bilhete.

Nenhum dos atributos da entidade **Bilhete** (**ID_bilhete** - chave primária, **tipo**, **preço** e **VagasDisponiveis**) é **composto**, **derivado** ou **multivalorado**. Todos os campos são de preenchimento obrigatório, pelo que não podem ser **nulos**, sendo todos **não nulos**, uma vez que definem o preço, as modalidades de acesso e o controlo de disponibilidade de vagas fundamentais ao sistema.

Entidade Patrocinador

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Patrocinador	ID_Patrocinado	Identificador único da empresa parceira.	Não	Não	Não	Não	Sim
	nome_empresa	Designação comercial da marca.	Não	Não	Não	Não	Não
	nif	Número de Identificação Fiscal.	Não	Não	Não	Não	Não
	pessoa_contact_	Nome do responsável de contacto.	Não	Não	Não	Não	Não

Tabela 3.12 Caracterização dos atributos da entidade Patrocinador.

Nenhum dos atributos da entidade **Patrocinador** (**ID_Patrocinado** - chave primária, **nome_empresa**, **nif** e **pessoa_contact_**) é **composto**, **derivado** ou **multivalorado**. Todos os atributos são **não nulos**, dada a necessidade de identificação fiscal e administrativa rigorosa de cada parceiro, pelo que nenhum pode ser **nulo**.

Entidade Local/Espaço

Entidade	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Local/Espaço	ID_local	Identificador único do recinto.	Não	Não	Não	Não	Sim
	Nome	Designação do anfiteatro ou sala.	Não	Não	Não	Não	Não
	Morada	Localização física do espaço.	Não	Sim	Não	Não	Não
	LotacaoMaxima	Número máximo de lugares sentados.	Não	Não	Não	Não	Não

Tabela 3.13 Caracterização dos atributos da entidade Local/Espaço.

Nenhum dos atributos da entidade **Local/Espaço** (**ID_local** - chave primária, **Nome**, **Morada** e **LotacaoMaxima**) é **multivalorado**, **derivado** ou **nulo**, sendo todos de preenchimento obrigatório, logo **não nulos**. Apenas o atributo **Morada** é **composto**, uma vez que se prevê a sua subdivisão lógica (Rua, Código Postal, Cidade), sendo os restantes atributos **não compostos**.

Relacionamento Patrocina

Relacionamento	Atributo	Descrição	Nulo	Comp.	Multi.	Deriv.	PK
Patrocina	Tipo	Categoria do patrocínio (ex: Gold).	Não	Não	Não	Não	Não
	DataContrato	Data de formalização da parceria.	Não	Não	Não	Não	Não
	Valor Decimal	Montante financeiro investido.	Não	Não	Não	Não	Não

Tabela 3.14 Caracterização dos atributos do relacionamento Patrocina.

Nenhum dos atributos do relacionamento **Patrocina** (**Tipo**, **DataContrato** e **Valor Decimal**) é **composto**, **derivado** ou **multivalorado**. Sendo essenciais para o registo do acordo financeiro entre a empresa parceira e o evento, todos os atributos são obrigatórios, logo **não nulos**. Tratando-se de atributos pertencentes a um relacionamento, nenhum atua isoladamente como **chave primária**.

3.5 Apresentação e Explicação do Diagrama ER Produzido

Após a identificação e caracterização detalhada de cada entidade, atributo e relacionamento nos tópicos anteriores, prosseguimos agora para a apresentação de todo o diagrama ER que suporta o modelo da base de dados em questão.

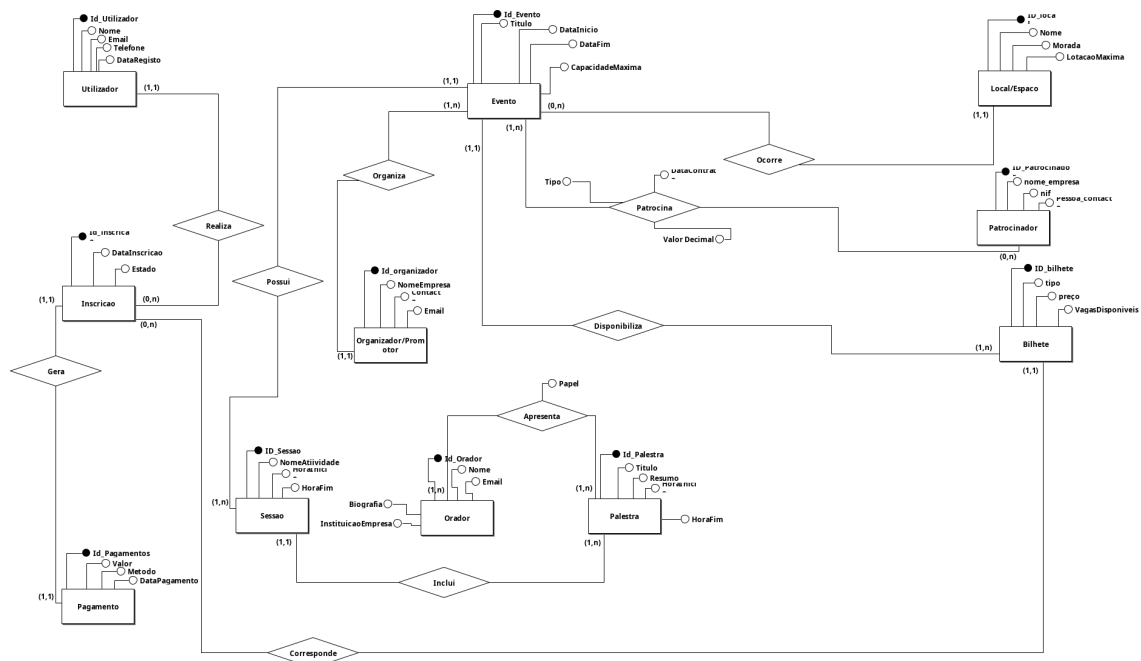


Figura 3.11 Ilustração do diagrama ER produzido.

Neste diagrama, é possível identificar de forma clara todas as entidades da base de dados, bem como os relacionamentos responsáveis por interligar as mesmas. Deste modo, o diagrama é interpretado da seguinte forma: um organizador/promotor organiza obrigatoriamente um evento; um evento ocorre num local/espaco definido, disponibiliza bilhetes e possui diversas sessões; um evento pode também ser patrocinado por vários patrocinadores; uma sessão engloba e inclui palestras; uma ou várias palestras são apresentadas por oradores; um utilizador participante realiza inscrições no sistema; uma inscrição corresponde obrigatoriamente a um bilhete e, por fim, gera um registo de pagamento.

Por fim, podemos também identificar, em cada entidade e em alguns dos relacionamentos (como o *Patrocina* e o *Apresenta*), os atributos que os definem. Em todas as entidades destacam-se os atributos que constituem uma chave primária da mesma, visto serem representados por um círculo totalmente preenchido. Para além disso, no diagrama são também visíveis outros tipos de atributos explorados anteriormente, tais como atributos multivalorados e opcionais.

Assim, com a conclusão desta fase, é possível passar para a construção de um novo modelo capaz de especificar a caracterização de cada um dos elementos, sendo este o modelo de dados lógico.

4. Modelação Lógica

4.1 Construção e Validação do Modelo de Dados Lógico

Após a conclusão e revisão da modelação conceptual, a equipa "Foreign Four" está agora a avançar para a elaboração do Modelo de Dados Lógico do sistema ENEI-Gest. Este tipo de modelo descreve os elementos de dados em detalhe e é utilizado para criar uma compreensão visual das tabelas, atributos, chaves (primárias e estrangeiras) e relacionamentos. O objetivo principal é estabelecer uma estrutura fundamental para os componentes da camada semântica em sistemas de gestão de dados, fazendo a ponte entre a abstração inicial e a implementação técnica, mas ainda sem depender de um motor de base de dados específico. Propôs-se a elaboração deste Modelo de Dados Lógico com o intuito de definir rigorosamente como o sistema em desenvolvimento deve ser implementado.

Em particular, o Modelo Lógico proporciona mais detalhes na representação das entidades convertidas em tabelas relacionais, oferecendo um contexto visual para os dados, os tipos de dados previstos e as restrições de integridade referencial entre eles. Para alcançar este objetivo, a equipa de desenvolvimento optou por utilizar a ferramenta MySQL Workbench para projetar e desenhar o modelo esperado.

O MySQL Workbench é uma plataforma de modelação de dados poderosa e versátil, projetada para auxiliar na criação, visualização e gestão de modelos de dados lógicos e físicos. Com uma interface intuitiva e diversas funcionalidades avançadas, o MySQL Workbench permite aos utilizadores criar representações precisas e compreensíveis da estrutura de dados e das suas inter-relações, facilitando assim o desenvolvimento de um sistema de informação robusto, escalável e eficaz para a gestão do ENEI.

4.2 Apresentação e Explicação do Modelo Lógico Produzido

Para a correta construção do modelo lógico, a equipa "Foreign Four" reuniu-se para considerar como proceder à execução do diagrama, apoiada, inicialmente, por uma ponderação sobre a quantidade de tabelas a utilizar. Procede-se à apresentação dos resultados:

- 11 tabelas, uma por cada entidade base;
- 2 tabelas associativas, uma por cada relacionamento de cardinalidade N para M (*Patrocina* e *Apresenta*).

A equipa iniciou a produção das 13 tabelas referidas acima de acordo com a ordem de formulação do modelo conceptual. Para tal, inaugurou o diagrama com a criação da tabela de **Utilizador** ao herdar os atributos já definidos no diagrama de sustento, que se define pelas seguintes colunas:

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Utilizador	"id_Utilizador"	INT	-
	"Name"	VARCHAR(100)	-
	"Email"	VARCHAR(100)	-
	"Data Registo"	DATETIME	-
	"Telefone"	VARCHAR(20)	-

Tabela 4.1 Caracterização da tabela lógica do Utilizador.

De seguida, a equipa avançou com a conversão das restantes entidades base (independentes e sem chaves estrangeiras), criando as respetivas tabelas e associando os atributos previamente definidos:

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Organizador	"id_Organizador"	INT	-
	"NomeEmpresa"	VARCHAR(100)	-
	"Email"	VARCHAR(100)	-
	"Contacto"	VARCHAR(20)	-

Tabela 4.2 Caracterização da tabela lógica do Organizador.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Local	"id_Local"	INT	-
	"Nome"	VARCHAR(100)	-
	"Morada"	VARCHAR(100)	-
	"LotacaoMaxima"	INT	-

Tabela 4.3 Caracterização da tabela lógica do Local.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Patrocinador	"id_Patrocinador"	INT	-
	"NomeEmpresa"	VARCHAR(100)	-
	"NIF"	VARCHAR(20)	-
	"PessoaContacto"	VARCHAR(100)	-

Tabela 4.4 Caracterização da tabela lógica do Patrocinador.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Orador	"id_Orador"	INT	-
	"Name"	VARCHAR(100)	-
	"Email"	VARCHAR(100)	-
	"Biografia"	TEXT	-
	"InstituicaoEmpresa"	VARCHAR(100)	-

Tabela 4.5 Caracterização da tabela lógica do Orador.

Com as entidades de suporte definidas, deu-se seguimento à criação da tabela central do sistema, **Evento**, que reflete os atributos da modelação concetual e herda as chaves primárias do Local e do Organizador:

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Evento	"id_Evento"	INT	-
	"Titulo"	VARCHAR(150)	-
	"DataInicio"	DATETIME	-
	"DataFim"	DATETIME	-
	"CapacidadeMaxima"	INT	-
	"Local_id_Local"	INT	"id_Local" (Local)
	"Organizador_id_Organizador"	INT	"id_Organizador" (Organizador)

Tabela 4.6 Caracterização da tabela lógica do Evento.

Na sequência da existência de um Evento, torna-se necessário fazer surgir as tabelas **Sessão**, **Palestra** e **Bilhete**, estabelecendo a hierarquia e as dependências organizacionais:

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Sessão	"id_Sessao"	INT	-
	"NomeAtividade"	VARCHAR(100)	-
	"HoraInicio"	DATETIME	-
	"HoraFim"	DATETIME	-
	"Evento_id_Evento"	INT	"id_Evento" (Evento)

Tabela 4.7 Caracterização da tabela lógica da Sessão.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Palestra	"id_Palestra"	INT	-
	"Titulo"	VARCHAR(100)	-
	"Resumo"	TEXT	-
	"HoraInicio"	DATETIME	-
	"HoraFim"	DATETIME	-
	"Sessao_id_Sessao"	INT	"id_Sessao" (Sessão)

Tabela 4.8 Caracterização da tabela lógica da Palestra.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Bilhete	"id_Bilhete"	INT	-
	"Tipo"	VARCHAR(50)	-
	"Preco"	DECIMAL(6,2)	-
	"Evento_id_Evento"	INT	"id_Evento" (Evento)

Tabela 4.9 Caracterização da tabela lógica do Bilhete.

Para assegurar o processo de participação, foram transfiguradas as tabelas **Inscrição** e **Pagamento**, que herdam as

respetivas chaves estrangeiras para garantir a integridade dos dados financeiros e de acesso:

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Inscrição	"id_Inscricao"	INT	-
	"DataInscricao"	DATETIME	-
	"Estado"	VARCHAR(20)	-
	"Utilizador_id_Utilizador"	INT	"id_Utilizador" (Utilizador)
	"Bilhete_id_Bilhete"	INT	"id_Bilhete" (Bilhete)

Tabela 4.10 Caracterização da tabela lógica da Inscrição.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Pagamento	"id_Pagamento"	INT	-
	"Valor"	DECIMAL(6,2)	-
	"Metodo"	VARCHAR(50)	-
	"DataPagamento"	DATETIME	-
	"Inscricao_id_Inscricao"	INT	"id_Inscricao" (Inscrição)

Tabela 4.11 Caracterização da tabela lógica do Pagamento.

Ao analisar o diagrama concetual, repara-se nos relacionamentos de cardinalidade N para M entre as entidades. No processo de conversão, estes culminam com o aparecimento de novas tabelas associativas (**Patrocínio** e **Apresenta**) que estabelecem os relacionamentos e guardam os atributos próprios dos mesmos:

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Patrocínio	"Evento_id_Evento"	INT	"id_Evento" (Evento)
	"Patrocinador_id_Patrocinador"	INT	"id_Patrocinador" (Patrocinador)
	"Tipo"	VARCHAR(45)	-
	"DataContrato"	DATETIME	-
	"Valor"	DECIMAL(6,2)	-

Tabela 4.12 Caracterização da tabela associativa Patrocínio.

Tabela	Nome da coluna	Tipo de dado	Referência (tabela)
Apresenta	"Orador_id_Orador"	INT	"id_Orador" (Orador)
	"Palestra_id_Palestra"	INT	"id_Palestra" (Palestra)
	"Papel"	VARCHAR(45)	-

Tabela 4.13 Caracterização da tabela associativa Apresenta.

Na modelação lógica, os relacionamentos entre entidades desempenham um papel crucial na representação da interação entre diferentes elementos de um sistema de gestão de base de dados:

Relacionamento binário de grau 1:N com participação obrigatória do lado N: São necessárias duas entidades lógicas, uma para cada entidade, e a chave primária da entidade do lado 1 tem de ser usada como atributo (chave estrangeira) na entidade correspondente à entidade do lado N. *Ocorrências no modelo:*

- "ocorre" entre Evento(N) e Local(1);
- "organiza" entre Evento(N) e Organizador(1);
- "possui" entre Sessão(N) e Evento(1);
- "engloba" entre Palestra(N) e Sessão(1);
- "realiza" entre Inscrição(N) e Utilizador(1);
- "corresponde" entre Inscrição(N) e Bilhete(1).

Relacionamento binário de grau N:M: São sempre necessárias três entidades lógicas neste tipo de relacionamentos, uma para cada entidade e uma para o relacionamento (tabela associativa). As chaves primárias das entidades base têm de ser atributos (formando uma chave primária composta e agindo como chaves estrangeiras) na entidade lógica do relacionamento. *Ocorrências no modelo:*

- "patrocina" entre Patrocinador e Evento (gera a tabela *Patrocínio*);
- "apresenta" entre Orador e Palestra (gera a tabela *Apresenta*).

Com base nos parâmetros e nos relacionamentos estabelecidos anteriormente, foi desenvolvido o modelo lógico no MySQL Workbench para representar de forma precisa e organizada as interações entre as entidades envolvidas. Este modelo visa proporcionar uma estrutura robusta para armazenar e gerir informações relacionadas aos diversos elementos do sistema *ENEI-Gest*.

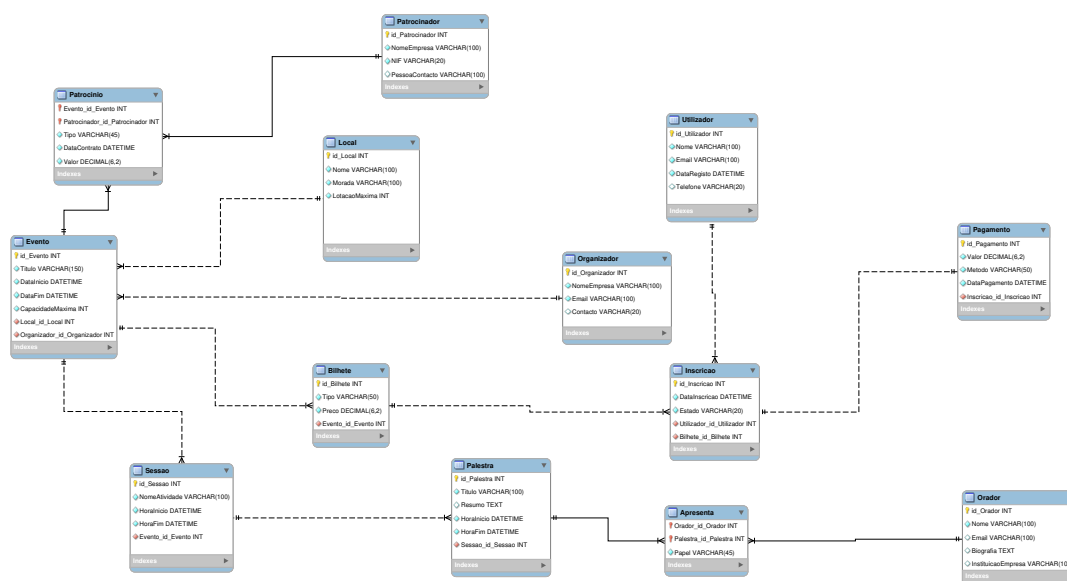


Figura 4.1 Ilustração do modelo lógico produzido no MySQL Workbench.

4.3 Normalização de Dados

A normalização de uma base de dados é um processo crucial na execução de um sistema de gestão de base de dados relacional. O objetivo principal da normalização é organizar os dados de forma a minimizar a redundância, ambiguidade

e inconsistência, garantindo, em simultâneo, a integridade da informação e facilitando as operações de consulta e manipulação da mesma.

Existem várias formas normais, habitualmente descritas pela nomenclatura 1NF (Primeira Forma Normal), 2NF (Segunda Forma Normal), 3NF (Terceira Forma Normal), entre outras. Cada uma destas formas normais aborda diferentes tipos de problemas de difusão e dependências funcionais entre os atributos das tabelas.

Na **Primeira Forma Normal (1NF)**, as tabelas são organizadas de forma a que cada célula contenha apenas um valor indivisível (atómico), eliminando grupos repetitivos ou listas de valores. Importa aqui referir uma decisão técnica da equipa: no modelo conceptual, os atributos **Telefone** (da entidade Utilizador) e **Contacto** (da entidade Organizador) foram identificados como multivalorados. Contudo, no modelo lógico, optou-se por representá-los como um simples `VARCHAR(20)`. Esta simplificação constitui uma violação consciente da forma estrita da 1NF, justificada pelo âmbito do projeto. A equipa optou por priorizar a agilidade na implementação e a simplicidade estrutural, assumindo o armazenamento de apenas um contacto principal por registo, evitando a criação de tabelas adicionais. À exceção desta decisão deliberada, todos os outros campos garantem a atomicidade e a unicidade da chave primária.

Na **Segunda Forma Normal (2NF)**, todas as dependências parciais são eliminadas, o que significa que cada atributo na tabela deve depender totalmente da chave primária e não apenas de parte dela. Esta regra é fundamental nas tabelas associativas do nosso modelo. Por exemplo, na tabela **Patrocínio**, os atributos *Tipo* e *Valor* dependem da junção de `id_Evento` e `id_Patrocinador`. Da mesma forma, na tabela **Apresenta**, o atributo **Papel** (ex: moderador, palestrante) subordina-se à totalidade da chave composta (`id_Orador` e `id_Palestra`). Valida-se, assim, o cumprimento da 2NF, garantindo que não existem atributos que dependam apenas de uma parte da chave primária composta.

Na **Terceira Forma Normal (3NF)**, as dependências transitivas são eliminadas, ou seja, não deve haver atributos que dependam de outros que não a própria chave primária da tabela. Constata-se a veracidade desta norma no nosso modelo lógico; por exemplo, na tabela **Bilhete**, o atributo *Preço* está diretamente associado ao `id_Bilhete`. Não existem campos que dependam de outros atributos não-chave, assegurando que qualquer alteração num dado não cause inconsistências em cascata noutras partes da tabela.

Ao considerar a homologação sequencial e progressiva destas três formas normais, confirma-se a validade e robustez do modelo lógico do *ENEI-Gest*. Complementarmente, existem mais fórmulas como a Forma Normal de Boyce-Codd (BCNF) ou a Quarta e Quinta Formas Normais (4NF e 5NF), no entanto, descartámo-las dada a complexidade excessiva face aos requisitos do sistema em desenvolvimento.

4.4 Validação do Modelo com Interrogações do Utilizador

No âmbito do projeto, é fundamental garantir que o modelo lógico concebido satisfaz os requisitos de manipulação estabelecidos. Uma das formas de validação do modelo ocorre através da execução de consultas representadas em Álgebra Relacional. Esta abordagem teórica permite comprovar se a estrutura relacional definida consegue fornecer as respostas às operações e extracções de dados previstas no sistema.

Para cumprir este fim, apresentam-se de seguida seis expressões em Álgebra Relacional, acompanhadas pelas respetivas árvores de consulta, que validam diretamente os requisitos de manipulação definidos na Tabela 2.2:

1. Listar todos os eventos futuros com vagas disponíveis para inscrição (Requisito de Manipulação 1, Tabela 2.2):

$$\begin{array}{c}
 \text{Bilhete} \\
 \downarrow \\
 \sigma_{VagasDisponiveis > 0} \\
 \downarrow \\
 \bowtie_{Evento_id_Evento = id_Evento} \text{Evento} \\
 \downarrow \\
 \sigma_{DataInicio > DataAtual} \\
 \downarrow \\
 \pi_{id_Evento, Titulo, DataInicio, DataFim}
 \end{array}$$

Figura 9: Ilustração do primeiro diagrama de Álgebra Relacional.

$Bilhetes_Disponiveis \leftarrow \sigma_{VagasDisponiveis > 0}(Bilhete)$
 $Eventos_Com_Bilhete \leftarrow Bilhetes_Disponiveis \bowtie_{Evento_id_Evento = id_Evento} Evento$
 $Eventos_Futuros \leftarrow \sigma_{DataInicio > DataAtual}(Eventos_Com_Bilhete)$
 $\pi_{id_Evento, Titulo, DataInicio, DataFim}(Eventos_Futuros)$

2. Consultar o histórico completo de eventos em que um determinado participante se inscreveu (Requisito de Manipulação 2, Tabela 2.2):

Para o exemplo do utilizador com o ID número 1.

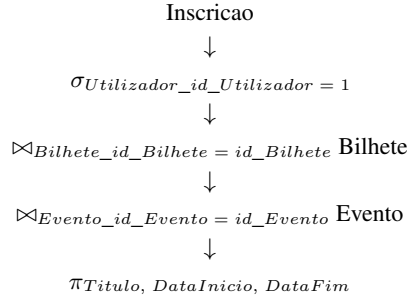


Figura 10: Ilustração do segundo diagrama de Álgebra Relacional.

$Insc_Utilizador \leftarrow \sigma_{Utilizador_id_Utilizador = 1}(Inscricao)$
 $Insc_Bilhete \leftarrow Insc_Utilizador \bowtie_{Bilhete_id_Bilhete = id_Bilhete} Bilhete$
 $Historico \leftarrow Insc_Bilhete \bowtie_{Evento_id_Evento = id_Evento} Evento$
 $\pi_{Titulo, DataInicio, DataFim}(Historico)$

3. Registrar o pagamento de uma inscrição e verificar o seu estado (Requisito de Manipulação 3, Tabela 2.2):

Para o exemplo da inscrição com o ID número 1.

Nota: A Álgebra Relacional é uma linguagem de consulta, pelo que apenas é possível modelar a operação de leitura de verificação do estado após o registo do pagamento, o qual é efetuado através de uma instrução de inserção na tabela Pagamento.

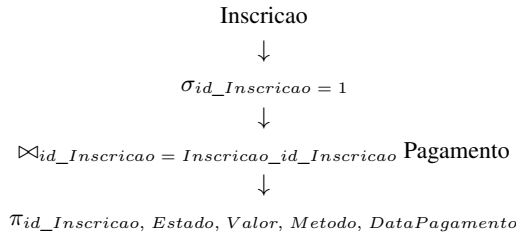


Figura 11: Ilustração do terceiro diagrama de Álgebra Relacional.

$Insc_Filtrada \leftarrow \sigma_{id_Inscricao = 1}(Inscricao)$
 $Insc_Paga \leftarrow Insc_Filtrada \bowtie_{id_Inscricao = Inscricao_id_Inscricao} Pagamento$
 $\pi_{id_Inscricao, Estado, Valor, Metodo, DataPagamento}(Insc_Paga)$

4. Gerar relatório com o valor total de receitas por evento, agrupado por tipo de bilhete (Requisito de Manipulação 4, Tabela 2.2):



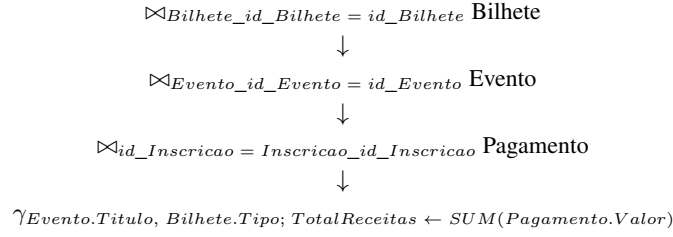


Figura 12: Ilustração do quarto diagrama de Álgebra Relacional.

$\text{Insc_Bilhete} \leftarrow \text{Inscricao} \bowtie_{\text{Bilhete_id_Bilhete} = \text{id_Bilhete}} \text{Bilhete}$
 $\text{Insc_Evento} \leftarrow \text{Insc_Bilhete} \bowtie_{\text{Evento_id_Evento} = \text{id_Evento}} \text{Evento}$
 $\text{Insc_Pag} \leftarrow \text{Insc_Evento} \bowtie_{\text{id_Inscricao} = \text{Inscricao_id_Inscricao}} \text{Pagamento}$
 $\gamma_{\text{Evento.Titulo, Bilhete.Tipo; TotalReceitas} \leftarrow \text{SUM}(\text{Pagamento.Valor})}(\text{Insc_Pag})$

5. Obter a lista cronológica das palestras agendadas para um evento, incluindo os respetivos oradores (*Requisito de Manipulação 5, Tabela 2.2*):

Para o exemplo do evento com o ID número 1.



Figura 13: Ilustração do quinto diagrama de Álgebra Relacional.

$\text{Sessoes_Ev} \leftarrow \sigma_{\text{Evento_id_Evento} = 1}(\text{Sessao})$
 $\text{Palestras_Ev} \leftarrow \text{Sessoes_Ev} \bowtie_{\text{id_Sessao} = \text{Sessao_id_Sessao}} \text{Palestra}$
 $\text{Apres_Ev} \leftarrow \text{Palestras_Ev} \bowtie_{\text{id_Palestra} = \text{Palestra_id_Palestra}} \text{Apresenta}$
 $\text{Com_Orador} \leftarrow \text{Apres_Ev} \bowtie_{\text{Orador_id_Orador} = \text{id_Orador}} \text{Orador}$
 $\text{Ordenado} \leftarrow \tau_{\text{Palestra.HoraInicio ASC}}(\text{Com_Orador})$
 $\pi_{\text{Palestra.Titulo, Palestra.HoraInicio, Palestra.HoraFim, Orador.Nome}}(\text{Ordenado})$

6. Consultar os dados de contacto de um participante específico (*Requisito de Manipulação 6, Tabela 2.2*):

Para o exemplo do utilizador com o ID número 1.

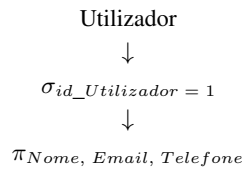


Figura 14: Ilustração do sexto diagrama de Álgebra Relacional.

$\text{Utilizador_Filtrado} \leftarrow \sigma_{\text{id_Utilizador} = 1}(\text{Utilizador})$
 $\pi_{\text{Nome, Email, Telefone}}(\text{Utilizador_Filtrado})$


```
CREATE DATABASE IF NOT EXISTS Enei_Gest;

USE Enei_Gest;
```

Figura 4.2 Instruções SQL para criar e utilizar a base de dados.

A primeira tabela a ser implementada no modelo físico é o Utilizador, esta que, ao seguir a estrutura habitual de SQL, é inaugurada com cinco colunas. Possui o identificador único "Id", que é definido pelos parâmetros "INT", que define o tipo de dados, "PRIMARY KEY" que garante a atomicidade e capacidade identificativa, e "AUTO_INCREMENT" para que o sistema atribua automaticamente um identificador sequencial. Representam-se também os atributos textuais "Nome" e "Email", ambos do tipo "VARCHAR(100)", apoiados pelo "NOT NULL", de forma a certificar a atribuição obrigatória de valor. A "DataRegisto" adota o tipo "DATETIME" e a restrição "NOT NULL" para garantir a cronologia exata de criação da conta. Por fim, a coluna "Telefone" é estruturada como "VARCHAR(20)", sendo a única sem a restrição de preenchimento obrigatório, permitindo valores nulos.



Figura 4.3 Tabela lógica do Utilizador.

```
create table Utilizador(
    Id int auto_increment,
    Nome varchar(100) not null,
    Email varchar(100) not null,
    DataRegisto datetime not null,
    Telefone varchar(20),
    primary key(Id)
);
```

Figura 4.4 Implementação correspondente ao Utilizador.

A tabela Organizador armazena os dados essenciais das entidades promotoras responsáveis pelos eventos. A coluna "Id" é um "INT" que serve como "PRIMARY KEY" e possui a propriedade "AUTO_INCREMENT", garantindo que o sistema atribua um identificador único e sequencial a cada nova empresa registada. A coluna "NomeEmpresa" é um "VARCHAR(100)" e "NOT NULL", permitindo guardar a designação comercial da entidade com até 100 caracteres de forma obrigatória. O "Email" é também um campo "VARCHAR(100)" e "NOT NULL", registrando o endereço de correio eletrónico oficial para comunicações. Por fim, "Contacto" é um "VARCHAR(20)" e, por não ter a restrição de obrigatoriedade, pode ser "NULL", indicando que a submissão de um número de telefone é opcional nesta fase.

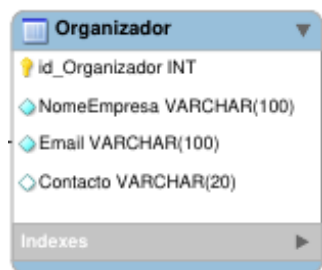


Figura 4.5 Tabela lógica do Organizador.

```
create table Organizador(
    Id int auto_increment,
    NomeEmpresa varchar(100) not null,
    Email varchar(100) not null,
    Contacto varchar(20),
    primary key(Id)
);
```

Figura 4.6 Implementação correspondente ao Organizador.

De seguida, apresenta-se a tabela Local, responsável por armazenar a informação relativa aos recintos e espaços físicos onde ocorrem as atividades. Importa destacar um pormenor técnico relevante na sua declaração: a utilização de acentos graves (crases) a envolver o nome da tabela (`Local `). Esta prática foi adotada de forma deliberada, uma vez que a

palavra "local" é uma palavra reservada no léxico do MySQL. Ao envolvê-la nestes caracteres, indicamos ao sistema que se trata de um identificador legítimo da base de dados, contornando assim potenciais conflitos ou erros de sintaxe.

Estruturalmente, a coluna "Id" é do tipo "INT", assumindo o papel de "PRIMARY KEY" com o atributo "AUTO_INCREMENT" para a geração sequencial e automática do identificador do espaço. Os atributos "Nome" e "Morada" são definidos como "VARCHAR(100)" e marcados como "NOT NULL", assegurando o registo obrigatório da designação e da localização exata do recinto. Por último, a coluna "LotacaoMaxima" adota o tipo "INT" acompanhado da restrição "NOT NULL", um dado imprescindível para o controlo logístico de capacidade do evento e para a posterior emissão e gestão de bilhetes disponíveis.

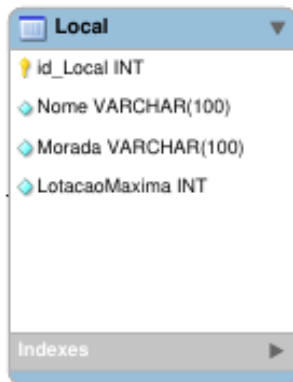


Figura 4.7 Tabela lógica do Local.

```
create table `Local` (  
    Id Int auto_increment,  
    Nome varchar(100) not null,  
    Morada varchar(100) not null,  
    LotacaoMaxima Int not null,  
    primary key(Id)  
);
```

Figura 4.8 Implementação correspondente ao Local.

A tabela Orador destina-se a armazenar os dados relativos aos especialistas e convidados que conduzirão as palestras. A coluna "Id" assume o tipo "INT" e atua como "PRIMARY KEY", incorporando o parâmetro "AUTO_INCREMENT" para a atribuição sequencial e automática de um identificador único a cada registo. O atributo "Nome" é definido como "VARCHAR(100)" com a restrição "NOT NULL", assegurando o preenchimento obrigatório da identidade do orador.

Relativamente às restantes colunas, denota-se uma flexibilidade propositada na implementação SQL: o "Email" (do tipo "VARCHAR(100)"), a "Biografia" (do tipo "TEXT", utilizado por permitir blocos de caracteres mais extensos) e a "InstituicaoEmpresa" (do tipo "VARCHAR(100)") não apresentam a diretiva de preenchimento obrigatório ("NOT NULL"). Isto permite a introdução de valores nulos, acautelando as situações em que um orador não faculte o seu currículo ou afiliação no momento inicial do registo no sistema.

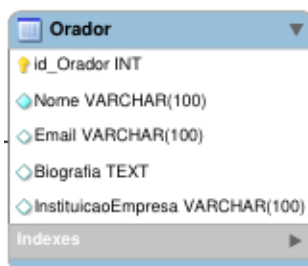


Figura 4.9 Tabela lógica do Orador.

```
create table Orador(  
    Id Int auto_increment,  
    Nome varchar(100) not null,  
    Email varchar(100),  
    Biografia Text,  
    InstituicaoEmpresa varchar(100),  
    primary key(Id)  
);
```

Figura 4.10 Implementação correspondente ao Orador.

A tabela Patrocinador tem como objetivo guardar os registos das empresas parceiras e financiadoras do evento. A coluna "Id" assume o tipo "INT" e é definida como "PRIMARY KEY", recorrendo também ao atributo "AUTO_INCREMENT" para gerar um identificador sequencial único de forma automática para cada novo parceiro. A coluna "NomeEmpresa" foi tipificada como "VARCHAR(100)" e aliada à restrição "NOT NULL", obrigando ao registo da designação comercial da entidade.

Destaca-se a definição da coluna "NIF" (estruturada como "VARCHAR(20)" e "NOT NULL"), à qual foi inteligentemente adicionada a cláusula "UNIQUE". Este é um pormenor técnico de extrema importância na modelação física, pois garante a integridade e unicidade dos dados ao nível do motor de base de dados, impedindo que sejam registadas duas empresas diferentes com o mesmo Número de Identificação Fiscal. Por fim, o atributo "PessoaContacto" (também "VARCHAR(100)") não impõe a restrição de obrigatoriedade, permitindo valores nulos caso o nome do representante da empresa parceira não seja imediatamente fornecido no momento do registo.

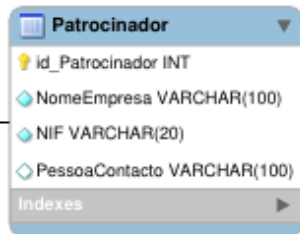


Figura 4.11 Tabela lógica do Patrocinador.

```
CREATE TABLE Patrocinador (
    Id INT AUTO_INCREMENT,
    NomeEmpresa VARCHAR(100) NOT NULL,
    NIF VARCHAR(20) NOT NULL UNIQUE,
    PessoaContacto VARCHAR(100),
    PRIMARY KEY(Id)
);
```

Figura 4.12 Implementação correspondente ao Patrocinador.

Com as entidades de suporte previamente estabelecidas, procede-se à implementação da tabela central do sistema: o Evento. A coluna "Id" assume o tipo "INT" e é definida como "PRIMARY KEY" com o parâmetro "AUTO_INCREMENT", assegurando um identificador sequencial exclusivo. Os atributos descritivos e temporais, nomeadamente o "Titulo" (tipo "VARCHAR(150)"), a "DataInicio" e a "DataFim" (ambos do tipo "DATETIME"), bem como a "CapacidadeMaxima" (tipo "INT"), estão todos acompanhados pela restrição "NOT NULL", garantindo que nenhum evento seja criado sem a sua definição estrutural e temporal completa.

O grande destaque técnico desta implementação reside na declaração das dependências relacionais. As colunas "Organizador_id_Organizador" e "Local_id_Local" (ambas "INT" e "NOT NULL") são explicitamente definidas através da cláusula "FOREIGN KEY". Estas chaves estrangeiras referenciam, respetivamente, as chaves primárias ("Id") das tabelas Organizador e Local. Esta estrutura é fundamental para garantir a integridade referencial da base de dados, impossibilitando a criação de um evento órfão, ou seja, sem um promotor responsável ou um recinto físico válido e previamente registado no sistema.



Figura 4.13 Tabela lógica do Evento.

```
CREATE TABLE Evento (
    Id INT AUTO_INCREMENT,
    Titulo VARCHAR(150) NOT NULL,
    DataInicio DATETIME NOT NULL,
    DataFim DATETIME NOT NULL,
    CapacidadeMaxima INT NOT NULL,
    Organizador_id_Organizador INT NOT NULL,
    Local_id_Local INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Organizador_id_Organizador) REFERENCES Organizador(Id),
    FOREIGN KEY(Local_id_Local) REFERENCES Local(Id)
);
```

Figura 4.14 Implementação correspondente ao Evento.

Avançando para a estruturação da agenda, apresenta-se a tabela Sessao, que representa os diferentes blocos de atividades disponíveis. A coluna "Id" mantém o padrão do sistema, sendo tipificada como "INT" com a propriedade "AUTO_INCREMENT" e designada como "PRIMARY KEY". Os dados fundamentais de cada bloco são guardados nas colunas "NomeAtividade" (do tipo "VARCHAR(100)"), "HoraInicio" e "HoraFim" (ambas do tipo "DATETIME"). Todos estes atributos possuem a restrição "NOT NULL", assegurando que qualquer sessão criada no sistema tenha obrigatoriamente um nome e um intervalo cronológico exato e definido.

Para garantir o enquadramento organizacional, a tabela inclui a coluna "Evento_id_Evento" (do tipo "INT" e "NOT

NULL"). Através da cláusula "FOREIGN KEY", este atributo estabelece uma chave estrangeira que referencia diretamente a chave primária ("Id") da tabela Evento. Esta dependência traduz tecnicamente a regra de negócio delineada na fase conceptual: uma sessão não pode existir de forma flutuante ou isolada, tendo de estar obrigatoriamente associada a um evento previamente registado na base de dados.

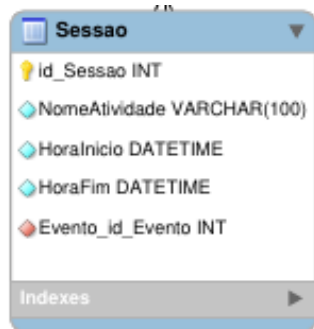


Figura 4.15 Tabela lógica da Sessão.

```
CREATE TABLE Sessao (
    Id INT AUTO_INCREMENT,
    NomeAtividade VARCHAR(100) NOT NULL,
    HoraInicio DATETIME NOT NULL,
    HoraFim DATETIME NOT NULL,
    Evento_id_Evento INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Evento_id_Evento) REFERENCES Evento(Id)
);
```

Figura 4.16 Implementação correspondente à Sessão.

Descendo mais um nível na hierarquia organizacional do evento, apresenta-se a tabela *Palestra*, concebida para detalhar as apresentações e intervenções específicas que ocorrem dentro de um determinado bloco de agenda. A coluna "Id" mantém o padrão estrutural adotado, assumindo o papel de *PRIMARY KEY* do tipo *INT* com o parâmetro *AUTO_INCREMENT*. O "Título" da palestra (*VARCHAR(100)*) e os respetivos limites temporais, representados pelas colunas "HoraInicio" e "HoraFim" (ambas *DATETIME*), estão acompanhados da restrição *NOT NULL*, garantindo uma planificação cronológica rigorosa e obrigatória.

A coluna "Resumo" adota o tipo de dados *TEXT*, uma escolha adequada para armazenar parágrafos descritivos ou a sinopse da intervenção, não possuindo a diretiva *NOT NULL* de forma a permitir o registo inicial de uma palestra antes de a sua descrição estar disponível. Por fim, a integridade referencial do modelo é assegurada pela declaração da *FOREIGN KEY* na coluna "Sessao_id_Sessao" (*INT* e *NOT NULL*), que referencia o "Id" da tabela *Sessao*, garantindo que toda e qualquer palestra esteja obrigatoriamente vinculada a um bloco de agenda previamente registado no sistema.

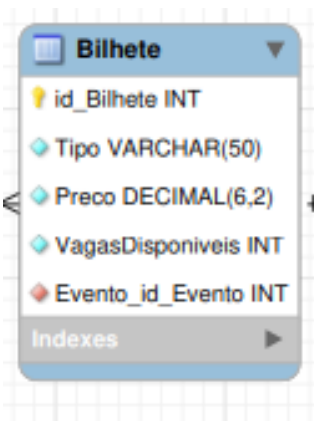


Figura 4.17 Tabela lógica do Bilhete.

```
CREATE TABLE Bilhete (
    Id INT AUTO_INCREMENT,
    Tipo VARCHAR(50) NOT NULL,
    Preco DECIMAL(6,2) NOT NULL,
    VagasDisponiveis INT NOT NULL,
    Evento_id_Evento INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Evento_id_Evento) REFERENCES Evento(Id)
);
```

Figura 4.18 Implementação correspondente ao Bilhete.

Descendo mais um nível na hierarquia organizacional do evento, procedemos à implementação da tabela *Palestra*, concebida para detalhar as apresentações ou intervenções específicas que ocorrem dentro de um determinado bloco de agenda. A coluna "Id" reitera o padrão estrutural, assumindo o papel de "PRIMARY KEY" do tipo "INT" com o parâmetro "AUTO_INCREMENT". O "Título" da palestra ("VARCHAR(100)") e os seus respetivos limites temporais, representados pelas colunas "HoraInicio" e "HoraFim" (ambas "DATETIME"), apresentam a restrição "NOT NULL", garantindo uma planificação cronológica rigorosa e obrigatória.

Por outro lado, a coluna "Resumo" adota o tipo de dados "TEXT", uma escolha adequada para armazenar parágrafos

descritivos ou a sinopse da intervenção, não possuindo a diretiva "NOT NULL" para tolerar submissões iniciais onde essa informação ainda não esteja disponível. Finalmente, a integridade referencial do modelo volta a ser assegurada pela declaração da "FOREIGN KEY" na coluna "Sessao_id_Sessao" ("INT" e "NOT NULL"). Esta chave estrangeira referencia o "Id" da tabela Sessao, garantindo que toda e qualquer palestra esteja estritamente vinculada e contida num bloco de agenda previamente validado pelo sistema.



Figura 4.19 Tabela lógica da Palestra.

```
CREATE TABLE Palestra (
    Id INT AUTO_INCREMENT,
    Titulo VARCHAR(100) NOT NULL,
    Resumo TEXT,
    HoraInicio DATETIME NOT NULL,
    HoraFim DATETIME NOT NULL,
    Sessao_id_Sessao INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Sessao_id_Sessao) REFERENCES Sessao(Id)
);
```

Figura 4.20 Implementação correspondente à Palestra.

Para materializar o processo de participação nos eventos, apresenta-se a tabela Inscricao. À semelhança das instâncias anteriores, a coluna "Id" assume o tipo "INT", definida como "PRIMARY KEY" com o parâmetro "AUTO_INCREMENT" para a identificação única e sequencial de cada registo. Para efeitos de auditoria e controlo logístico, a "DataInscricao" utiliza o tipo "DATETIME", enquanto a coluna "Estado" adota o tipo "VARCHAR(20)" (permitindo guardar valores como 'Pendente' ou 'Confirmado'). Ambas estão acompanhadas pela restrição "NOT NULL", garantindo que não existam inscrições no sistema sem uma validação temporal e de estado.

O aspeto central desta tabela reside na sua função de intersecção relacional. Para tal, recorre a duas dependências cruciais: "Utilizador_id_Utilizador" e "Bilhete_id_Bilhete" (ambas "INT" e "NOT NULL"). Através da instrução "FOREIGN KEY", estas colunas referenciam diretamente as chaves primárias ("Id") das tabelas Utilizador e Bilhete, respetivamente. Esta dupla subordinação assegura a integridade referencial do sistema, certificando de forma intransigente que uma inscrição tem de estar associada a um utilizador validado e a um bilhete existente.



Figura 4.21 Tabela lógica da Inscricao.

```
CREATE TABLE Inscricao (
    Id INT AUTO_INCREMENT,
    DataInscricao DATETIME NOT NULL,
    Estado VARCHAR(20) NOT NULL,
    Utilizador_id_Utilizador INT NOT NULL,
    Bilhete_id_Bilhete INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Utilizador_id_Utilizador) REFERENCES Utilizador(Id),
    FOREIGN KEY(Bilhete_id_Bilhete) REFERENCES Bilhete(Id)
);
```

Figura 4.22 Implementação correspondente à Inscricao.

Complementando a componente de participação com a gestão financeira, apresenta-se a tabela Pagamento. A coluna "Id" mantém o referencial estrutural adotado, sendo do tipo "INT", designada como "PRIMARY KEY" e dotada do parâmetro "AUTO_INCREMENT" para a numeração automática e unívoca de cada transação. O montante da transação é armazenado no atributo "Valor", recorrendo ao tipo "DECIMAL(6,2)" associado à restrição "NOT NULL", garantindo a precisão contabilística exigida para o manuseamento de moeda.

Para efeitos de histórico e auditoria, o "Metodo" de pagamento (do tipo "VARCHAR(50)") e a respetiva "DataPagamento" (do tipo "DATETIME") são igualmente de preenchimento obrigatório ("NOT NULL"), assegurando a rastreabilidade

total de como e quando a liquidação ocorreu. A coesão deste ecossistema financeiro é garantida pela coluna "Inscricao_id_Inscricao" ("INT" e "NOT NULL"), estabelecida sob a cláusula de "FOREIGN KEY". Esta chave estrangeira referencia diretamente o "Id" da tabela Inscricao, certificando de forma rigorosa que não existem pagamentos isolados ou órfãos, estando cada transação inexoravelmente justificada por uma inscrição efetiva no sistema.



Figura 4.23 Tabela lógica do Pagamento.

```
CREATE TABLE Pagamento (
    Id INT AUTO_INCREMENT,
    Valor DECIMAL(6,2) NOT NULL,
    Metodo VARCHAR(50) NOT NULL,
    DataPagamento DATETIME NOT NULL,
    Inscricao_id_Inscricao INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Inscricao_id_Inscricao) REFERENCES Inscricao(Id)
);
```

Figura 4.24 Implementação correspondente ao Pagamento.

Chegando às tabelas associativas do modelo, responsáveis por resolver os relacionamentos de cardinalidade N para M (muitos-para-muitos), apresenta-se a tabela Patrocinio. O grande destaque técnico desta implementação é a ausência de um "Id" singular e a utilização de uma chave primária composta (*Composite Primary Key*). As colunas "Evento_id_Evento" e "Patrocinador_id_Patrocinador" (ambas "INT" e "NOT NULL") unem-se na declaração "PRIMARY KEY" para identificar de forma unívoca cada acordo de financiamento.

Simultaneamente, estas duas colunas assumem o papel de chaves estrangeiras ("FOREIGN KEY"), referenciando as tabelas Evento e Patrocinador. Esta dupla função garante que um patrocínio só pode ser registrado se associar uma entidade parceira e um evento que já existam previamente no sistema. Relativamente aos atributos próprios deste relacionamento — o "Tipo" ("VARCHAR(45)"), a "DataContrato" ("DATETIME") e o "Valor" ("DECIMAL(6,2)") para assegurar a precisão financeira —, denota-se a ausência da restrição "NOT NULL". Esta decisão de modelação oferece a flexibilidade necessária para registrar intenções de patrocínio no sistema cujos detalhes contratuais ou tranches financeiras exatas ainda se encontrem em fase de negociação.



Figura 4.25 Tabela lógica do Patrocínio.

```
CREATE TABLE Patrocinio (
    Evento_id_Evento INT NOT NULL,
    Patrocinador_id_Patrocinador INT NOT NULL,
    Tipo VARCHAR(45),
    DataContrato datetime,
    Valor Decimal(6,2),
    PRIMARY KEY(Evento_id_Evento, Patrocinador_id_Patrocinador),
    FOREIGN KEY(Evento_id_Evento) REFERENCES Evento(Id),
    FOREIGN KEY(Patrocinador_id_Patrocinador) REFERENCES Patrocinador(Id)
);
```

Figura 4.26 Implementação correspondente ao Patrocínio.

Para finalizar a transposição do modelo lógico para o modelo físico, apresenta-se a segunda tabela associativa do sistema: Apresenta. À semelhança da estrutura de patrocínios, esta tabela tem como propósito a resolução do relacionamento de cardinalidade N para M (muitos-para-muitos) identificado no modelo conceptual, neste caso, entre os oradores e as palestras em que participam.

Técnica e estruturalmente, a tabela abdica de um identificador singular em prol de uma chave primária composta (*Composite Primary Key*). As colunas "Orador_id_Orador" e "Palestra_id_Palestra" (tipificadas como "INT" e "NOT NULL") unem-se para formar o identificador único de cada participação. Simultaneamente, operam como chaves estrangeiras ("FOREIGN KEY"), referenciando as tabelas originais Orador e Palestra. Este mecanismo blindado de integridade referencial garante que nenhuma apresentação pode ser agendada sem que o respetivo interveniente e o bloco da palestra já constem na base de dados. Por fim, o atributo "Papel" ("VARCHAR(45)") não possui a restrição

"NOT NULL", conferindo flexibilidade ao sistema para registar participações cujas funções exatas (ex: orador principal, moderador, comentador de painel) ainda estejam por definir.

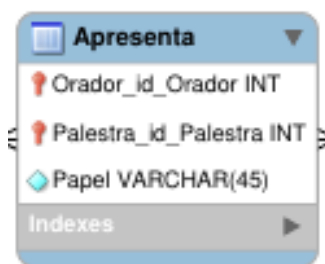


Figura 4.27 Tabela lógica de Apresenta.

```
CREATE TABLE Apresenta(
    Orador_id_Orador INT NOT NULL,
    Palestra_id_Palestra INT NOT NULL,
    Papel VARCHAR(45),
    PRIMARY KEY(Orador_id_Orador, Palestra_id_Palestra),
    FOREIGN KEY(Orador_id_Orador) REFERENCES Orador(Id),
    FOREIGN KEY(Palestra_id_Palestra) REFERENCES Palestra(Id)
);
```

Figura 4.28 Implementação correspondente a Apresenta.

4.5 Criação de utilizadores da base de dados

Beneficiando agora dos requisitos de controlo recolhidos inicialmente, a equipa passou a desenvolver e definir toda a caracterização relativa aos utilizadores da base de dados em questão. Neste caso, os únicos utilizadores a serem criados no sistema seriam os gestores de conteúdos e os gestores de participantes. Seguida da criação dos mesmos, são definidas as suas permissões de trabalho na base de dados, de forma a garantir a segurança e integridade da mesma.

Na base de dados, o utilizador "gestor_conteudos" foi criado com permissões limitadas ao controlo da agenda e logística do evento. Para tal, a este utilizador são atribuídas permissões de "SELECT", "INSERT" e "UPDATE" nas tabelas Evento, Sessao, Palestra, Orador, Local e Organizador. Para além disso, também conta com permissões de execução do procedimento GerarAgendaFinal.

```
CREATE USER 'gestor_conteudos'@'localhost' IDENTIFIED BY 'Conteudos@123';

GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Evento TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Sessao TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Palestra TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Orador TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Local TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Organizador TO 'gestor_conteudos'@'localhost';
GRANT EXECUTE ON PROCEDURE Enei_Gest.GerarAgendaFinal TO 'gestor_conteudos'@'localhost';
```

Figura 4.29 Criação do utilizador "gestor_conteudos" e devidas permissões.

Foi também criado o utilizador "gestor_participantes" com permissões limitadas ao controlo de inscrições, bilhética e vertente financeira. Assim, o mesmo tem acesso às tabelas Utilizador, Inscricao, Bilhete, Pagamento, Patrocinador e Patrocinio, onde está restrito às operações de "SELECT", "INSERT" e "UPDATE" nas mesmas. Adicionalmente, também dispõe da permissão de execução da função CalcularReceitaTotal.

```
CREATE USER 'gestor_participantes'@'localhost' IDENTIFIED BY 'Participantes@123';

GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Utilizador TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Inscricao TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Bilhete TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Pagamento TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Patrocinador TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Patrocinio TO 'gestor_participantes'@'localhost';
GRANT EXECUTE ON FUNCTION Enei_Gest.CalcularReceitaTotal TO 'gestor_participantes'@'localhost';
```

Figura 4.30 Criação do utilizador "gestor_participantes" e devidas permissões.

Desta forma, ficam definidos os utilizadores da base de dados, assim como as suas permissões. Os gestores de conteúdos apenas têm autorização para manipular a estrutura física e cronológica do evento, enquanto que os gestores de participantes detêm o controlo exclusivo sobre o fluxo de pessoas e parcerias monetárias, garantindo uma separação clara e segura de responsabilidades dentro do sistema.

4.6 Povoamento da base de dados

Para o povoamento da base de dados, implementámos uma abordagem baseada na execução de um ficheiro SQL, onde, através de instruções `INSERT INTO`, inserimos um conjunto representativo de entradas para facilitar a visualização e a execução de *queries*. Esta abordagem permite uma compreensão mais clara das operações básicas da base de dados e assegura que a integridade referencial do modelo é rigorosamente testada, respeitando a ordem de dependência das chaves estrangeiras.

Inicia-se o processo pela introdução de dados nas entidades base do sistema (tabelas independentes). A primeira é a tabela "Utilizador", que contém informações sobre os estudantes e participantes registados na plataforma. Adicionam-se diversos utilizadores, especificando o seu nome, endereço eletrónico, momento exato de registo e, de forma opcional, o contacto telefónico.

```
INSERT INTO Utilizador (Nome, Email, DataRegisto, Telefone) VALUES
('João Silva', 'joao.silva@email.com', '2024-01-10 10:30:00', '912345678'),
('Ana Rita Santos', 'ana.santos@email.com', '2024-01-12 14:15:00', '961234567'),
('Carlos Mendes', 'carlos.m@estudante.uminho.pt', '2024-02-15 09:20:00', '933445566'),
('Beatriz Sousa', 'bsousa@estudante.feup.pt', '2024-02-18 11:45:00', '911223344'),
('Tiago Ribeiro', 'tiago.ribeiro@gmail.com', '2024-03-01 16:10:00', '966778899'),
('Mariana Lima', 'mariana.l@isep.ipp.pt', '2024-03-05 14:00:00', '922334455'),
('Pedro Gonçalves', 'pedro.goncalves@dev.com', '2024-03-10 18:30:00', NULL);
```

Figura 4.31 Inserção de dados na tabela Utilizador com SQL.

De seguida, procedemos à inserção na tabela "Organizador", responsável por guardar os dados das entidades promotoras e associações de estudantes, registando o nome da instituição, email institucional e contacto telefónico.

```
INSERT INTO Organizador (NomeEmpresa, Email, Contacto) VALUES
('Tech Events Portugal', 'geral@techevents.pt', '210987654'),
('Cultura Porto', 'info@culturaporto.pt', '220123456'),
('Núcleo de Estudantes de Ciências da Computação', 'geral@necc.pt', '253112233'),
('Associação Académica', 'eventos@academica.pt', '210001122');
```

Figura 4.32 Inserção de dados na tabela Organizador com SQL.

A tabela "Local" é também povoada nesta fase inicial, armazenando os recintos e pavilhões disponíveis para o acolhimento das atividades, acompanhados das respetivas moradas e da sua lotação máxima permitida.

```
INSERT INTO `Local` (Nome, Morada, LotacaoMaxima) VALUES
('Auditório Principal FEUP', 'Porto', 500),
('Altice Arena', 'Lisboa', 2000),
('Pavilhão Rosa Mota', 'Jardins do Palácio de Cristal, Porto', 4000),
('Sala B204 - FEUP', 'Rua Dr. Roberto Frias, Porto', 120);
```

Figura 4.33 Inserção de dados na tabela Local com SQL.

De igual modo, registam-se os dados na tabela "Orador", introduzindo os especialistas, professores e convidados que conduzirão as atividades. São armazenados os seus contactos, uma breve biografia e a sua instituição ou empresa de origem.


```
INSERT INTO Orador (Nome, Email, Biografia, InstituicaoEmpresa) VALUES
('Maria Fernandes', 'maria.f@tech.com', 'Especialista em IA.', 'TechCorp'),
('Rui Costa', 'rui.costa@univ.pt', 'Professor Catedrático.', 'Universidade do Porto'),
('Joana Batista', 'jbatista@microsoft.com', 'Cloud Architect e MVP.', 'Microsoft'),
('Nuno Marques', 'nuno.marques@blip.pt', 'Lead Software Engineer.', 'Blip'),
('Sara Oliveira', 'sara.o@deloitte.pt', 'Consultora de Cibersegurança.', 'Deloitte');
```

Figura 4.34 Inserção de dados na tabela Orador com SQL.

Ainda no leque de entidades independentes, a tabela "Patrocinador" armazena informações detalhadas sobre as empresas parceiras que financiam ou apoiam os eventos, incluindo a sua designação comercial, o NIF exclusivo e o respetivo representante de contacto.

```
INSERT INTO Patrocinador (NomeEmpresa, NIF, PessoaContacto) VALUES
('MEO Empresas', '500500500', 'Miguel Almeida'),
('Worten', '501501501', 'Sofia Lemos'),
('Microsoft Portugal', '502502502', 'Rita Lemos'),
('Deloitte', '503503503', 'Hugo Silva'),
('Blip', '504504504', 'Inês Castro');
```

Figura 4.35 Inserção de dados na tabela Patrocinador com SQL.

Garantida a base estrutural, avança-se para as tabelas com dependências relacionais. A tabela "Evento" regista as iniciativas e conferências globais planeadas, incluindo os seus limites temporais e capacidade máxima. Este passo vincula cada evento a um organizador e a um local previamente registados no sistema.

```
INSERT INTO Evento (Titulo, DataInicio, DataFim, CapacidadeMaxima, Organizador_id_Organizador, Local_id_Local) VALUES
('Future Tech Conference', '2024-05-10 09:00:00', '2024-05-11 18:00:00', 400, 1, 1),
('Lisbon Cultural Fest', '2024-09-20 10:00:00', '2024-09-22 20:00:00', 1500, 2, 2),
('Hackathon Code4Good', '2024-11-15 18:00:00', '2024-11-17 18:00:00', 200, 3, 4),
('ENEI 2025 - Encontro Nacional', '2025-03-20 09:00:00', '2025-03-23 20:00:00', 3500, 1, 3);
```

Figura 4.36 Inserção de dados na tabela Evento com SQL.

Na tabela "Sessao", registamos os blocos cronológicos de agenda, associando horários específicos de início e fim a cada atividade e garantindo que todas as sessões ficam indissociavelmente ligadas ao evento a que pertencem.

```
INSERT INTO Sessao (NomeAtividade, HoraInicio, HoraFim, Evento_id_Evento) VALUES
('Abertura e Keynotes', '2024-05-10 09:00:00', '2024-05-10 12:30:00', 1),
('Painel de Cibersegurança', '2025-03-21 14:00:00', '2025-03-21 16:00:00', 4),
('Workshops de Programação', '2025-03-22 10:00:00', '2025-03-22 13:00:00', 4),
('Apresentação de Projetos (Pitches)', '2024-11-17 15:00:00', '2024-11-17 17:30:00', 3);
```

Figura 4.37 Inserção de dados na tabela Sessao com SQL.

A tabela "Bilhete" armazena as modalidades de ingresso disponíveis para cada certame, como passes gerais, de estudante ou VIP, estabelecendo uma ligação direta entre a tipologia, o seu valor monetário e o ID do evento correspondente.

```
INSERT INTO Bilhete (Tipo, Preco, Evento_id_Evento) VALUES
('Geral', 50.00, 1),
('VIP', 100.00, 1),
('Estudante', 15.00, 4),
('Profissional', 60.00, 4),
('Equipa Hackathon (4 pax)', 20.00, 3);
```

Figura 4.38 Inserção de dados na tabela Bilhete com SQL.

Na tabela "Palestra", registam-se os tópicos e palestras específicas que serão apresentadas. Cada palestra é alocada a uma sessão existente, detalhando a agenda do evento com os respetivos resumos e horários precisos.

```
INSERT INTO Palestra (Titulo, Resumo, HoraInicio, HoraFim, Sessao_id_Sessao) VALUES
('O Futuro da IA', 'Impacto da IA nas empresas.', '2024-05-10 10:00:00', '2024-05-10 11:00:00', 1),
('Ataques Ransomware em 2024', 'Análise dos maiores ataques e como prevenir.', '2025-03-21 14:15:00', '2025-03-21 15:00:00', 2),
('Escalar com Kubernetes', 'Workshop prático de DevOps e Cloud.', '2025-03-22 10:00:00', '2025-03-22 12:00:00', 3);
```

Figura 4.39 Inserção de dados na tabela Palestra com SQL.

Entrando na fase de gestão de participantes, a tabela "Inscricao" cruza a informação dos utilizadores com os bilhetes que pretendem adquirir, registrando a data exata do pedido e o seu estado de aprovação, garantindo o correto controlo de acessos.

```
INSERT INTO Inscricao (DataInscricao, Estado, Utilizador_id_Utilizador, Bilhete_id_Bilhete) VALUES
('2024-02-01 10:00:00', 'Confirmada', 1, 1),
('2024-02-05 15:30:00', 'Pendente', 2, 2),
('2024-03-10 10:00:00', 'Confirmada', 3, 3),
('2024-03-15 14:20:00', 'Confirmada', 4, 3);
```

Figura 4.40 Inserção de dados na tabela Inscricao com SQL.

De modo a acompanhar e validar as inscrições pendentes, a tabela "Pagamento" regista as liquidações efetuadas. Cada entrada guarda o valor cobrado, o método de transação utilizado e o identificador da inscrição suportada.

```
INSERT INTO Pagamento (Valor, Metodo, DataPagamento, Inscricao_id_Inscricao) VALUES
(50.00, 'MBWay', '2024-02-01 10:05:00', 1),
(15.00, 'Referência Multibanco', '2024-03-10 10:15:00', 3),
(15.00, 'MBWay', '2024-03-15 14:25:00', 4);
```

Figura 4.41 Inserção de dados na tabela Pagamento com SQL.

As tabelas puramente associativas encerram o processo de povoamento. A tabela "Patrocinio" regista os acordos financeiros, estabelecendo a relação N para M entre parceiros e eventos, caracterizando o tipo de apoio prestado e o valor do contrato.

```
INSERT INTO Patrocinio (Evento_id_Evento, Patrocinador_id_Patrocinador, Tipo, DataContrato, Valor) VALUES
(1, 1, 'Principal', '2024-01-15 10:00:00', 5000.00),
(1, 2, 'Apoio Prata', '2024-02-10 14:00:00', 1500.00),
(4, 3, 'Sponsor Gold', '2025-01-10 11:00:00', 10000.00),
(4, 4, 'Sponsor Silver', '2025-01-15 15:30:00', 5000.00),
(3, 5, 'Apoio Tecnológico', '2024-09-01 10:00:00', 2000.00);
```

Figura 4.42 Inserção de dados na tabela Patrocinio com SQL.

Por fim, a tabela "Apresenta" materializa a participação dos especialistas na agenda do evento, interligando o Orador à sua respectiva Palestra, e especificando o papel que irá desempenhar (por exemplo, moderador, formador ou orador principal).

```
INSERT INTO Apresenta (Orador_id_Orador, Palestra_id_Palestra, Papel) VALUES
(1, 1, 'Orador Principal'),
(2, 1, 'Moderador'),
(5, 2, 'Orador Principal'),
(3, 3, 'Formador'),
(4, 3, 'Assistente');
```

Figura 4.43 Inserção de dados na tabela Apresenta com SQL.

4.7 Cálculo do espaço da base de dados (inicial e taxa de crescimento anual)

De forma a perceber o espaço que a base de dados pode vir a ocupar, calculamos inicialmente o mesmo quando existe apenas um registo por tabela e, numa fase final, estimou-se o espaço total necessário para armazenar toda a informação contida na base de dados, assim como a informação que surgirá com um determinado crescimento anual esperado.

Assim, representa-se, a seguir, uma tabela que evidencia os tipos de dados usados no sistema ENEI-Gest, acompanhados pelos seus tamanhos respetivos em *bytes*, de acordo com o manual de referência do MySQL 8.0.

Tipo de dados	Tamanho (bytes)
INT	4
DATETIME	8
DECIMAL(M,D)	3 ou 4 bytes (dependendo da precisão, 3 bytes para 6 dígitos)
TEXT	L + 2 bytes (onde L é o comprimento da string)
VARCHAR(M)	L + 1 bytes caso a coluna necessite de 0 a 255 bytes

Tabela 4.14 Tipos de dados utilizados e o seu respetivo tamanho em bytes.

Seguidamente, representam-se os espaços ocupados por algumas das principais tabelas do sistema, em bytes, assumindo o tamanho máximo dos campos de texto (*VARCHAR* no limite e *TEXT* estimado a uma média de 256 carateres) para quando existe apenas um registo na mesma.

Tabela	Nome da coluna	Tipo de dado	Tamanho (bytes)
Utilizador	"Id"	INT	4
	"Nome"	VARCHAR(100)	101
	"Email"	VARCHAR(100)	101
	"DataRegisto"	DATETIME	8
	"Telefone"	VARCHAR(20)	21
Total	-	-	235

Tabela 4.15 Espaço ocupado pela tabela Utilizador com um registo.

Tabela	Nome da coluna	Tipo de dado	Tamanho (bytes)
Evento	“Id”	INT	4
	“Titulo”	VARCHAR(150)	151
	“DataInicio”	DATETIME	8
	“DataFim”	DATETIME	8
	“CapacidadeMaxima”	INT	4
	“Organizador_id_Organizador”	INT	4
	“Local_id_Local”	INT	4
Total	-	-	183

Tabela 4.16 Espaço ocupado pela tabela Evento com um registo.

Tabela	Nome da coluna	Tipo de dado	Tamanho (bytes)
Inscrição	“Id”	INT	4
	“DataInscricao”	DATETIME	8
	“Estado”	VARCHAR(20)	21
	“Utilizador_id_Utilizador”	INT	4
	“Bilhete_id_Bilhete”	INT	4
Total	-	-	41

Tabela 4.17 Espaço ocupado pela tabela Inscrição com um registo.

Tabela	Nome da coluna	Tipo de dado	Tamanho (bytes)
Pagamento	“Id”	INT	4
	“Valor”	DECIMAL(6,2)	3
	“Metodo”	VARCHAR(50)	51
	“DataPagamento”	DATETIME	8
	“Inscricao_id_Inscricao”	INT	4
Total	-	-	70

Tabela 4.18 Espaço ocupado pela tabela Pagamento com um registo.

Com esta mesma configuração aplicada às restantes tabelas do modelo lógico, o espaço total ocupado pelas 13 tabelas, preenchidas com exatamente um registo cada, seria igual a **2 447 bytes** (cerca de 2.4 KB).

Após o povoamento inicial, estimou-se um cenário de arranque da base de dados com 500 utilizadores registados (estudantes e participantes). O evento principal (composto por 2 dias, 10 sessões e 20 palestras) conta com 30 oradores convidados e 15 empresas patrocinadoras. Relativamente aos acessos, foram emitidos 6 tipos de bilhetes, resultando em 400 inscrições efetivas e 380 pagamentos processados (estando 20 inscrições pendentes). Deste modo, é possível calcular o espaço total ocupado pelas tabelas na primeira implementação do sistema de gestão de base de dados.

Tabela	Quantidade de registos	Tamanho da tabela (bytes)
Utilizador	500	117 500
Organizador	5	1 135
Local	10	2 100
Orador	30	16 950
Patrocinador	15	3 405
Evento	2	366
Sessao	10	1 250
Bilhete	6	372
Palestra	20	7 660
Inscricao	400	16 400
Pagamento	380	26 600
Patrocinio	20	1 300
Apresenta	35	1 890
Total	-	196 928 bytes

Tabela 4.19 Espaço total ocupado pelas tabelas com o povoamento inicial.

Com base nas perspectivas de expansão da organização, é esperado que o número de utilizadores registados e a respetiva afluência a eventos (inscrições e pagamentos) aumente cerca de 50% a cada ano letivo, devido à divulgação nas universidades. Por outro lado, a logística central (Locais e Organizadores) manter-se-á mais estável, com um crescimento marginal, estimando-se a criação de apenas 1 grande evento adicional por ano, que trará consigo cerca de 10 novos oradores e 8 novas palestras. Com isto, prevemos que no primeiro ano as tabelas vão ter, respetivamente, as seguintes variações:

Tabela	Quantidade de registros	Diferença da tabela (bytes)
Utilizador	+ 250	+ 58 750
Inscricao	+ 250	+ 10 250
Pagamento	+ 240	+ 16 800
Orador	+ 10	+ 5 650
Palestra	+ 8	+ 3 064
Sessao	+ 4	+ 500
Patrocinador	+ 3	+ 681
Apresenta	+ 12	+ 648
Evento	+ 1	+ 183
Restantes tabelas	Marginais	≈ + 500
Total	-	+ 97 026 bytes

Tabela 4.20 Variações do espaço ocupado pelas tabelas após um ano.

Como foi evidenciado nas tabelas, o espaço necessário mínimo para a primeira implementação da base de dados em ambiente de produção seria de aproximadamente **197 KBytes**. Estimando a taxa de crescimento referida, alavancada pelo aumento do corpo estudantil e pela criação de novos eventos, prevê-se um crescimento anual na ordem dos **49%**, o que no primeiro ano se traduz num aumento de cerca de **97 KBytes** de armazenamento dedicado. qual1

4.8 Definição e caracterização de vistas de utilização em SQL

Nesta secção, apresenta-se a definição de um conjunto de vistas (*views*) essenciais para uma utilização eficiente da base de dados implementada. As vistas foram criadas para fornecer informações relevantes sobre a agenda do evento e a situação financeira das inscrições, facilitando o trabalho da aplicação pública e dos gestores de participantes. Para a caracterização destas vistas, foram usadas um conjunto de seleções e junções (*JOINS*).

A primeira vista cria uma tabela virtual que contém informações vitais para a visualização pública da agenda, incluindo o nome do evento, da sessão e da palestra, os respetivos horários de início e fim, além de dados cruciais sobre o orador e o papel que desempenha, permitindo uma compreensão completa e agregada do cronograma.

```

CREATE VIEW AgendaPublica AS
SELECT
    e.Titulo AS NomeEvento,
    s.NomeAtividade AS NomeSessao,
    p.Titulo AS NomePalestra,
    p.HoraInicio,
    p.HoraFim,
    o.Nome AS NomeOrador,
    a.Papel
FROM Evento e
INNER JOIN Sessao s ON e.Id=s.Evento_id_Evento
INNER JOIN Palestra p ON s.Id= p.Sessao_id_Sessao
LEFT JOIN Apresenta a ON p.Id=a.Palestra_id_Palestra
LEFT JOIN Orador o ON a.Orador_id_Orador=o.Id;

```

Figura 4.44 Vista que demonstra a agenda pública consolidada com os respetivos oradores.

A segunda vista foca-se na utilização pelos gestores de participantes, permitindo-lhes aceder de forma exclusiva e centralizada às informações de identificação de um participante, entre elas, o nome e o email, assim como os dados relativos ao evento, tipo de bilhete e o seu preço, estado da inscrição, valor pago e método de pagamento associado.

```

CREATE VIEW DetalhesInscricoes AS
SELECT
    u.Nome AS Participante,
    u.Email,
    e.Titulo AS Evento,
    b.Tipo AS TipoBilhete,
    b.Preco AS PrecoBilhete,
    i.Estado AS EstadoInscricao,
    pg.Valor AS ValorPago,
    pg.Metodo AS MetodoPagamento
FROM Utilizador u
INNER JOIN Inscricao i ON u.Id = i.Utilizador_id_Utilizador
INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete=b.Id
INNER JOIN Evento e ON b.Evento_id_Evento=e.Id
LEFT JOIN Pagamento pg ON i.Id=pg.Inscricao_id_Inscricao;

```

Figura 4.45 Vista que demonstra os detalhes dos participantes, inscrições e pagamentos.

Segue abaixo, um exemplo de formato de utilização de uma das vistas criadas, demonstrando a facilidade de acesso a dados complexos através de uma simples instrução de seleção:

```
SELECT * FROM DetalhesInscricoes;
```

Figura 4.46 Exemplo de utilização da vista "DetalhesInscricoes".

4.9 Tradução das interrogações do utilizador para SQL

Nesta secção, apresenta-se uma análise detalhada das consultas SQL, convertidas a partir dos requisitos de manipulação, utilizadas para extrair informações relevantes da base de dados *ENEI-Gest*. Cada consulta aos dados foi cuidadosamente projetada para fornecer informações específicas e métricas de gestão essenciais, relacionadas com os eventos, oradores, inscrições e pagamentos. Ao examinar cada interrogação, é possível entender melhor como os dados são cruzados, agregados e analisados de forma a auxiliar a organização na monitorização e tomada de decisões.

No início do ficheiro, declaram-se duas variáveis locais para permitir indicar qual o evento e o orador a serem utilizados na filtragem de algumas das consultas, garantindo a reutilização do código. Neste exemplo, propõe-se a utilização dos IDs 4 e 5 para o evento e para o orador, respetivamente.

```
SET @Evento_ID = 4;
SET @Orador_ID = 5;
```

Figura 4.47 Declaração dos IDs para utilização nas consultas.

A primeira consulta calcula o lucro total de um determinado evento. Para isso, recorre a subconsultas (*subqueries*) utilizando a função de agregação `SUM` e a função de proteção `IFNULL` (para precaver resultados vazios). O código calcula independentemente o valor gerado pela venda de bilhetes e os fundos angariados por patrocínios, somando-os no final para apresentar a receita total combinada.

```
-- Lucro Total de um Evento bilhetes+patrocinios

SELECT
  e.Titulo AS Evento,
  (SELECT SUM(Valor) FROM Pagamento p
   INNER JOIN Inscricao i ON p.Inscricao_id_Inscricao=i.Id
   INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete = b.Id
   WHERE b.Evento_id_Evento = @Evento_ID) AS Total_Bilhetes,
  (SELECT SUM(Valor) FROM Patrocinio WHERE Evento_id_Evento = @Evento_ID) AS Total_Patrocinios,
  (SELECT SUM(Valor) FROM Pagamento p
   INNER JOIN Inscricao i ON p.Inscricao_id_Inscricao = i.Id
   INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete = b.Id
   WHERE b.Evento_id_Evento = @Evento_ID) + (SELECT SUM(Valor) FROM Patrocinio WHERE Evento_id_Evento = @Evento_ID) AS Receita_Total
FROM Evento e
WHERE e.Id = @Evento_ID;
```

Figura 4.48 Cálculo do lucro total de um evento com SQL.

A segunda consulta lista a agenda de palestras de um determinado orador. A interrogação une as tabelas "Orador", "Apresenta" e "Palestra" (`INNER JOIN`) para exibir o tema da intervenção, o horário de início e o papel desempenhado pelo especialista, ordenando os resultados de forma cronológica (`ORDER BY ...ASC`).

```
-- Listar as palestras de um orador

SELECT
  o.Nome AS Orador,
  p.Titulo AS Tema_Palestra,
  p.HoraInicio,
  a.Papel
FROM Orador o
INNER JOIN Apresenta a ON o.Id = a.Orador_id_Orador
INNER JOIN Palestra p ON a.Palestra_id_Palestra = p.Id
WHERE o.Id = @Orador_ID
ORDER BY p.HoraInicio ASC;
```

Figura 4.49 Listar as palestras de um orador com SQL.

A terceira consulta realiza a contagem do sucesso de vendas, enumerando as inscrições confirmadas por evento. Através

da junção das tabelas "Evento", "Bilhete" e "Inscrição", a pesquisa filtra exclusivamente as marcações com o estado 'Confirmada'. Em seguida, agrupa os dados pelo título do evento (GROUP BY) e apresenta-os de forma decrescente.

```
-- Contagem de inscrições Confirmadas por Evento

SELECT
    e.Titulo AS Evento,
    COUNT(i.Id) AS Bilhetes_Vendidos
FROM Evento e
INNER JOIN Bilhete b ON e.Id = b.Evento_id_Evento
INNER JOIN Inscricao i ON b.Id = i.Bilhete_id_Bilhete
WHERE i.Estado = 'Confirmada'
GROUP BY e.Titulo
ORDER BY Bilhetes_Vendidos DESC;
```

Figura 4.50 Contagem de inscrições confirmadas por evento com SQL.

A quarta consulta foca-se na tesouraria e verifica qual é o método de pagamento mais popular. Agrupa os registos financeiros pelo método utilizado, contabilizando a quantidade absoluta de utilizações e o volume transacionado em euros. Os resultados são ordenados de forma decrescente e restringidos apenas ao primeiro lugar através do comando LIMIT 1.

```
-- Verificar o metodo de pagamento mais popular

SELECT
    Metodo AS Metodo_Pagamento,
    COUNT(id) AS Quantidade_Utilizacoes,
    SUM(Valor) AS Volume_Euros
FROM Pagamento
GROUP BY Metodo
ORDER BY Quantidade_Utilizacoes DESC
LIMIT 1;
```

Figura 4.51 Verificar o método de pagamento mais popular com SQL.

A quinta consulta lista os utilizadores que se encontram com as suas inscrições pendentes de pagamento (dívidas). Para garantir este cruzamento de dados de forma fiável, recorre a um LEFT JOIN com a tabela "Pagamento" e exige na condição (WHERE) que a transação correspondente seja nula (p.Id IS NULL), isolando assim as marcações não faturadas.

```
-- Listar utilizadores com inscricoes pendentes

SELECT
    u.Nome,
    u.Email,
    b.Tipo AS Tipo_Bilhete,
    b.Preco AS Valor_Em_Divida
FROM Utilizador u
INNER JOIN Inscricao i ON u.Id = i.Utilizador_id_Utilizador
INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete=b.Id
LEFT JOIN Pagamento p ON i.Id = p.Inscricao_id_Inscricao
WHERE i.Estado = 'Pendente' AND p.Id IS NULL;
```

Figura 4.52 Listar utilizadores com inscrições pendentes com SQL.

A última consulta extrai os top 3 oradores com maior participação ativa na agenda global. Junta as informações das tabelas "Orador" e "Apresenta", agrupa-as pelo perfil do especialista e conta o número de palestras a que este se encontra alocado. A ordenação é feita do maior para o menor número de participações, apresentando as três personalidades de maior destaque.

```
-- TOP 3 oradores com mais participantes ativos

SELECT
  o.Nome AS Orador,
  o.InstituicaoEmpresa AS Empresa,
  COUNT(a.Palestra_id_Palestra) AS Total_Participantes
FROM Orador o
INNER JOIN Apresenta a ON o.Id=a.Orador_id_Orador
GROUP BY o.Id, o.Nome, o.InstituicaoEmpresa
ORDER BY Total_Participacoes DESC
LIMIT 3;
```

Figura 4.53 Listar os *top 3* oradores mais ativos com SQL.

4.10 Indexação do Sistema de Dados

Nesta secção aborda-se a necessidade de aplicar uma metodologia de indexação à base de dados *ENEI-Gest*, com o objetivo de otimizar a eficiência das operações de pesquisa e manipulação de dados. O primeiro passo é a avaliação e previsão dos pontos de incidência resultantes da utilização da base de dados.

O processo de avaliação inicia-se com a identificação das tabelas e colunas que necessitarão de um índice. Após estudo detalhado da situação, concluiu-se que, dado que os utilizadores principais da base de dados (estudantes e organizadores) acederão frequentemente à agenda de eventos e à gestão de bilhética, prevê-se que as tabelas Inscrição, Sessão e Palestra sejam frequentemente acedidas. Tornam-se assim os seus identificadores de ligação (chaves estrangeiras) pontos críticos de acesso.

Recorre-se à utilização de indexação por três motivos principais:

1. **Melhoria do desempenho das operações de consulta:** A indexação permite um acesso mais rápido aos registos, uma vez que os índices facilitam a localização eficiente dos dados. Isto é particularmente útil para operações de leitura, como a consulta da agenda pública, que são frequentes e exigem tempos de resposta rápidos.
2. **Redução da necessidade de leitura de dados repetitivos:** Com a utilização de índices, o sistema pode aceder diretamente aos elementos específicos de uma tabela sem necessidade de percorrer todos os registos (*Full Table Scan*). Isto reduz a carga de trabalho no sistema de armazenamento e melhora a eficiência geral da aplicação.
3. **Facilitação da escalabilidade do sistema:** À medida que o número de registos (participantes e eventos) aumenta, os índices são projetados para lidar com pesquisas simultâneas de forma eficiente, sem degradar o desempenho geral do sistema de gestão de base de dados. Isto é essencial para manter a performance do sistema em picos de alto tráfego, como na abertura de inscrições.

Na situação do sistema *ENEI-Gest*, após identificar os pontos de maior incidência, prossegue-se à criação dos índices nas chaves estrangeiras das tabelas com maior frequência. Note-se que não há necessidade de criar índices para chaves primárias, uma vez que naturalmente já são constituídas num índice no momento da sua criação.

Primeiramente, criaram-se os índices na tabela "Inscricao", focando as colunas referentes ao Utilizador e ao Bilhete, de forma a acelerar as pesquisas relativas ao estado financeiro e de participação de cada estudante.

```
CREATE INDEX idx_inscricao_utilizador_id ON Inscricao(Utilizador_id Utilizador);
CREATE INDEX idx_inscricao_bilhete_id ON Inscricao(Bilhete_id Bilhete);
```

Figura 4.54 Criação de índices para a tabela Inscrição.

Em seguida, aplicou-se um índice na tabela "Palestra", focado na chave estrangeira da Sessão, para otimizar o carregamento e a filtragem da agenda de atividades detalhadas.

```
CREATE INDEX idx_palestra_sessao_id ON Palestra(Sessao_id Sessao);
```

Figura 4.55 Criação de índice para a tabela Palestra.

Por fim, criou-se um índice na tabela "Sessao", associando-a de forma mais célere ao seu respetivo Evento, garantindo que a hierarquia estrutural da conferência responde rapidamente às interrogações dos utilizadores.

```
CREATE INDEX idx_sessao_evento_id ON Sessao(Evento_id_Evento);
```

Figura 4.56 Criação de índice para a tabela Sessão.

4.11 Implementação de procedimentos, funções e gatilhos

Como forma de automatizar e simplificar processos dentro do contexto do sistema de gestão da base de dados *ENEI-Gest*, foram definidos um conjunto de gatilhos (*triggers*), funções e procedimentos. De seguida, evidenciam-se os mesmos, assim como a explicação técnica e a justificação para a sua implementação.

A função "CalcularReceitaTotal" permite ao sistema calcular rapidamente o volume total de faturação de um evento, recebendo, para o efeito, apenas o identificador do evento em questão (*p_Evento_ID*). Desta forma, a função efetua duas consultas internas: a primeira soma os valores faturados na venda de bilhetes (cruzando os pagamentos com as inscrições) e a segunda soma as injeções de capital provenientes de patrocínios. Por fim, a função devolve a soma destas duas vertentes. Esta função revela-se bastante útil para o utilizador com o perfil de "gestor_participantes", uma vez que simplifica o processo de auditoria financeira, recorrendo à função *IFNULL* para prevenir falhas de cálculo caso um evento ainda não possua receitas registadas.

```
DELIMITER $$
CREATE FUNCTION CalcularReceitaTotal(p_Evento_ID INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE total_bilhetes DECIMAL(10,2);
    DECLARE total_patrocinios DECIMAL(10,2);

    SELECT IFNULL(SUM(p.Valor), 0) INTO total_bilhetes
    FROM Pagamento p
    INNER JOIN Inscricao i ON p.Inscricao_id_Inscricao = i.Id
    INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete = b.Id
    WHERE b.Evento_id_Evento = p_Evento_ID;

    SELECT IFNULL(SUM(Valor), 0) INTO total_patrocinios
    FROM Patrocinio
    WHERE Evento_id_Evento = p_Evento_ID;

    RETURN total_bilhetes + total_patrocinios;
END $$
DELIMITER ;
```

Figura 4.57 Função que calcula a receita total (bilheteira e patrocínios) de um evento.

O gatilho "AtualizarEstadoPagamento" é responsável por alterar automaticamente o estado de uma inscrição para "Confirmada" logo após a liquidação do respetivo bilhete. Assim sendo, configurado para disparar após a inserção de um novo registo na tabela *Pagamento* (*AFTER INSERT ON Pagamento*), o sistema utiliza o identificador da nova entrada (*NEW.Inscricao_id_Inscricao*) para localizar a inscrição correspondente e efetuar a atualização (*UPDATE*) do seu estado. Desta forma, este gatilho elimina a necessidade de alteração manual das inscrições por parte da organização, garantindo a consistência dos dados e promovendo uma maior abstração no que toca à interação entre as transações financeiras e o controlo de acessos.

```

DELIMITER $$
CREATE TRIGGER AtualizarEstadoPagamento
AFTER INSERT ON Pagamento
FOR EACH ROW
BEGIN

    UPDATE Inscricao
    SET Estado = 'Confirmada'
    WHERE Id = NEW.Inscricao_id_Inscricao;
END $$
DELIMITER ;

```

Figura 4.58 Gatilho responsável por atualizar o estado da inscrição após o pagamento.

O procedimento "GerarAgendaFinal" tem como propósito automatizar a extração e estruturação cronológica do programa completo de um evento. O mesmo recebe o identificador do evento (`p_Evento_ID`) e consolida toda a informação das sessões, palestras e respetivos oradores. Para tal, faz uso de um conjunto de *JOINS* e utiliza a função `IFNULL` para garantir que, caso uma palestra ainda não tenha um orador atribuído, o sistema apresente a mensagem 'Orador a definir' em vez de um valor nulo. Os resultados são obrigatoriamente ordenados de forma ascendente pela hora de início (`ORDER BY p.HoraInicio ASC`), entregando um cronograma pronto a ser consumido pela aplicação ou *website* do evento.

```

DELIMITER $$
CREATE PROCEDURE GerarAgendaFinal(IN p_Evento_ID INT)
BEGIN
    SELECT
        s.NomeAtividade AS Sessao,
        p.HoraInicio,
        p.HoraFim,
        p.Titulo AS Palestra,
        IFNULL(o.Nome, 'Orador a definir') AS Orador
    FROM Sessao s
    INNER JOIN Palestra p ON s.Id = p.Sessao_id_Sessao
    LEFT JOIN Apresenta a ON p.Id = a.Palestra_id_Palestra
    LEFT JOIN Orador o ON a.Orador_id_Orador = o.Id
    WHERE s.Evento_id_Evento = p_Evento_ID
    ORDER BY p.HoraInicio ASC;
END $$
DELIMITER ;

```

Figura 4.59 Procedimento para gerar a agenda final e ordenada de um evento.

O gatilho *ReporVagaCancelamento* é responsável por repor automaticamente uma vaga disponível no bilhete correspondente sempre que uma inscrição transita para o estado "Cancelada". Configurado para disparar após a atualização de um registo na tabela *Inscricao* (*AFTER UPDATE ON Inscricao*), o sistema verifica se o novo estado (*NEW.Estado*) passou a ser "Cancelada" e se o estado anterior (*OLD.Estado*) era diferente, evitando assim execuções redundantes em atualizações que não envolvam cancelamentos.

Confirmada esta condição, é efetuado um incremento de uma unidade no campo "VagasDisponiveis" do bilhete associado à inscrição cancelada. Desta forma, este gatilho implementa diretamente a regra de negócio acordada pela Comissão Organizadora, garantindo que nenhuma vaga fique bloqueada por uma desistência e que a ocupação dos espaços seja maximizada de forma automática e sem intervenção manual.

```

DELIMITER $$
CREATE TRIGGER ReporVagaCancelamento
AFTER UPDATE ON Inscricao
FOR EACH ROW
BEGIN
    IF NEW.Estado = 'Cancelada' AND OLD.Estado != 'Cancelada' THEN
        UPDATE Bilhete
        SET VagasDisponiveis = VagasDisponiveis + 1
        WHERE Id = NEW.Bilhete_id_Bilhete;
    END IF;
END $$
DELIMITER ;

```

Figura 4.60 Gatilho responsavel por atualizar as vagas caso desistencia.

5. Conclusões e Trabalho Futuro

5.1 Conclusões

Com a conceção e implementação do sistema de gestão de base de dados *ENEI-Gest*, a equipa conseguiu criar uma infraestrutura de dados robusta e escalável, capaz de suportar toda a logística, bilhética e agenda de eventos académicos de grande dimensão. A adoção deste sistema relacional representa um passo fundamental para garantir a integridade, confidencialidade e disponibilidade das informações críticas da organização.

Através de uma análise crítica ao trabalho desenvolvido, destacam-se como principais pontos fortes do projeto a rigorosa aplicação das regras de normalização (que eliminou redundâncias nos dados) e a forte aposta na segurança, materializada através do Princípio do Menor Privilégio na criação de utilizadores específicos (*gestor_conteudos* e *gestor_participantes*). Adicionalmente, a implementação de vistas (*Views*), índices, gatilhos (*Triggers*) e funções provou ser essencial para automatizar processos de tesouraria e otimizar os tempos de resposta da agenda pública.

Durante a execução deste projeto, o principal desafio incidiu na modelação das relações complexas de muitos-para-muitos (N:M), nomeadamente a alocação de oradores a múltiplas palestras e a gestão de diferentes escalões de patrocínio. Embora tenhamos demonstrado alguma inexperiência inicial na demarcação das dependências estruturais durante o modelo conceptual, essas inconformidades foram rapidamente superadas nas fases de refinamento do modelo lógico, não causando impedimentos ao cumprimento dos objetivos propostos.

A formulação íntegra e consolidada deste caso de estudo permitiu à equipa a solidificação da composição de todo o sistema, que teve por base o estrito cumprimento de todas as etapas do ciclo de vida de desenvolvimento de uma base de dados.

5.2 Trabalho Futuro

Apesar de o sistema desenvolvido se apresentar funcional e robusto para o âmbito atual, identificou-se um vasto potencial de crescimento. De forma a complementar e evoluir a infraestrutura *ENEI-Gest*, propõem-se as seguintes melhorias para trabalho futuro:

- **Integração com APIs de Pagamento:** Atualmente, os métodos de pagamento (como o MBWay) são registados de forma declarativa. No futuro, a base de dados deverá estar preparada para comunicar com *gateways* de pagamento externos, permitindo a inserção automática na tabela *Pagamento* mal a transação seja validada pelo banco.
- **Sistema de *Check-in* em Tempo Real:** Criação de uma tabela de *Controlo_Acessos* interligada aos *Bilhetes*, de forma a registar a hora exata de entrada dos participantes no recinto através da leitura de códigos QR, permitindo análises estatísticas de lotação em tempo real.
- **Módulo de Avaliação (*Feedback*):** Implementação de uma estrutura que permita aos utilizadores classificar de 1 a 5 estrelas e deixar comentários nas palestras a que assistiram, gerando dados valiosos para a escolha de oradores em edições futuras.
- **Arquivamento Histórico:** Criação de procedimentos automatizados (*Stored Procedures* calendarizados) para migrar dados de edições antigas do evento para tabelas de arquivo (*Data Warehousing*), garantindo que o sistema de produção se mantém leve e rápido a longo prazo.

Referências

Apresentar a lista de referências bibliográficas referidas ao longo do relatório.

MySQL 8.0 Reference Manual. MySQL. (2024). <https://dev.mysql.com/doc/refman/8.0/en/>

Lista de Siglas e Acrónimos

Apresentar uma lista com todas as siglas e acrónimos utilizados durante a realização do trabalho.

BD	Base de Dados
BDOO	Base de Dados Orientada a Objetos
BDRN	Base de Dados Relacional
CE	Chave Estrangeira
CP	Chave Primária
DW	Data Warehouse
ENEI	Encontro Nacional de Estudantes de Informática
ER	Entidade-Relacionamento
FK	Foreign Key (Chave Estrangeira)
HTML	HyperText Markup Language
ID	Identificador
INT	Integer (Inteiro)
MB	Megabyte
NIF	Número de Identificação Fiscal
PK	Primary Key (Chave Primária)
SQL	Structured Query Language
UC	Unidade Curricular
UM	Universidade do Minho
VIP	Very Important Person

Anexos

Anexo 1: Ata da Primeira Reunião de Alinhamento

Data de realização: 25 de fevereiro de 2026

Hora de início: 14:30h

Hora de término: 16:15h

Local: Sala de Reuniões da Associação de Estudantes, Campus de Gualtar, Universidade do Minho [cite: 1]

Participantes:

- João Taveira - Membro da equipa "Foreign Four"
- Miguel Gonçalves - Membro da equipa "Foreign Four"
- Mariana Silva - Representante da Comissão Organizadora do ENEI
- Tiago Costa - Representante da Comissão Organizadora do ENEI

Contextualização:

Ao vigésimo quinto dia do mês de fevereiro de dois mil e vinte e seis, com início às catorze horas e trinta minutos, na sala de reuniões da Associação de Estudantes da Universidade do Minho, realizou-se a primeira reunião presencial com os representantes da Comissão Organizadora do ENEI, Mariana Silva e Tiago Costa. O principal objetivo deste encontro consistiu na apresentação e discussão de uma solução tecnológica capaz de dar resposta aos desafios logísticos associados a um caso de estudo no domínio da "Organização de Eventos com Inscrições".

Levantamentos e Ordem de Trabalhos:

Iniciaram-se os trabalhos previstos com a apresentação formal dos intervenientes de ambas as partes. João Taveira começou por apresentar a equipa "Foreign Four", constituída por alunos da Licenciatura em Ciências da Computação, descrevendo as metodologias de trabalho e os princípios arquiteturais que a equipa aplica no desenvolvimento tecnológico. Em seguida, Miguel Gonçalves enumerou as competências técnicas dos elementos do grupo, destacando a sua aptidão para o planeamento, arquitetura e implementação de sistemas.

No decorrer da ordem de trabalhos, Mariana Silva, na qualidade de coordenadora-geral do evento, procedeu à descrição da questão-problema. A representante evidenciou a enorme sobrecarga administrativa decorrente do volume de inscrições esperado para a edição do ENEI em Braga, sublinhando que os métodos manuais baseados em folhas de cálculo se revelaram insuficientes para gerir participantes, lotações de anfiteatros, patrocinadores e oradores. Tiago Costa complementou a intervenção, manifestando uma forte preocupação com a segurança dos dados pessoais dos estudantes e com a urgência de evitar o colapso logístico nos dias do encontro.

Após a exposição das dificuldades enfrentadas pela organização, João Taveira e Miguel Gonçalves propuseram formalmente o desenvolvimento e implementação de um Sistema de Bases de Dados relacional. A equipa "Foreign Four" argumentou que este sistema centralizado permitiria não só o controlo rigoroso das inscrições e pagamentos, como também a alocação dinâmica de espaços e a emissão eficiente de bilhetes.

A proposta foi recebida com notório entusiasmo pela comissão organizadora. Mariana Silva agradeceu a proatividade da equipa e destacou a importância desta colaboração técnica para elevar a qualidade do evento nacional. Em resposta à solução apresentada, os participantes concordaram em avançar com o projeto, validando a viabilidade da ideia de imediato.

Em adição, ficou acordada a criação de um canal de comunicação direto entre a "Foreign Four" e o departamento de logística do ENEI. João Taveira enfatizou que o sucesso do projeto dependerá de uma recolha exaustiva de requisitos, de modo a garantir que o modelo conceptual reflita com precisão as regras de negócio do evento.

Para dar continuidade ao processo, ficou agendada uma nova reunião de trabalho para o início do mês de março, com o propósito exclusivo de proceder ao levantamento detalhado de requisitos, mapear as entidades nucleares do sistema e definir os atributos necessários para a base de dados.

A reunião foi dada por encerrada às dezasseis horas e quinze minutos, com a promessa de um esforço conjunto e contínuo para garantir que a gestão tecnológica do ENEI seja um marco de sucesso e eficiência.

(nao sei se querem por assinaturas)??????????

Anexo 2: Ata da Segunda Reunião - Levantamento de Requisitos

Data de realização: 4 de março de 2026

Hora de início: 09:30h

Hora de término: 12:00h

Local: Sala de Reuniões da Associação de Estudantes, Campus de Gualtar, Universidade do Minho

Participantes:

- João Neiva - Membro da equipa "Foreign Four"
- João Durães - Membro da equipa "Foreign Four"
- Mariana Silva - Representante da Comissão Organizadora do ENEI
- Tiago Costa - Representante da Comissão Organizadora do ENEI

Contextualização:

Ao quarto dia do mês de março de dois mil e vinte e seis, com início às nove horas e trinta minutos da manhã, realizou-se a segunda reunião com os representantes da comissão organizadora do ENEI. O objetivo central deste encontro consistiu em idealizar e formalizar os requisitos mandatórios para o desenvolvimento da base de dados, focando-se na estrutura de dados, manipulação da informação e permissões de segurança essenciais para a gestão logística do encontro.

Tópicos abordados:

No decorrer da reunião, foram esclarecidos os aspetos técnicos e as regras de negócio a ter em conta no domínio da gestão de participantes, eventos, espaços físicos e regras de faturação. Estes aspetos revelam-se cruciais para garantir a consistência e a integridade da base de dados que irá suportar o evento.

Levantamentos e Discussão:

Com o desenrolar da reunião, foi possível recolher diversas exigências relacionadas com a estruturação da informação, documentadas pela equipa técnica.

Numa fase inicial, Mariana Silva referiu que o sistema deve registar todos os participantes como utilizadores da plataforma, assegurando que cada um possui um identificador único. Acrescentou ainda que o nome, o email, o telefone e a data do registo são dados de contacto essenciais que precisam de ser armazenados.

Relativamente à grelha do encontro, Tiago Costa destacou que o ENEI é composto por múltiplos subeventos. Ficou estabelecido que cada evento deve possuir um código de identificação único, um título e as respetivas datas de início e fim. Em resposta, João Neiva sublinhou que os eventos não acontecem no vazio, questionando sobre a gestão dos espaços. A comissão concordou que o sistema deve armazenar a informação dos locais físicos, sendo obrigatório registar o nome do espaço, a morada completa e, crucialmente, a sua lotação máxima para efeitos de segurança.

No que toca à vertente financeira, Mariana Silva explicou a necessidade de gerir diferentes modalidades de acesso. Foi decidido que tipos de bilhetes como "VIP" ou "Estudante" devem ser associados a cada evento, contendo sempre a designação, o preço e o número de vagas disponíveis. Consequentemente, as inscrições efetuadas devem conter uma referência obrigatória ao participante e ao evento correspondente, registando também a data em que a inscrição foi realizada e o seu estado atual isto é se o mesmo esta Pendente, Confirmado ou Cancelado.

A discussão avançou para o conteúdo das sessões do encontro. Tiago Costa mencionou que os dados específicos das palestras o seu título, resumo, hora de início e hora de fim devem ser armazenados com uma referência direta à sessão em que estão integradas. Além disso, cada orador convidado deve ser registado com um identificador, o seu nome, a sua afiliação ou seja instituição ou empresa e uma breve biografia para figurar no cartaz do evento.

Para além da estrutura, a equipa "Foreign Four" guiou a conversa para as necessidades de manipulação de dados. A comissão organizadora frisou que o sistema deve permitir, a qualquer momento, listar os eventos futuros que ainda tenham vagas disponíveis para inscrição. Do ponto de vista do utilizador, João Durães sugeriu, e foi aprovado, que deve ser possível consultar o histórico completo de eventos em que um determinado participante se inscreveu. O sistema também deve ter a capacidade de registar pagamentos, alterando automaticamente o estado da inscrição de "Pendente" para "Confirmado". A nível de relatórios de gestão, ficou exigido que os organizadores consigam extrair o valor total de receitas geradas por evento, agrupadas por tipo de bilhete, bem como gerar listas cronológicas de palestras com os respetivos oradores associados. Como funcionalidade básica, o sistema deve permitir aos participantes a atualização dos seus próprios dados de contacto.

Por fim, a reunião abordou os requisitos de controlo de acessos e segurança. Por unanimidade, ficou definido que apenas perfis com o nível de "Administrador" ou "Organizador" poderão criar, editar ou apagar eventos e sessões. Um participante comum terá o seu acesso restrito à consulta e edição dos seus próprios dados e histórico, sendo-lhe vedado o acesso aos dados de terceiros. Tiago Costa enfatizou ainda que a operação sensível de alterar a lotação máxima de um local ficará estritamente reservada ao administrador da base de dados. Por último, João Neiva propôs uma regra de ouro para a integridade financeira: a eliminação física de registos de pagamentos será terminantemente proibida para todos os perfis. Qualquer correção terá de ser realizada através de transações de reembolso ou cancelamento, mantendo o rasto de auditoria.

Deste modo, após a formalização de todas estas condições e requisitos, a reunião foi encerrada, fornecendo à equipa "Foreign Four" a base documental necessária para iniciar a fase de Modelação Conceptual.

Anexo 3: Ata da Terceira Reunião - Regras de Segurança e Interrogações

Data de realização: 8 de março de 2026

Hora de início: 09:00h

Hora de término: 11:30h

Local: Sala de Reuniões da Associação de Estudantes, Campus de Gualtar, Universidade do Minho

Participantes:

- Miguel Gonçalves - Membro da equipa "Foreign Four"
- João Taveira - Membro da equipa "Foreign Four"
- Mariana Silva - Representante da Comissão Organizadora do ENEI
- Ricardo Mendes - Responsável de Segurança Informática do ENEI
- Inês Correia - Coordenadora de Logística do ENEI

Contextualização:

Ao oitavo dia do mês de março de dois mil e vinte e seis, com início às nove horas da manhã, realizou-se a terceira reunião de trabalho. Este encontro contou, pela primeira vez, com a presença de Ricardo Mendes, o responsável máximo pela segurança informática da comissão do ENEI, e Inês Correia, da logística. Após o levantamento estrutural da reunião anterior, o objetivo desta sessão foi definir exaustivamente as regras de controlo, segurança e as interrogações avançadas que o sistema terá de suportar, garantindo que o *ENEI-Gest* seja não só funcional, mas também inviolável e capaz de gerar métricas úteis.

Tópicos abordados:

O debate centrou-se na definição de perfis de acesso, políticas de alteração de dados sensíveis (como lotações e

pagamentos) e na listagem de relatórios críticos necessários para a gestão logística em tempo real durante os dias do evento.

Levantamentos e Discussão:

A reunião iniciou-se com uma forte intervenção de Ricardo Mendes, que demonstrou particular preocupação com a integridade dos dados face ao volume esperado de acessos. O responsável de segurança foi categórico ao afirmar que as permissões devem ser estritamente hierarquizadas. Ricardo determinou que apenas utilizadores com o perfil de "Administrador" poderão ter o privilégio de criar, editar ou apagar eventos e sessões.

Em consonância com esta preocupação, João Taveira propôs uma política restritiva para o público geral: um participante comum, ao aceder à plataforma, deverá estar confinado a um perfil de "Utilizador" que apenas lhe permita consultar e editar os seus próprios dados pessoais e visualizar o seu histórico de inscrições. Foi unanimemente aceite que é imperativo bloquear qualquer acesso cruzado, impedindo que um participante consiga visualizar dados de terceiros.

Inês Correia, focada na logística dos espaços, levantou a questão dos problemas de sobrelotação vividos em edições passadas. Para mitigar esse risco, ficou estabelecido que a operação de alterar a "lotação máxima" de um local não pode ser feita por qualquer membro da organização. Esta ação sensível ficará exclusivamente reservada ao administrador principal da base de dados, prevenindo alterações acidentais que pudessem comprometer a segurança física dos espaços.

Avançando para as regras financeiras, Miguel Gonçalves relembrou a regra de ouro discutida informalmente e formalizou-a no documento: a eliminação física de registos de pagamentos (o comando *DELETE* na base de dados) está estritamente proibida, independentemente do perfil do utilizador. Ricardo Mendes apoiou a medida, concordando que as correções financeiras deverão ser sempre realizadas através de transações registadas de "reembolso" ou "cancelamento", mantendo um histórico auditável de todas as movimentações.

No domínio da extração de dados e métricas, Mariana Silva frisou a necessidade de o sistema fornecer *insights* imediatos. Solicitou que a equipa garanta a capacidade de gerar um relatório global de receitas por evento, obrigatoriamente agrupado por tipo de bilhete. Adicionalmente, Inês Correia pediu que o sistema permitisse extrair uma lista cronológica das palestras agendadas para cada subevento, incluindo os nomes dos respetivos oradores associados, para facilitar a impressão dos programas de sala.

A equipa "Foreign Four" confirmou que a estrutura de dados planeada na reunião anterior conseguiria suportar estas interrogações através da criação de *Views* (vistas) em SQL, o que agradou à comissão.

Para finalizar a reunião, Mariana Silva e Ricardo Mendes acordaram que, estando definidos os requisitos estruturais e de segurança, a equipa "Foreign Four" teria "luz verde" para avançar com a criação dos primeiros rascunhos visuais do sistema (Modelação Conceptual). Ficou estipulado que o próximo contacto seria feito no prazo de duas semanas, altura em que a equipa apresentaria o Diagrama Entidade-Relacionamento para validação final.

Anexo 4: Ata da Quarta Reunião - Revisão e Validação Final

Data de realização: 14 de março de 2026

Hora de início: 09:15h

Hora de término: 10:45h

Local: Sala de Reuniões da Associação de Estudantes, Campus de Gualtar, Universidade do Minho

Participantes:

- João Taveira - Membro da equipa "Foreign Four"
- João Neiva - Membro da equipa "Foreign Four"
- Miguel Gonçalves - Membro da equipa "Foreign Four"
- João Durães - Membro da equipa "Foreign Four"
- Mariana Silva - Representante da Comissão Organizadora do ENEI
- Tiago Costa - Representante da Comissão Organizadora do ENEI
- Ricardo Mendes - Responsável de Segurança Informática do ENEI

Contextualização:

Ao décimo quarto dia do mês de março de dois mil e vinte e seis, realizou-se a quarta e última reunião de alinhamento com a totalidade da comissão organizadora do ENEI e a equipa "Foreign Four". O objetivo exclusivo desta sessão foi a leitura, revisão e aprovação final de todos os requisitos levantados nas reuniões anteriores, garantindo que não existiam ambiguidades antes de se iniciar a fase de modelação e implementação da base de dados.

Levantamentos e Validação:

João Neiva deu início à reunião fazendo um resumo global da arquitetura planeada, passando em revista as entidades principais: Utilizador/Participante, Evento, Local/Espaço, Organizador/Promotor, Inscrição, Pagamento, Sessão, Orador, Palestra, Patrocinador e Bilhete.

Durante a revisão, Tiago Costa pediu um pequeno esclarecimento técnico relativo às vagas dos bilhetes. Questionou o que aconteceria ao número de "vagas disponíveis" caso uma inscrição passasse para o estado de "Cancelado". Miguel Gonçalves clarificou prontamente que a equipa já tinha previsto essa situação: sempre que uma inscrição for cancelada, o sistema deverá repor automaticamente a vaga disponível no respetivo evento, garantindo que a lotação é maximizada. A comissão aplaudiu esta solução automatizada.

Em seguida, Ricardo Mendes tomou a palavra para rever as permissões de acesso. Solicitou uma pequena clarificação sobre o identificador único dos utilizadores. Após uma breve discussão sobre a privacidade dos dados, ficou acordado que o identificador único na base de dados será um código gerado pelo sistema (ID), mas que o endereço de *email* também deverá ser único para evitar contas duplicadas por parte do mesmo estudante. A equipa "Foreign Four" registou esta restrição de unicidade para a futura normalização de dados.

João Durães procedeu à leitura das interrogações complexas e dos relatórios exigidos, confirmando com a logística e a tesouraria se faltava alguma métrica vital. Mariana Silva confirmou que a listagem de receitas por tipo de bilhete e o programa cronológico com os oradores cobriam as necessidades imediatas da organização.

Não havendo mais alterações ou requisitos a acrescentar, a comissão organizadora validou oficialmente o caderno de encargos. Mariana Silva expressou a sua satisfação com o rigor demonstrado pela equipa de Ciências da Computação.

A reunião foi encerrada às dez horas e quarenta e cinco minutos. Com esta validação, a equipa "Foreign Four" obteve a autorização formal para transitar da fase de Análise de Requisitos para a fase de Modelação Conceptual, dando início à construção do Diagrama Entidade-Relacionamento (ER).

Anexo 5: Figuras do Projeto

Esta secção contém todas as figuras utilizadas ao longo do relatório, organizadas pela respetiva ordem de apresentação no documento.

Identificação Visual



Figura 5.1 Logo da Universidade do Minho utilizado na capa

Modelação Conceptual (Diagrama ER e Relacionamentos)

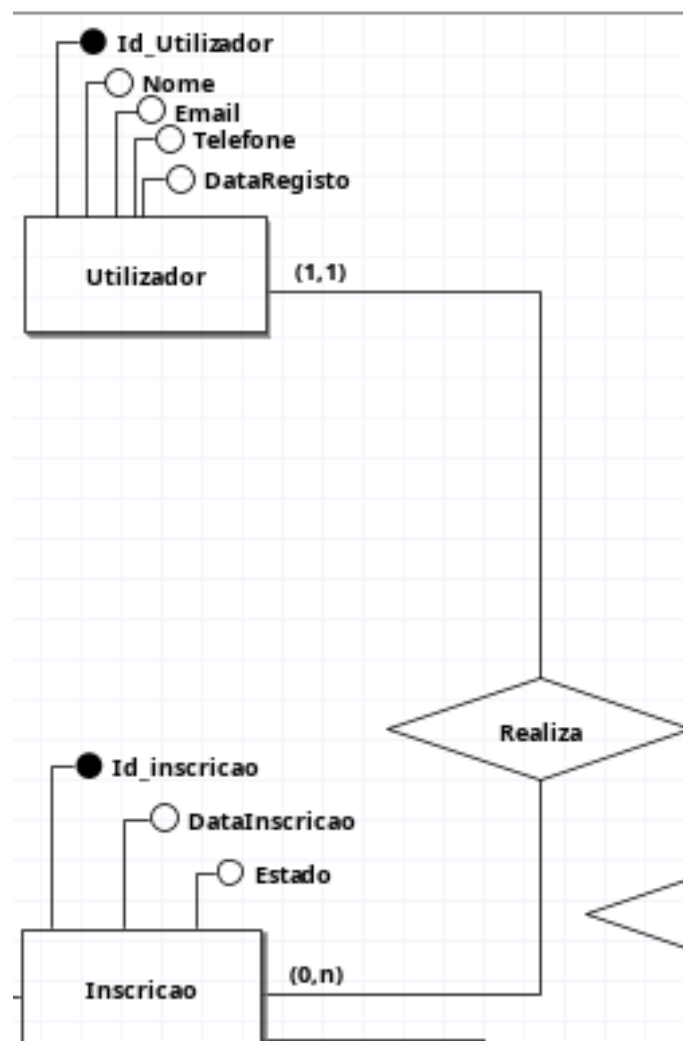


Figura 5.2 Ilustração do Relacionamento Utilizador - Inscrição

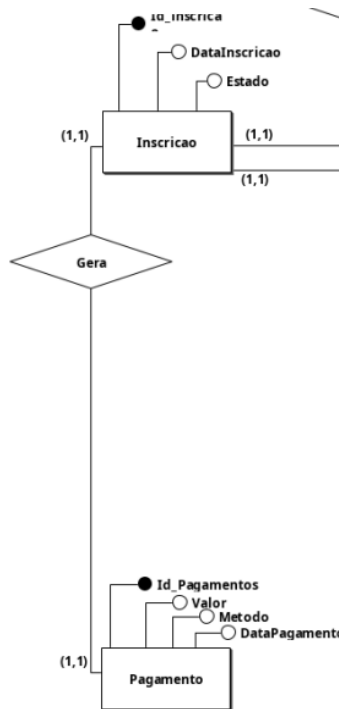


Figura 5.3 Ilustração do Relacionamento Inscrição - Pagamento



Figura 5.4 Ilustração do Relacionamento Inscrição - Bilhete

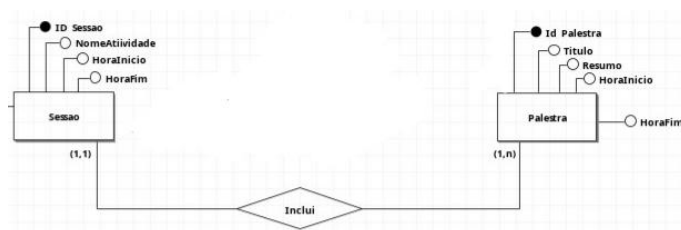


Figura 5.5 Ilustração do Relacionamento Sessão - Palestra

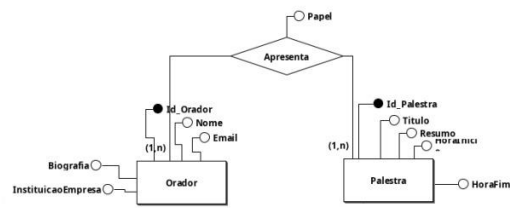


Figura 5.6 Ilustração do Relacionamento Orador - Palestra

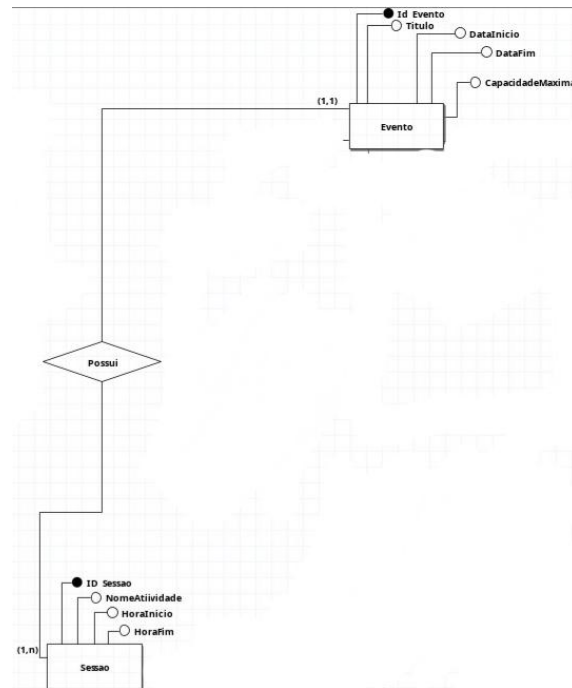


Figura 5.7 Ilustração do Relacionamento Evento - Sessão

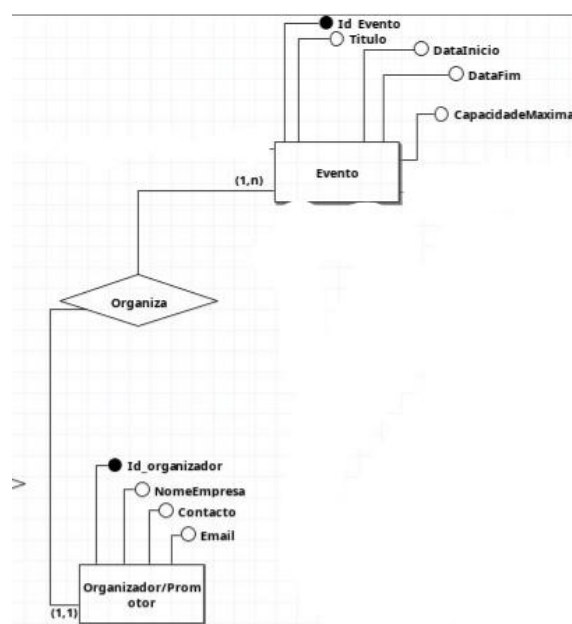


Figura 5.8 Ilustração do Relacionamento Organizador - Evento

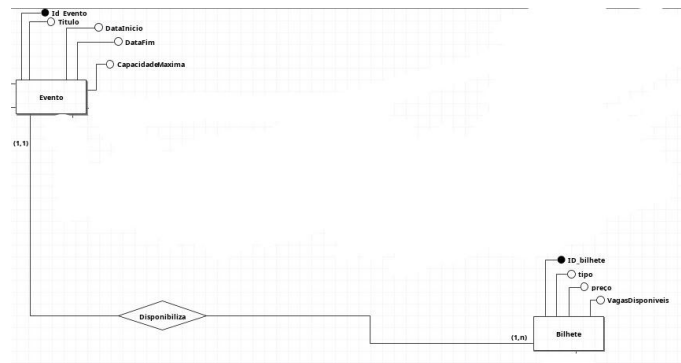


Figura 5.9 Ilustração do Relacionamento Evento - Bilhete

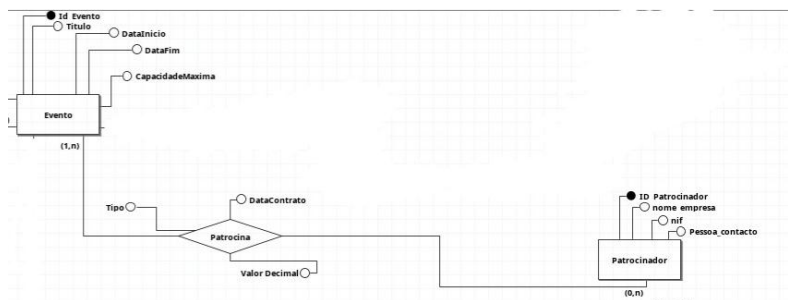


Figura 5.10 Ilustração do Relacionamento Evento - Patrocinador

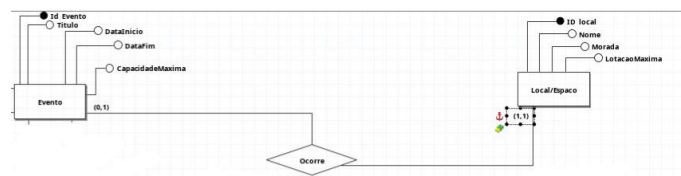


Figura 5.11 Ilustração do Relacionamento Evento - Local/Espaço

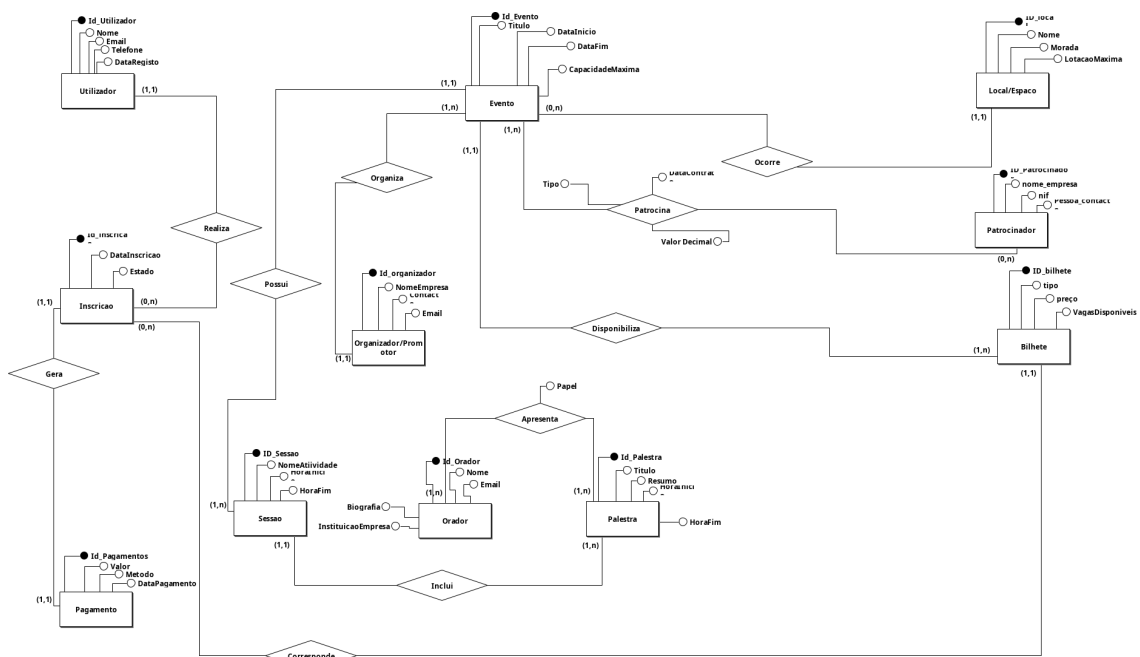


Figura 5.12 Ilustração do diagrama ER completo produzido

Modelação Lógica

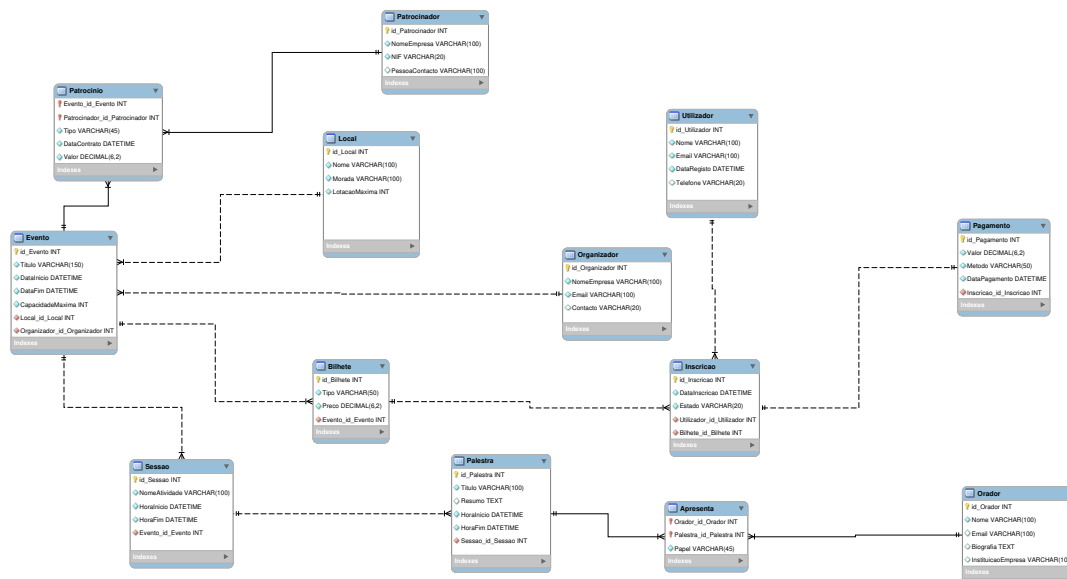


Figura 5.13 Ilustração do modelo lógico completo produzido no MySQL Workbench

Implementação Física (Criação de Base de Dados e Tabelas)

```
CREATE DATABASE IF NOT EXISTS Enei_Gest;
```

```
USE Enei_Gest;
```

Figura 5.14 Instruções SQL para criar e utilizar a base de dados

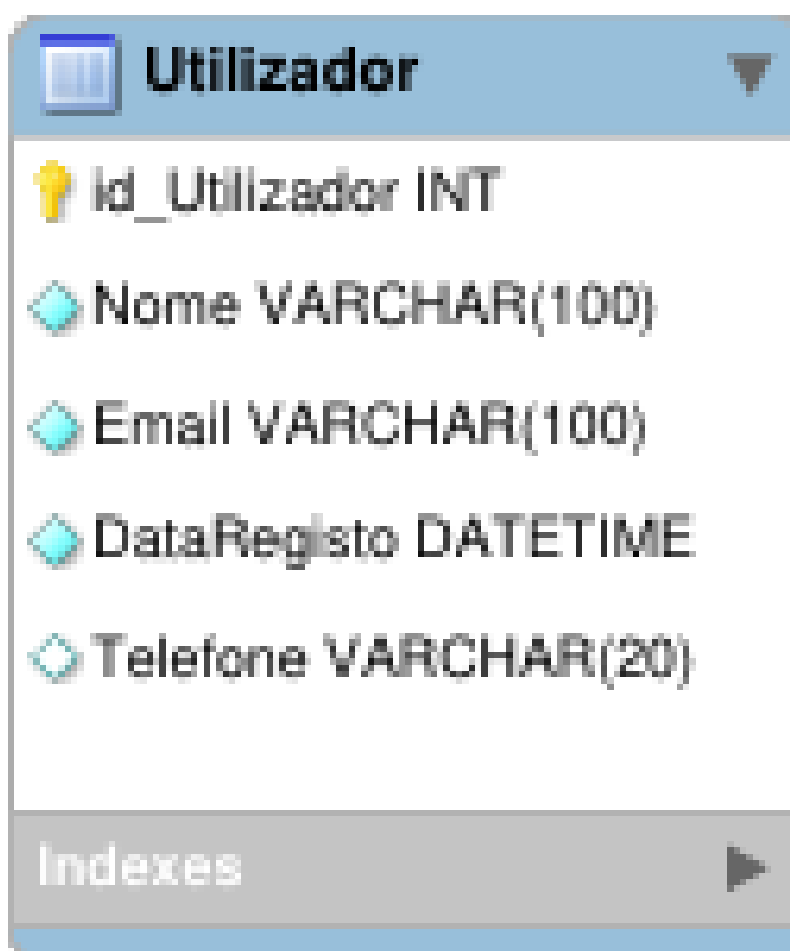


Figura 5.15 Tabela lógica do Utilizador

```

create table Utilizador(
    Id Int auto_increment,
    Nome varchar(100) not null,
    Email varchar(100) not null,
    DataRegisto Datetime not null,
    Telefone Varchar(20),
    primary key(Id)
);

```

Figura 5.16 Implementação correspondente ao Utilizador

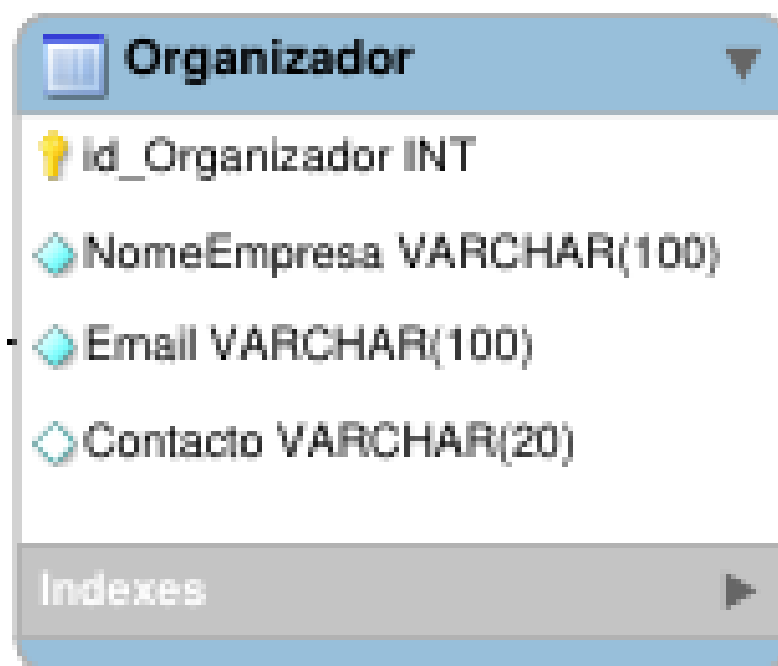


Figura 5.17 Tabela lógica do Organizador

```

create table Organizador(
    Id Int auto_increment,
    NomeEmpresa varchar(100) not null,
    Email varchar(100) not null,
    Contacto Varchar(20),
    primary key(Id)
);

```

Figura 5.18 Implementação correspondente ao Organizador

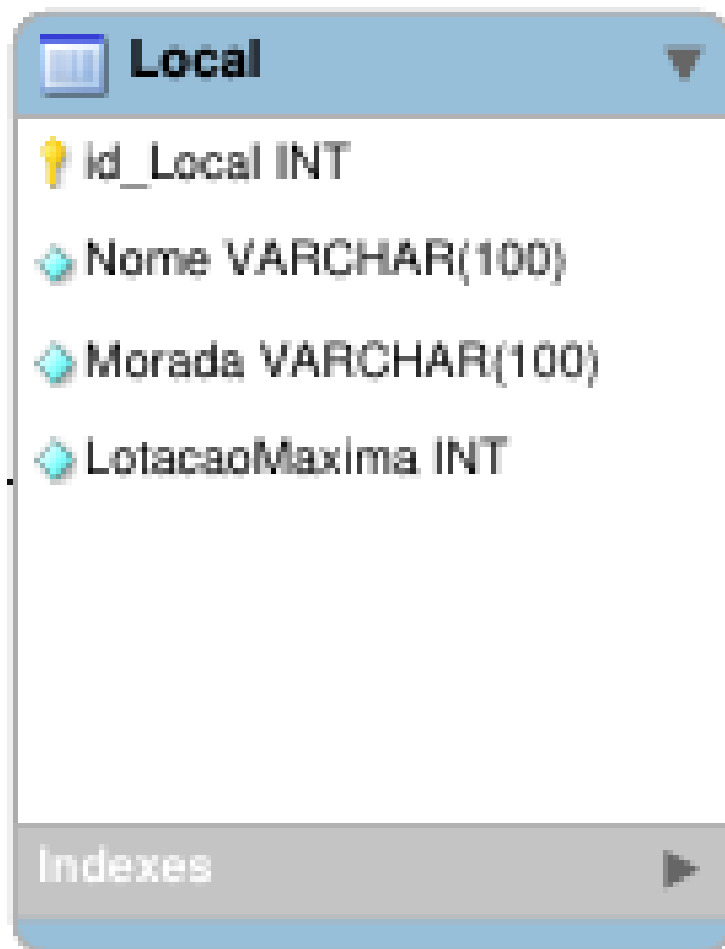


Figura 5.19 Tabela lógica do Local

```
create table `Local`(  
    Id Int auto_increment,  
    Nome varchar(100) not null,  
    Morada varchar(100) not null,  
    LotacaoMaxima Int not null,  
    primary key(Id)  
);
```

Figura 5.20 Implementação correspondente ao Local



Figura 5.21 Tabela lógica do Orador

```

create table Orador(
    Id Int auto_increment,
    Nome varchar(100) not null,
    Email varchar(100),
    Biografia Text,
    InstituicaoEmpresa varchar(100),
    primary key(Id)
);

```

Figura 5.22 Implementação correspondente ao Orador

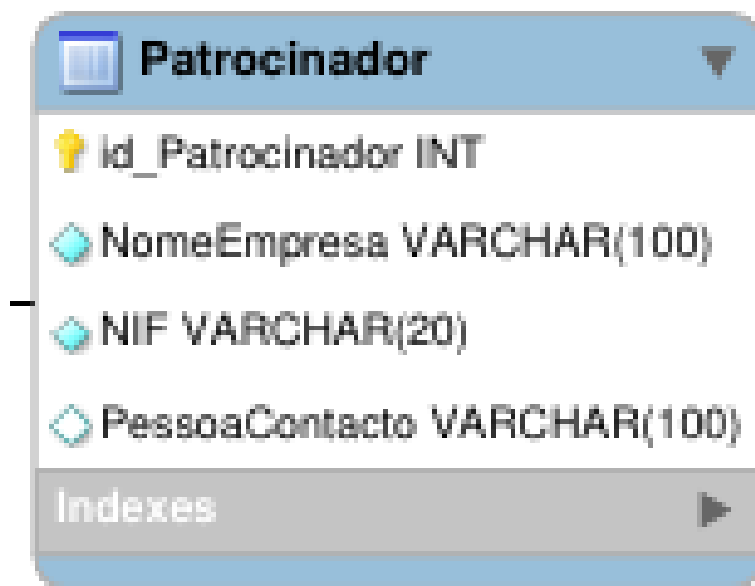


Figura 5.23 Tabela lógica do Patrocinador


```

CREATE TABLE Patrocinador (
    Id INT AUTO_INCREMENT,
    NomeEmpresa VARCHAR(100) NOT NULL,
    NIF VARCHAR(20) NOT NULL UNIQUE,
    PessoaContacto VARCHAR(100),
    PRIMARY KEY(Id)
);

```

Figura 5.24 Implementação correspondente ao Patrocinador



Figura 5.25 Tabela lógica do Evento

```

CREATE TABLE Evento (
    Id INT AUTO_INCREMENT,
    Titulo VARCHAR(150) NOT NULL,
    DataInicio DATETIME NOT NULL,
    DataFim DATETIME NOT NULL,
    CapacidadeMaxima INT NOT NULL,
    Organizador_id_Organizador INT NOT NULL,
    Local_id_Local INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Organizador_id_Organizador) REFERENCES Organizador(Id),
    FOREIGN KEY(Local_id_Local) REFERENCES Local(Id)
);

```

Figura 5.26 Implementação correspondente ao Evento



Figura 5.27 Tabela lógica da Sessão

```

CREATE TABLE Sessao (
    Id INT AUTO_INCREMENT,
    NomeAtividade VARCHAR(100) NOT NULL,
    HoraInicio DATETIME NOT NULL,
    HoraFim DATETIME NOT NULL,
    Evento_id_Evento INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Evento_id_Evento) REFERENCES Evento(Id)
);

```

Figura 5.28 Implementação correspondente à Sessão



Figura 5.29 Tabela lógica do Bilhete

```

CREATE TABLE Bilhete (
    Id INT AUTO_INCREMENT,
    Tipo VARCHAR(50) NOT NULL,
    Preco DECIMAL(6,2) NOT NULL,
    Evento_id_Evento INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Evento_id_Evento) REFERENCES Evento(Id)
);

```

Figura 5.30 Implementação correspondente ao Bilhete

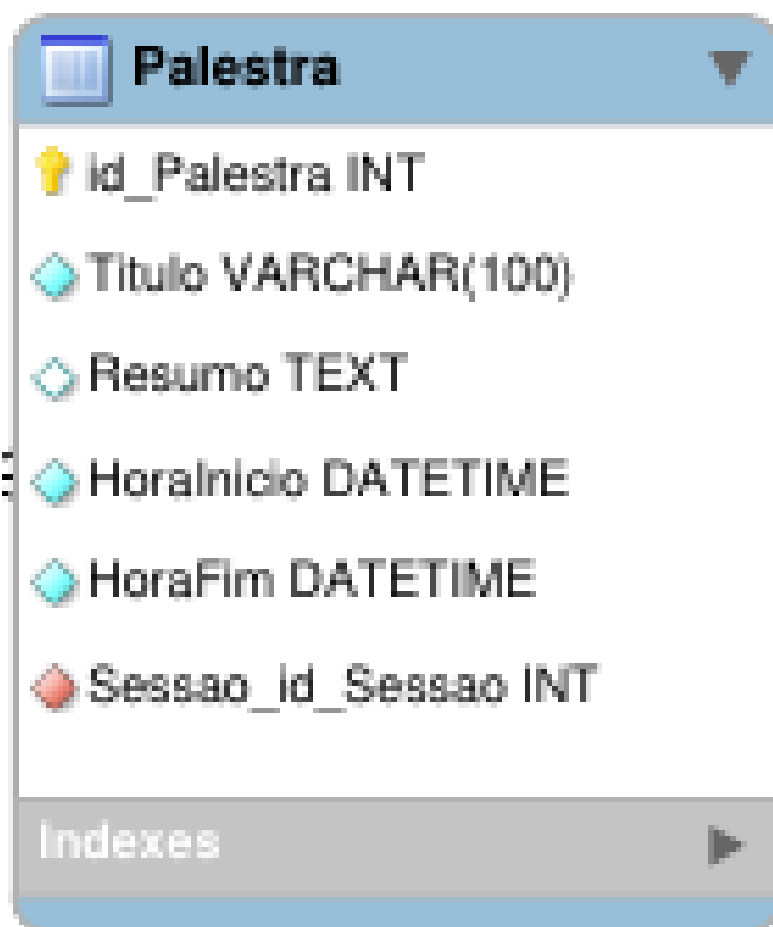


Figura 5.31 Tabela lógica da Palestra

```

CREATE TABLE Palestra (
    Id INT AUTO_INCREMENT,
    Titulo VARCHAR(100) NOT NULL,
    Resumo TEXT,
    HoraInicio DATETIME NOT NULL,
    HoraFim DATETIME NOT NULL,
    Sessao_id_Sessao INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Sessao_id_Sessao) REFERENCES Sessao(Id)
);

```

Figura 5.32 Implementação correspondente à Palestra



Figura 5.33 Tabela lógica da Inscrição

```

CREATE TABLE Inscricao (
    Id INT AUTO_INCREMENT,
    DataInscricao DATETIME NOT NULL,
    Estado VARCHAR(20) NOT NULL,
    Utilizador_id_Utilizador INT NOT NULL,
    Bilhete_id_Bilhete INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Utilizador_id_Utilizador) REFERENCES Utilizador(Id),
    FOREIGN KEY(Bilhete_id_Bilhete) REFERENCES Bilhete(Id)
);

```

Figura 5.34 Implementação correspondente à Inscrição



Figura 5.35 Tabela lógica do Pagamento

```
CREATE TABLE Pagamento (
    Id INT AUTO_INCREMENT,
    Valor DECIMAL(6,2) NOT NULL,
    Metodo VARCHAR(50) NOT NULL,
    DataPagamento DATETIME NOT NULL,
    Inscricao_id_Inscricao INT NOT NULL,
    PRIMARY KEY(Id),
    FOREIGN KEY(Inscricao_id_Inscricao) REFERENCES Inscricao(Id)
);
```

Figura 5.36 Implementação correspondente ao Pagamento

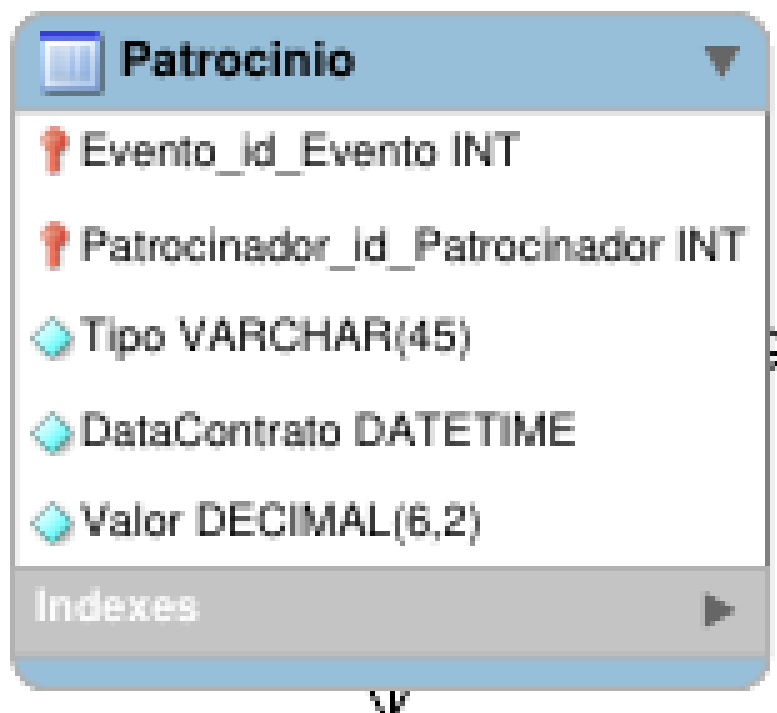


Figura 5.37 Tabela lógica do Patrocínio

```
CREATE TABLE Patrocinio (
    Evento_id_Evento INT NOT NULL,
    Patrocinador_id_Patrocinador INT NOT NULL,
    Tipo VARCHAR(45),
    DataContrato datetime,
    Valor Decimal(6,2),
    PRIMARY KEY(Evento_id_Evento, Patrocinador_id_Patrocinador),
    FOREIGN KEY(Evento_id_Evento) REFERENCES Evento(Id),
    FOREIGN KEY(Patrocinador_id_Patrocinador) REFERENCES Patrocinador(Id)
);
```

Figura 5.38 Implementação correspondente ao Patrocínio



Figura 5.39 Tabela lógica de Apresenta

```
CREATE TABLE Apresenta(  
    Orador_id_Orador INT NOT NULL,  
    Palestra_id_Palestra INT NOT NULL,  
    Papel VARCHAR(45),  
    PRIMARY KEY(Orador_id_Orador, Palestra_id_Palestra),  
    FOREIGN KEY(Orador_id_Orador) REFERENCES Orador(Id),  
    FOREIGN KEY(Palestra_id_Palestra) REFERENCES Palestra(Id)  
);
```

Figura 5.40 Implementação correspondente a Apresenta

Permissões e Povoamento de Dados

```
CREATE USER 'gestor_conteudos'@'localhost' IDENTIFIED BY 'Conteudos@123';

GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Evento TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Sessao TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Palestra TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Orador TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Local TO 'gestor_conteudos'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Organizador TO 'gestor_conteudos'@'localhost';
GRANT EXECUTE ON PROCEDURE Enei_Gest.GerarAgendaFinal TO 'gestor_conteudos'@'localhost';
```

Figura 5.41 Criação do utilizador "gestor_conteudos" e devidas permissões

```
CREATE USER 'gestor_participantes'@'localhost' IDENTIFIED BY 'Participantes@123';

GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Utilizador TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Inscricao TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Bilhete TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Pagamento TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Patrocinador TO 'gestor_participantes'@'localhost';
GRANT SELECT, INSERT, UPDATE ON Enei_Gest.Patrocinio TO 'gestor_participantes'@'localhost';
GRANT EXECUTE ON FUNCTION Enei_Gest.CalcularReceitaTotal TO 'gestor_participantes'@'localhost';
```

Figura 5.42 Criação do utilizador "gestor_participantes" e devidas permissões

```
INSERT INTO Utilizador (Nome, Email, DataRegisto, Telefone) VALUES
('João Silva', 'joao.silva@email.com', '2024-01-10 10:30:00', '912345678'),
('Ana Rita Santos', 'ana.santos@email.com', '2024-01-12 14:15:00', '961234567'),
('Carlos Mendes', 'carlos.m@estudante.uminho.pt', '2024-02-15 09:20:00', '933445566'),
('Beatriz Sousa', 'bsousa@estudante.feup.pt', '2024-02-18 11:45:00', '911223344'),
('Tiago Ribeiro', 'tiago.ribeiro@gmail.com', '2024-03-01 16:10:00', '966778899'),
('Mariana Lima', 'mariana.l@isep.ipp.pt', '2024-03-05 14:00:00', '922334455'),
('Pedro Gonçalves', 'pedro.goncalves@dev.com', '2024-03-10 18:30:00', NULL);
```

Figura 5.43 Inserção de dados na tabela Utilizador com SQL

```
INSERT INTO Organizador (NomeEmpresa, Email, Contacto) VALUES
('Tech Events Portugal', 'geral@techevents.pt', '210987654'),
('Cultura Porto', 'info@culturaporto.pt', '220123456'),
('Núcleo de Estudantes de Ciências da Computação', 'geral@necc.pt', '253112233'),
('Associação Académica', 'eventos@academica.pt', '210001122');
```

Figura 5.44 Inserção de dados na tabela Organizador com SQL

```
INSERT INTO `Local` (Nome, Morada, LotacaoMaxima) VALUES
('Auditório Principal FEUP', 'Porto', 500),
('Altice Arena', 'Lisboa', 2000),
('Pavilhão Rosa Mota', 'Jardins do Palácio de Cristal, Porto', 4000),
('Sala B204 - FEUP', 'Rua Dr. Roberto Frias, Porto', 120);
```

Figura 5.45 Inserção de dados na tabela Local com SQL

```
INSERT INTO Orador (Nome, Email, Biografia, InstituicaoEmpresa) VALUES
('Maria Fernandes', 'maria.f@tech.com', 'Especialista em IA.', 'TechCorp'),
('Rui Costa', 'rui.costa@univ.pt', 'Professor Catedrático.', 'Universidade do Porto'),
('Joana Batista', 'jbatista@microsoft.com', 'Cloud Architect e MVP.', 'Microsoft'),
('Nuno Marques', 'nuno.marques@blip.pt', 'Lead Software Engineer.', 'Blip'),
('Sara Oliveira', 'sara.o@deloitte.pt', 'Consultora de Cibersegurança.', 'Deloitte');
```

Figura 5.46 Inserção de dados na tabela Orador com SQL

```
INSERT INTO Patrocinador (NomeEmpresa, NIF, PessoaContacto) VALUES
('MEO Empresas', '500500500', 'Miguel Almeida'),
('Worten', '501501501', 'Sofia Lemos'),
('Microsoft Portugal', '502502502', 'Rita Lemos'),
('Deloitte', '503503503', 'Hugo Silva'),
('Blip', '504504504', 'Inês Castro');
```

Figura 5.47 Inserção de dados na tabela Patrocinador com SQL

```
INSERT INTO Evento (Titulo, DataInicio, DataFim, CapacidadeMaxima, Organizador_id_Organizador, Local_id_Local) VALUES
('Future Tech Conference', '2024-05-10 09:00:00', '2024-05-11 18:00:00', 400, 1, 1),
('Lisbon Cultural Fest', '2024-09-20 10:00:00', '2024-09-22 20:00:00', 1500, 2, 2),
('Hackathon Code4Good', '2024-11-15 18:00:00', '2024-11-17 18:00:00', 200, 3, 4),
('ENEI 2025 - Encontro Nacional', '2025-03-20 09:00:00', '2025-03-23 20:00:00', 3500, 1, 3);
```

Figura 5.48 Inserção de dados na tabela Evento com SQL

```
INSERT INTO Sessao (NomeAtividade, HoraInicio, HoraFim, Evento_id_Evento) VALUES
('Abertura e Keynotes', '2024-05-10 09:00:00', '2024-05-10 12:30:00', 1),
('Painel de Cibersegurança', '2025-03-21 14:00:00', '2025-03-21 16:00:00', 4),
('Workshops de Programação', '2025-03-22 10:00:00', '2025-03-22 13:00:00', 4),
('Apresentação de Projetos (Pitches)', '2024-11-17 15:00:00', '2024-11-17 17:30:00', 3);
```

Figura 5.49 Inserção de dados na tabela Sessao com SQL

```
INSERT INTO Bilhete (Tipo, Preco, Evento_id_Evento) VALUES
('Geral', 50.00, 1),
('VIP', 100.00, 1),
('Estudante', 15.00, 4),
('Profissional', 60.00, 4),
('Equipa Hackathon (4 pax)', 20.00, 3);
```

Figura 5.50 Inserção de dados na tabela Bilhete com SQL

```
INSERT INTO Palestra (Titulo, Resumo, HoraInicio, HoraFim, Sessao_id_Sessao) VALUES
('O Futuro da IA', 'Impacto da IA nas empresas.', '2024-05-10 10:00:00', '2024-05-10 11:00:00', 1),
('Ataques Ransomware em 2024', 'Análise dos maiores ataques e como prevenir.', '2025-03-21 14:15:00', '2025-03-21 15:00:00', 2),
('Escalar com Kubernetes', 'Workshop prático de DevOps e Cloud.', '2025-03-22 10:00:00', '2025-03-22 12:00:00', 3);
```

Figura 5.51 Inserção de dados na tabela Palestra com SQL

```
INSERT INTO Inscricao (DataInscricao, Estado, Utilizador_id_Utilizador, Bilhete_id_Bilhete) VALUES
('2024-02-01 10:00:00', 'Confirmada', 1, 1),
('2024-02-05 15:30:00', 'Pendente', 2, 2),
('2024-03-10 10:00:00', 'Confirmada', 3, 3),
('2024-03-15 14:20:00', 'Confirmada', 4, 3);
```

Figura 5.52 Inserção de dados na tabela Inscrição com SQL

```
INSERT INTO Pagamento (Valor, Metodo, DataPagamento, Inscricao_id_Inscricao) VALUES
(50.00, 'MBWay', '2024-02-01 10:05:00', 1),
(15.00, 'Referência Multibanco', '2024-03-10 10:15:00', 3),
(15.00, 'MBWay', '2024-03-15 14:25:00', 4);
```

Figura 5.53 Inserção de dados na tabela Pagamento com SQL

```
INSERT INTO Patrocinio (Evento_id_Evento, Patrocinador_id_Patrocinador, Tipo, DataContrato, Valor) VALUES
(1, 1, 'Principal', '2024-01-15 10:00:00', 5000.00),
(1, 2, 'Apoio Prata', '2024-02-10 14:00:00', 1500.00),
(4, 3, 'Sponsor Gold', '2025-01-10 11:00:00', 10000.00),
(4, 4, 'Sponsor Silver', '2025-01-15 15:30:00', 5000.00),
(3, 5, 'Apoio Tecnológico', '2024-09-01 10:00:00', 2000.00);
```

Figura 5.54 Inserção de dados na tabela Patrocínio com SQL

```
INSERT INTO Apresenta (Orador_id_Orador, Palestra_id_Palestra, Papel) VALUES
(1, 1, 'Orador Principal'),
(2, 1, 'Moderador'),
(5, 2, 'Orador Principal'),
(3, 3, 'Formador'),
(4, 3, 'Assistente');
```

Figura 5.55 Inserção de dados na tabela Apresenta com SQL

Vistas (Views) em SQL

```
CREATE VIEW AgendaPublica AS
SELECT
    e.Titulo AS NomeEvento,
    s.NomeAtividade AS NomeSessao,
    p.Titulo AS NomePalestra,
    p.HoraInicio,
    p.HoraFim,
    o.Nome AS NomeOrador,
    a.Papel
FROM Evento e
INNER JOIN Sessao s ON e.Id=s.Evento_id_Evento
INNER JOIN Palestra p ON s.Id= p.Sessao_id_Sessao
LEFT JOIN Apresenta a ON p.Id=a.Palestra_id_Palestra
LEFT JOIN Orador o ON a.Orador_id_Orador=o.Id;
```

Figura 5.56 Vista que demonstra a agenda pública consolidada com os respetivos oradores

```
CREATE VIEW DetalhesInscricoes AS
SELECT
    u.Nome AS Participante,
    u.Email,
    e.Titulo AS Evento,
    b.Tipo AS TipoBilhete,
    b.Preco AS PrecoBilhete,
    i.Estado AS EstadoInscricao,
    pg.Valor AS ValorPago,
    pg.Metodo AS MetodoPagamento
FROM Utilizador u
INNER JOIN Inscricao i ON u.Id = i.Utilizador_id_Utilizador
INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete=b.Id
INNER JOIN Evento e ON b.Evento_id_Evento=e.Id
LEFT JOIN Pagamento pg ON i.Id=pg.Inscricao_id_Inscricao;
```

Figura 5.57 Vista que demonstra os detalhes dos participantes, inscrições e pagamentos

```
SELECT * FROM DetalhesInscricoes;
```

Figura 5.58 Exemplo de utilização da vista "DetalhesInscricoes"

Tradução de Interrogações e Consultas em SQL

```
SET @Evento_ID = 4;  
SET @Orador_ID = 5;
```

Figura 5.59 Declaração dos IDs para utilização nas consultas

```
-- Lucro Total de um Evento bilhetes+patrocinios  
  
SELECT  
    e.Titulo AS Evento,  
    (SELECT SUM(Valor) FROM Pagamento p  
     INNER JOIN Inscricao i ON p.Inscricao_id_Inscricao=i.Id  
     INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete = b.Id  
     WHERE b.Evento_id_Evento = @Evento_ID) AS Total_Bilhetes,  
    (SELECT SUM(Valor) FROM Patrocinio WHERE Evento_id_Evento = @Evento_ID) AS Total_Patrocinios,  
    (SELECT SUM(Valor) FROM Pagamento p  
     INNER JOIN Inscricao i ON p.Inscricao_id_Inscricao = i.Id  
     INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete = b.Id  
     WHERE b.Evento_id_Evento = @Evento_ID) + (SELECT SUM(Valor) FROM Patrocinio WHERE Evento_id_Evento = @Evento_ID) AS Receita_Total  
FROM Evento e  
WHERE e.Id = @Evento_ID;
```

Figura 5.60 Cálculo do lucro total de um evento com SQL

```
-- Listar as palestras de um orador  
  
SELECT  
    o.Nome AS Orador,  
    p.Titulo AS Tema_Palestra,  
    p.HoraInicio,  
    a.Papel  
FROM Orador o  
INNER JOIN Apresenta a ON o.Id = a.Orador_id_Orador  
INNER JOIN Palestra p ON a.Palestra_id_Palestra = p.Id  
WHERE o.Id = @Orador_ID  
ORDER BY p.HoraInicio ASC;
```

Figura 5.61 Listar as palestras de um orador com SQL

```
-- Contagem de inscrições Confirmadas por Evento  
  
SELECT  
    e.Titulo AS Evento,  
    COUNT(i.Id) AS Bilhetes_Vendidos  
FROM Evento e  
INNER JOIN Bilhete b ON e.Id = b.Evento_id_Evento  
INNER JOIN Inscricao i ON b.Id = i.Bilhete_id_Bilhete  
WHERE i.Estado = 'Confirmada'  
GROUP BY e.Titulo  
ORDER BY Bilhetes_Vendidos DESC;
```

Figura 5.62 Contagem de inscrições confirmadas por evento com SQL

```
-- Verificar o metodo de pagamento mais popular  
  
SELECT  
    Metodo AS Metodo_Pagamento,  
    COUNT(id) AS Quantidade_Utilizacoes,  
    SUM(Valor) AS Volume_Euros  
FROM Pagamento  
GROUP BY Metodo  
ORDER BY Quantidade_Utilizacoes DESC  
LIMIT 1;
```

Figura 5.63 Verificar o método de pagamento mais popular com SQL

```
-- Listar utilizadores com inscricoes pendentes

SELECT
    u.Nome,
    u.Email,
    b.Tipo AS Tipo Bilhete,
    b.Preco AS Valor_Em_Divida
FROM Utilizador u
INNER JOIN Inscricao i ON u.Id = i.Utilizador_id_Utilizador
INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete=b.Id
LEFT JOIN Pagamento p ON i.Id = p.Inscricao_id_Inscricao
WHERE i.Estado = 'Pendente' AND p.Id IS NULL;
```

Figura 5.64 Listar utilizadores com inscrições pendentes com SQL

```
-- TOP 3 oradores com mais participantes ativos

SELECT
    o.Nome AS Orador,
    o.InstituicaoEmpresa AS Empresa,
    COUNT(a.Palestra_id_Palestra) AS Total_Participantes
FROM Orador o
INNER JOIN Apresenta a ON o.Id=a.Orador_id_Orador
GROUP BY o.Id, o.Nome, o.InstituicaoEmpresa
ORDER BY Total_Participacoes DESC
LIMIT 3;
```

Figura 5.65 Listar os top 3 oradores mais ativos com SQL

Indexação do Sistema, Funções e Procedimentos

```
CREATE INDEX idx_inscricao_utilizador_id ON Inscricao(Utilizador_id_Utilizador);  
CREATE INDEX idx_inscricao_bilhete_id ON Inscricao(Bilhete_id_Bilhete);
```

Figura 5.66 Criação de índices para a tabela Inscrição

```
CREATE INDEX idx_palestra_sessao_id ON Palestra(Sessao_id_Sessao);
```

Figura 5.67 Criação de índice para a tabela Palestra

```
CREATE INDEX idx_sessao_evento_id ON Sessao(Evento_id_Evento);
```

Figura 5.68 Criação de índice para a tabela Sessão

```
DELIMITER $$  
CREATE FUNCTION CalcularReceitaTotal(p_Evento_ID INT)  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    DECLARE total_bilhetes DECIMAL(10,2);  
    DECLARE total_patrocinios DECIMAL(10,2);  
  
    SELECT IFNULL(SUM(p.Valor), 0) INTO total_bilhetes  
    FROM Pagamento p  
    INNER JOIN Inscricao i ON p.Inscricao_id_Inscricao = i.Id  
    INNER JOIN Bilhete b ON i.Bilhete_id_Bilhete = b.Id  
    WHERE b.Evento_id_Evento = p_Evento_ID;  
  
    SELECT IFNULL(SUM(Valor), 0) INTO total_patrocinios  
    FROM Patrocinio  
    WHERE Evento_id_Evento = p_Evento_ID;  
  
    RETURN total_bilhetes + total_patrocinios;  
END $$  
DELIMITER ;
```

Figura 5.69 Função que calcula a receita total (bilheteira e patrocínios) de um evento

```

DELIMITER $$
CREATE TRIGGER AtualizarEstadoPagamento
AFTER INSERT ON Pagamento
FOR EACH ROW
BEGIN

    UPDATE Inscricao
    SET Estado = 'Confirmada'
    WHERE Id = NEW.Inscricao_id_Inscricao;
END $$
DELIMITER ;

```

Figura 5.70 Gatilho responsável por atualizar o estado da inscrição após o pagamento

```

DELIMITER $$
CREATE PROCEDURE GerarAgendaFinal(IN p_Evento_ID INT)
BEGIN
    SELECT
        s.NomeAtividade AS Sessao,
        p.HoraInicio,
        p.HoraFim,
        p.Titulo AS Palestra,
        IFNULL(o.Nome, 'Orador a definir') AS Orador
    FROM Sessao s
    INNER JOIN Palestra p ON s.Id = p.Sessao_id_Sessao
    LEFT JOIN Apresenta a ON p.Id = a.Palestra_id_Palestra
    LEFT JOIN Orador o ON a.Orador_id_Orador = o.Id
    WHERE s.Evento_id_Evento = p_Evento_ID
    ORDER BY p.HoraInicio ASC;
END $$
DELIMITER ;

```

Figura 5.71 Procedimento para gerar a agenda final e ordenada de um evento